

第3章 组合逻辑电路

3.1 组合逻辑电路的分析与设计方法

3.2 加法器和数值比较器

3.3 编码器和译码器

3.4 数据选择器和分配器

3.5 只读存储器 (ROM)

3.6 奇偶校验器

3.7 组合电路中的竞争冒险

3.1 组合逻辑电路的分析与设计方法

3.1.1 组合逻辑电路的分析方法

3.1.2 组合逻辑电路的设计方法

3.1.1 组合逻辑电路的分析方法

组合逻辑电路：输出仅由输入决定，与电路当前状态无关；电路结构中**无**反馈环路（无记忆）。



一般表示为：

$$\begin{aligned} Y_0 &= F_0(I_0, I_1, \dots, I_{n-1}) \\ Y_1 &= F_1(I_0, I_1, \dots, I_{n-1}) \\ &\dots \\ Y_{m-1} &= F_{m-1}(I_0, I_1, \dots, I_{n-1}) \end{aligned}$$

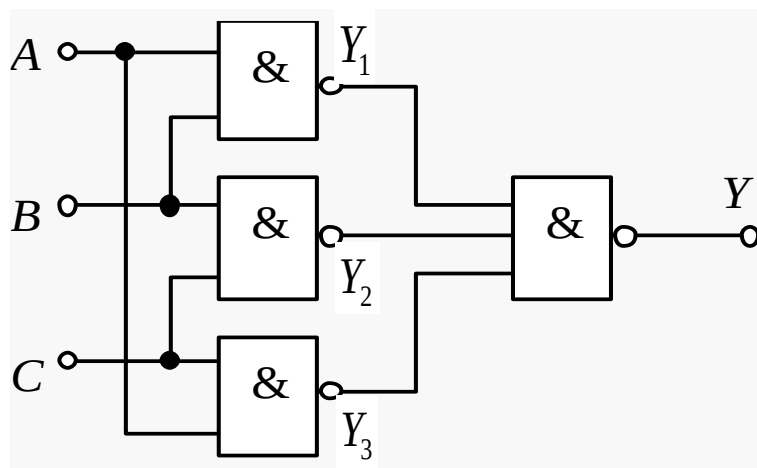
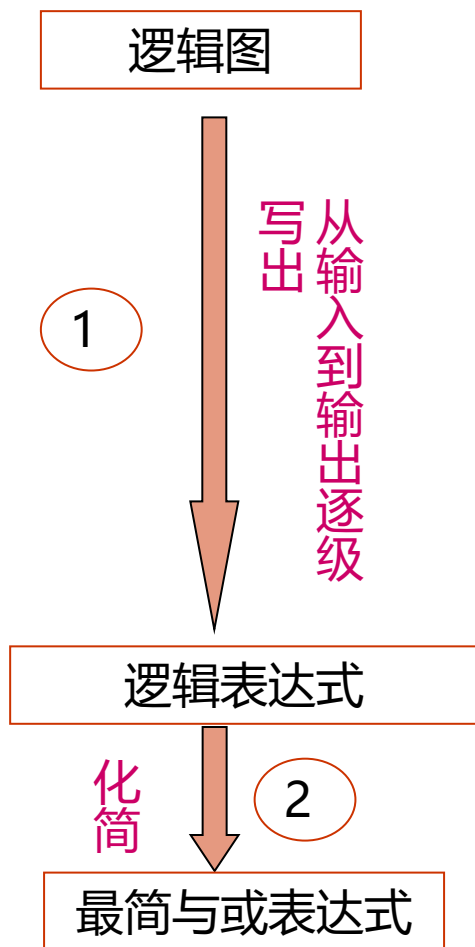
向量形式为： $Y(t_n) = F[I(t_n)]$

t_n 时刻的稳定输出 $Y(t_n)$,仅取决于 t_n 时刻的输入 $I(t_n)$, $Y(t_n)$ 与 $I(t_n)$ 的函数关系用 $F[I(t_n)]$ 表示。

组合逻辑电路的分析方法

- **电路分析**：已知逻辑电路，寻找输出与输入之间的逻辑关系，确定电路功能的过程。
- 组合逻辑电路的**分析步骤**：
 - 根据给定的逻辑图，从输入到输出逐级写出逻辑函数表达式；
 - 用公式法或卡诺图法化简逻辑函数表达式；
 - 概括出给定组合逻辑电路的逻辑功能。

组合逻辑电路的分析



1

$$Y_1 = \overline{AB}$$

$$Y_2 = \overline{BC}$$

$$Y_3 = \overline{CA}$$

$$Y = \overline{Y_1 Y_2 Y_3} = \overline{\overline{AB} \overline{BC} \overline{AC}}$$

2

$$Y = AB + BC + CA$$

最简与或表达式

3

真值表

4

说明电路的功能

$$Y = AB + BC + CA$$

3

A	B	C	Y
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

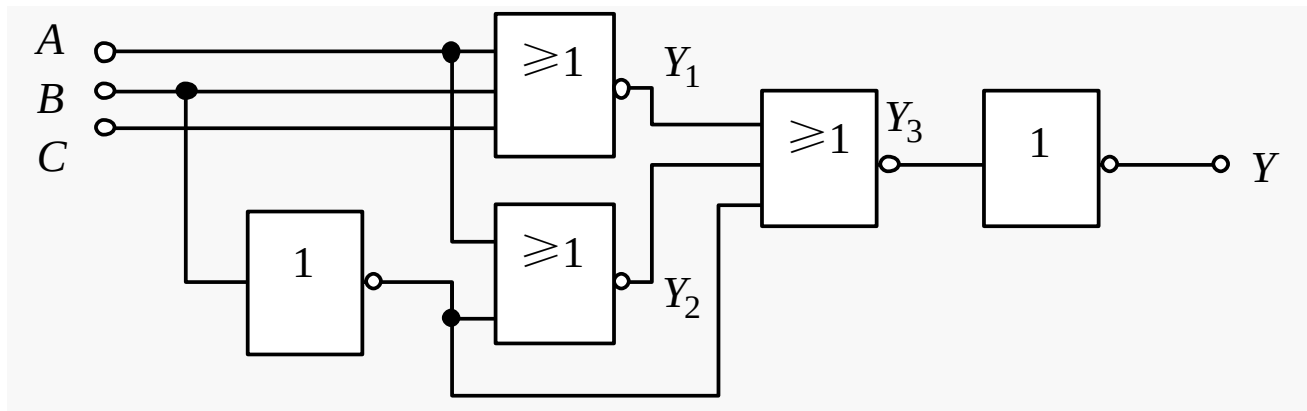
4

当输入A、B、C中有2个或3个为1时，输出Y为1，否则输出Y为0。

电路可以看做是一种3人表决用的组合电路：只要有2票或3票同意，表决就通过。

例:

逻辑图



逻辑表达式

$$\left. \begin{aligned} Y_1 &= \overline{A + B + C} \\ Y_2 &= \overline{A + \overline{B}} \\ Y_3 &= \overline{Y_1 + Y_2 + \overline{B}} \end{aligned} \right\} Y = \overline{Y_3} = Y_1 + Y_2 + \overline{B} = \overline{A + B + C} + \overline{A + \overline{B}} + \overline{B}$$

最简与或表达式

$$Y = \overline{A}\overline{B}\overline{C} + \overline{A}B + \overline{B} = \overline{A}B + \overline{B} = \overline{A} + \overline{B}$$

真值表

<i>A</i>	<i>B</i>	<i>C</i>	<i>Y</i>
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	1
1	0	0	1
1	0	1	1
1	1	0	0
1	1	1	0

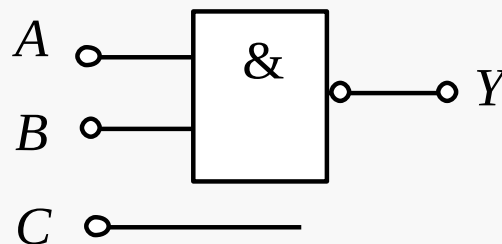
功能说明

输出Y和输入的逻辑关系为：A、B中只要一个为0，Y=1；A、B全为1时，Y=0。

电路的输出Y只与输入A、B有关，而与输入C无关。Y和A、B的逻辑关系为与非运算的关系。

用与非门实现

$$Y = \bar{A} + \bar{B} = \overline{AB}$$



3.1.2 组合逻辑电路的设计方法

- **电路设计**：根据所要求实现的逻辑功能，设计出相应的逻辑电路。
- 通常以电路简单、所用器件最少为目标。
- **设计步骤**：
 1. 分析设计要求，对问题逻辑抽象，确定输入、输出变量。
 2. 对输入和输出变量赋予0、1值，根据输入输出之间的逻辑关系，列出真值表。
 - 一个特定的逻辑问题，其对应的真值表是唯一的。
 3. 根据真值表写出逻辑表达式、卡诺图，并根据设计要求进行化简和变换。
 - 设计时，如果可用的门电路部件有要求，可以通过变换逻辑表达式改变电路设计。
 4. 根据化简式画出逻辑电路图。

组合逻辑电路的设计

例：设计一个楼上、楼下双联开关控制楼梯上的灯，使之在上楼前，可以用楼下开关打开电灯，上楼后，用楼上开关关灭电灯；或者在下楼前，用楼上开关打开电灯，下楼后，用楼下开关关灭电灯。

逻辑抽象

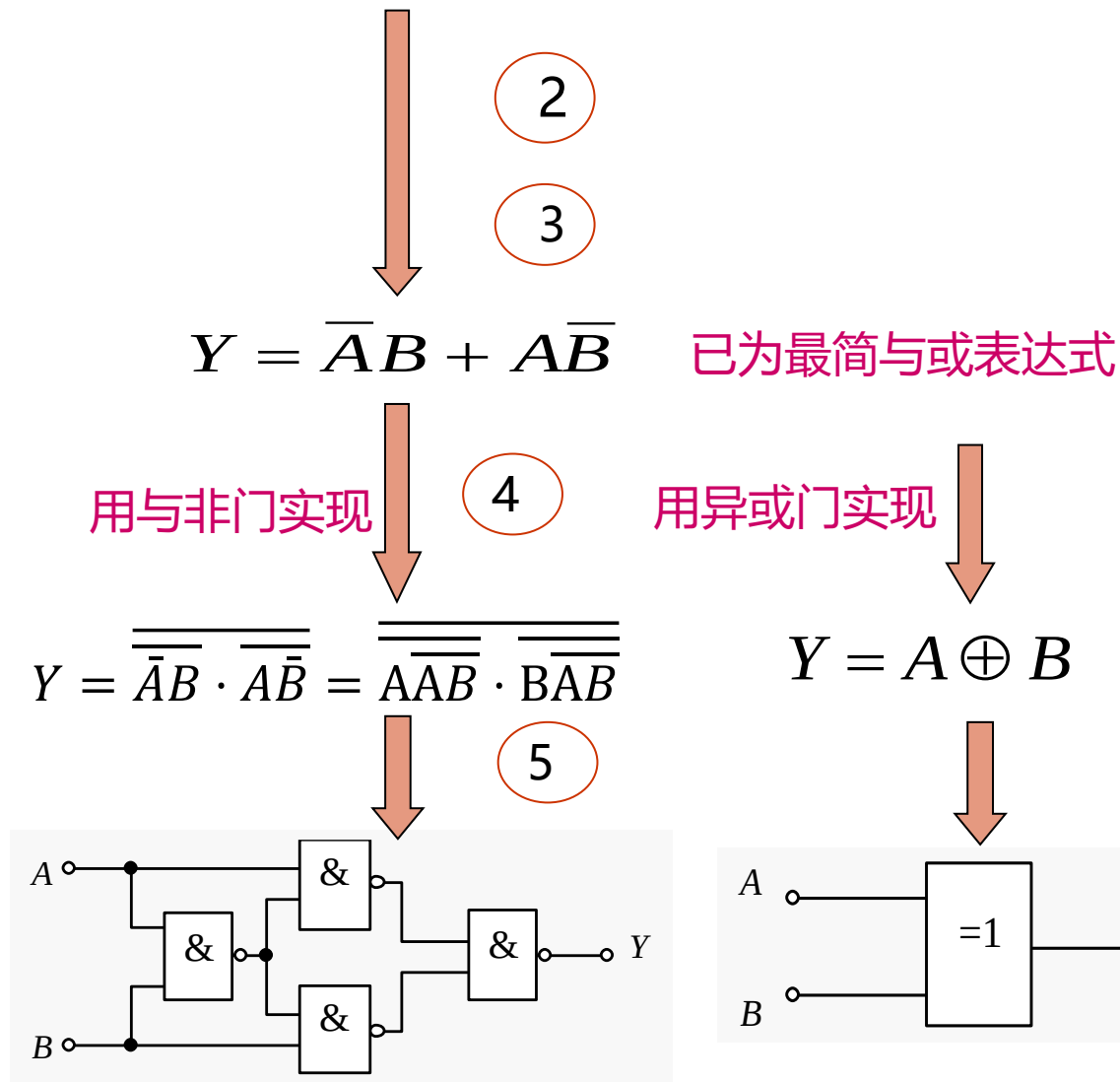
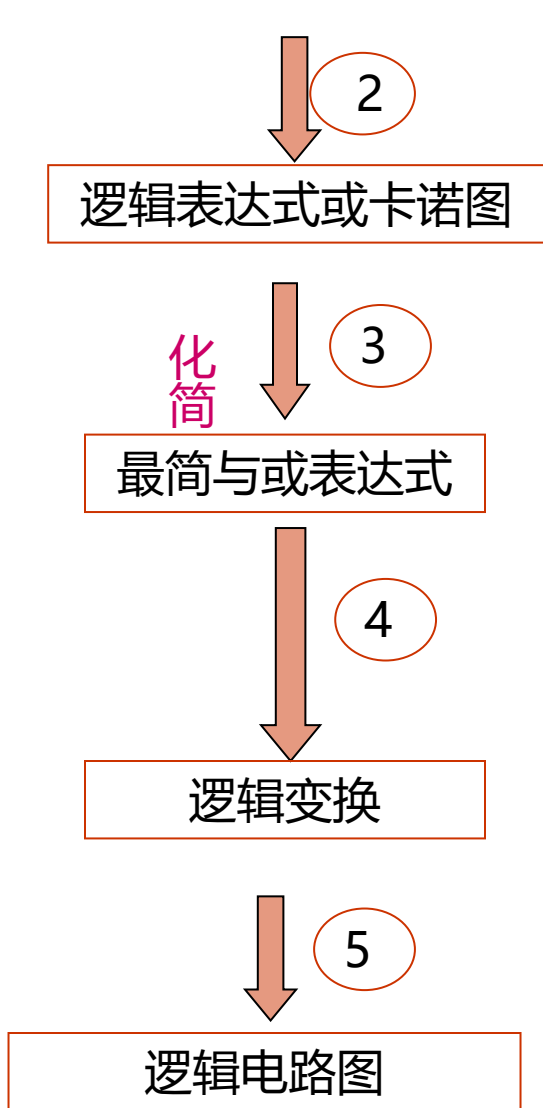
1

真值表

设楼上开关为A，楼下开关为B，灯泡为Y。并设A、B闭合时为1，断开时为0；灯亮时Y为1，灯灭时Y为0。

1

A	B	Y
0	0	0
0	1	1
1	0	1
1	1	0



例：用与非门设计一个举重裁判表决电路。设举重比赛有3个裁判，一个主裁判和两个副裁判。杠铃成功举起的裁决由每一个裁判按一下自己面前的按钮来确定。只有当两个或两个以上裁判判明成功，并且其中有一个为主裁判时，表明成功的灯才亮。

逻辑抽象

1

真值表

2

逻辑表达式

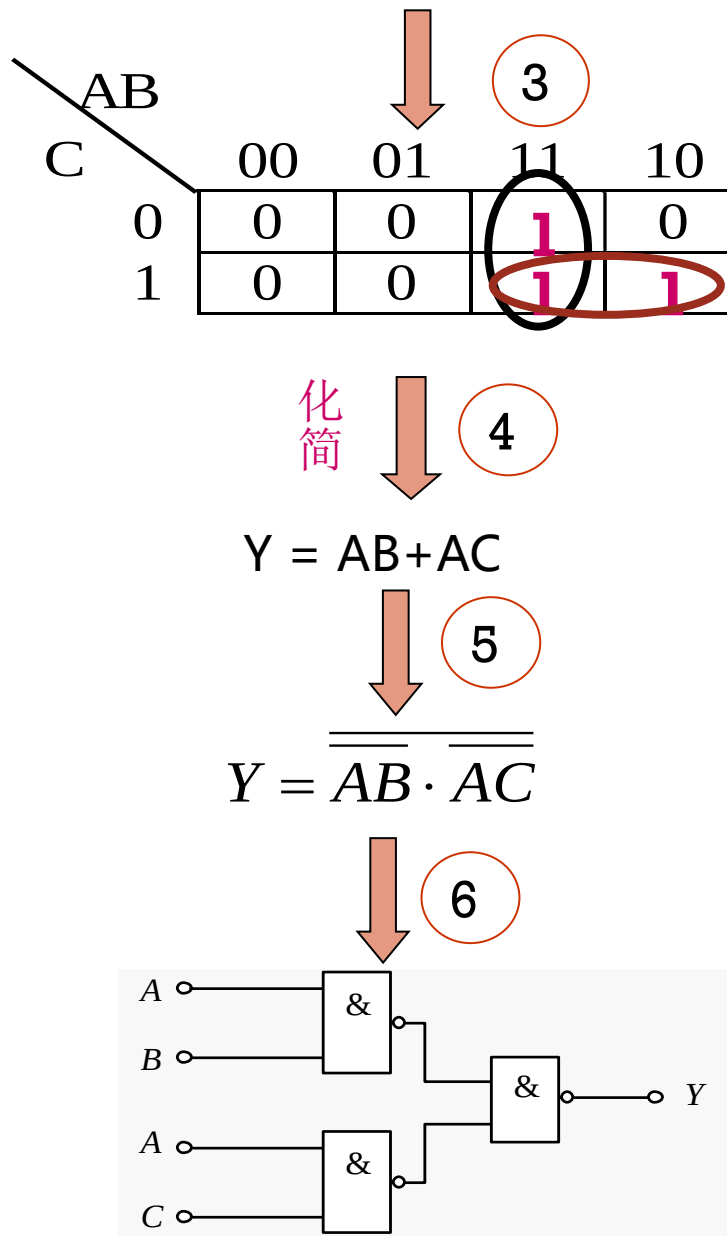
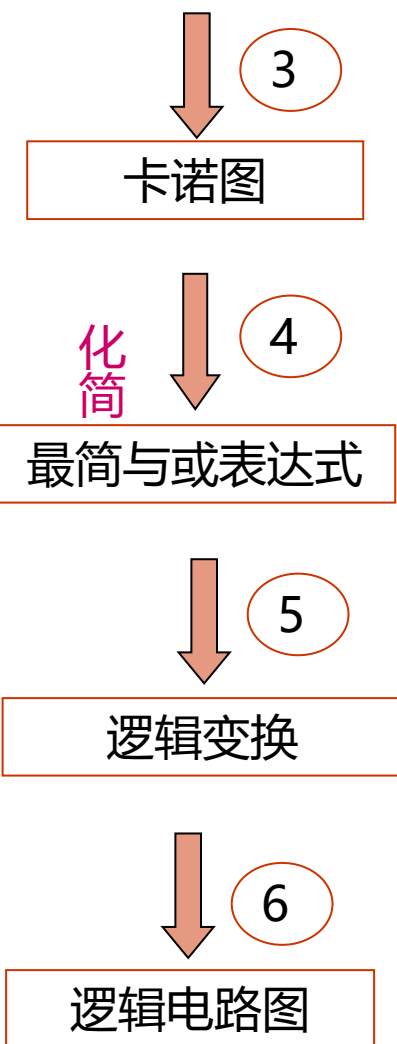
设主裁判为变量A，副裁判分别为B和C；表示成功与否的灯为Y。

1

A	B	C	Y	A	B	C	Y
0	0	0	0	1	0	0	0
0	0	1	0	1	0	1	1
0	1	0	0	1	1	0	1
0	1	1	0	1	1	1	1

2

$$Y = m_5 + m_6 + m_7 = \overline{A}BC + A\overline{B}\overline{C} + ABC$$



小 结

①组合电路的特点：在任何时刻的输出只取决于当时的输入信号，而与电路原来所处的状态无关。实现组合电路的基础是逻辑代数和门电路。

②组合电路的逻辑功能可用逻辑图、真值表、逻辑表达式、卡诺图和波形图等5种方法来描述，它们在本质上是相通的，可以互相转换。

③组合电路的分析方法：根据给出的逻辑图写出输出函数的逻辑表达式→化简后求出最简与或表达式→列出真值表→根据真值表说明电路功能。

④组合电路的设计步骤：进行逻辑抽象→列出真值表→写出逻辑表达式→逻辑表达式化简(最简与或式)→逻辑变换→画出逻辑电路图。

3.2 加法器和数值比较器

3.2.1 加法器

3.2.2 数值比较器

3.2.1 加法器

- 半加器：能对两个1位二进制数进行相加，而求得和及进位的逻辑电路

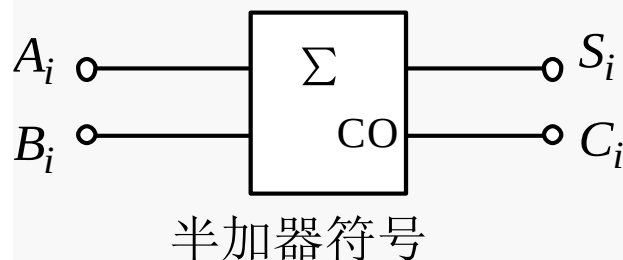
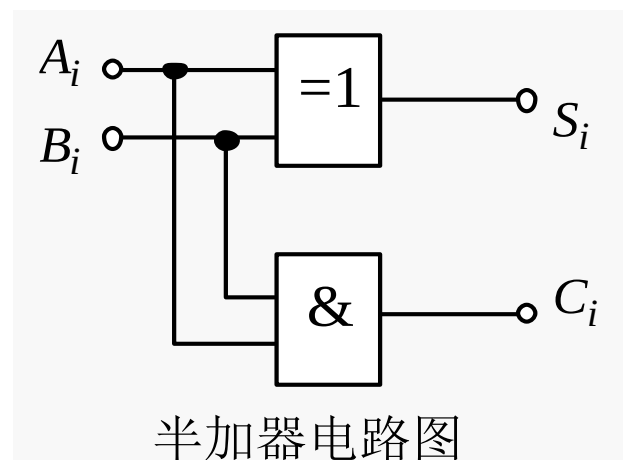
半加器真值表

加数		本位的和	
A_i	B_i	S_i	C_i
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

向高位的进位

$$S_i = \bar{A}_i B_i + A_i \bar{B}_i = A_i \oplus B_i$$

$$C_i = A_i B_i$$



- 全加器：能对两个1位二进制数进行相加并考虑低位来的进位，即相当于3个1位二进制数相加，求得和及进位的逻辑电路

A_i	B_i	C_{i-1}	S_i	C_i
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

A_i, B_i : 加数, C_{i-1} : 低位来的进位,
 S_i : 本位的和, C_i : 向高位的进位。

		$A_i B_i$			
		00	01	11	10
C_{i-1}	0	0	1	0	1
	1	1	0	1	0

S_i 的卡诺图

$$S_i = m_1 + m_2 + m_4 + m_7$$

		$A_i B_i$			
		00	01	11	10
C_{i-1}	0	0	0	1	0
	1	0	1	1	1

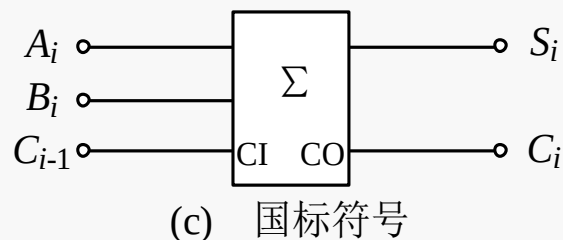
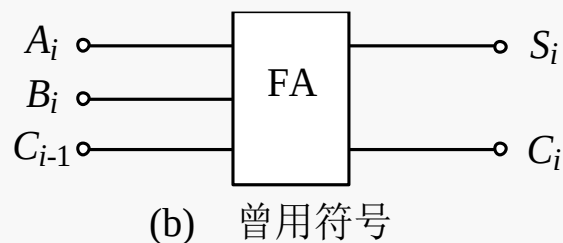
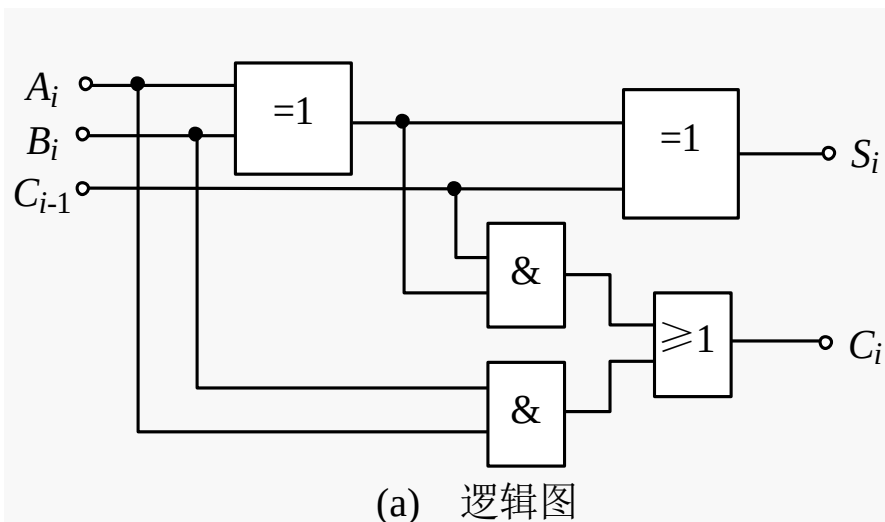
C_i 的卡诺图

$$C_i = A_i B_i + A_i C_{i-1} + B_i C_{i-1}$$

$$C_i = A_i B_i + m_5 + m_3$$

全加器的逻辑图和逻辑符号

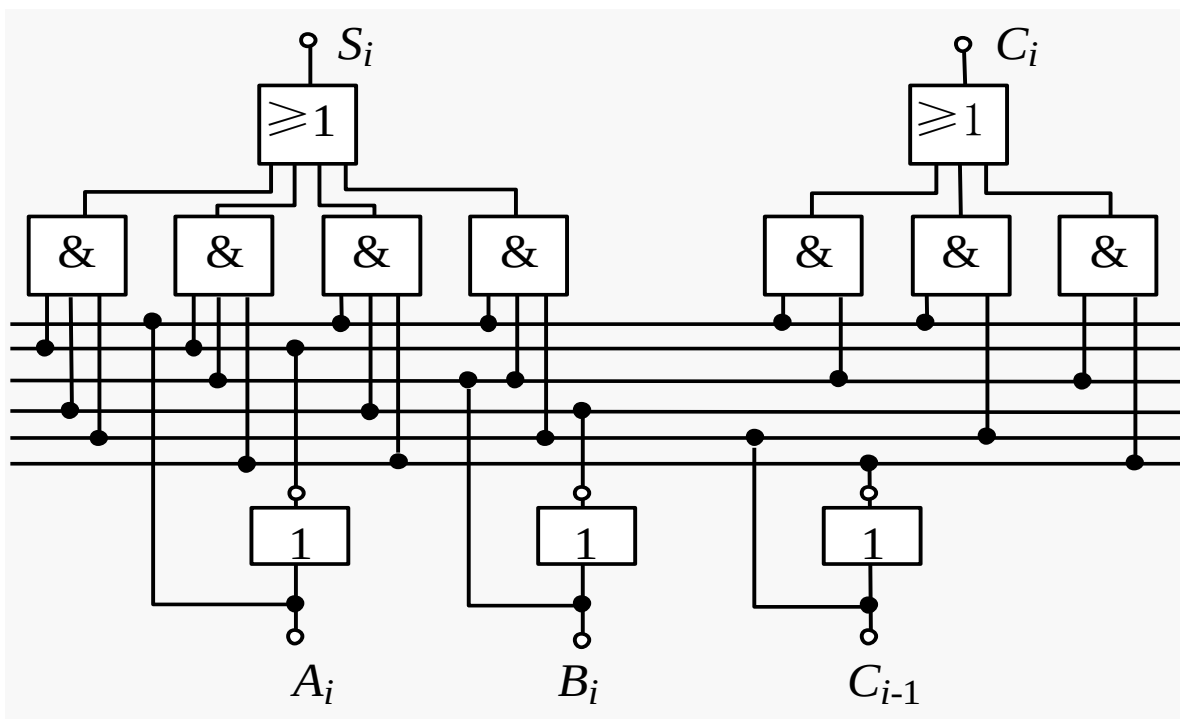
$$\begin{aligned}
 S_i &= m_1 + m_2 + m_4 + m_7 = \bar{A}_i \bar{B}_i C_{i-1} + \bar{A}_i B_i \bar{C}_{i-1} + A_i \bar{B}_i \bar{C}_{i-1} + A_i B_i C_{i-1} \\
 &= \bar{A}_i (\bar{B}_i C_{i-1} + B_i \bar{C}_{i-1}) + A_i (\bar{B}_i \bar{C}_{i-1} + B_i C_{i-1}) = \bar{A}_i (B_i \oplus C_{i-1}) + A_i \overline{(B_i \oplus C_{i-1})} \\
 &= A_i \oplus B_i \oplus C_{i-1} \\
 C_i &= m_3 + m_5 + A_i B_i = \bar{A}_i B_i C_{i-1} + A_i \bar{B}_i C_{i-1} + A_i B_i = (\bar{A}_i B_i + A_i \bar{B}_i) C_{i-1} + A_i B_i \\
 &= (A_i \oplus B_i) C_{i-1} + A_i B_i
 \end{aligned}$$



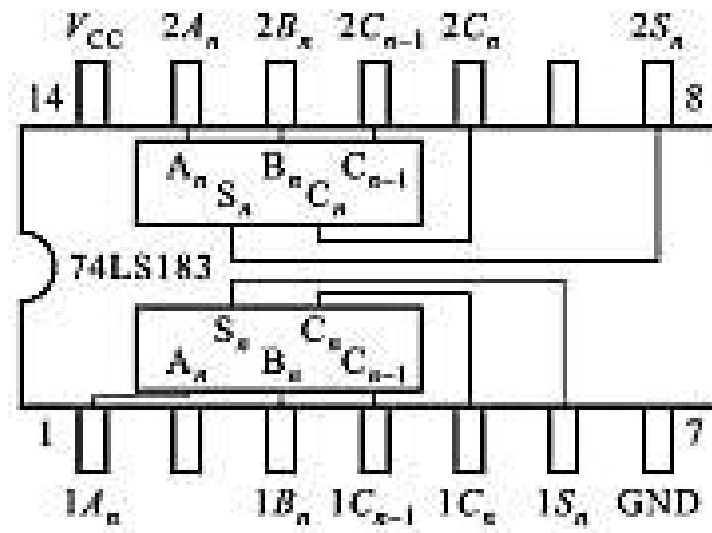
用与门和或门实现

$$S_i = \overline{A_i} \overline{B_i} C_{i-1} + \overline{A_i} B_i \overline{C_{i-1}} + A_i \overline{B_i} \overline{C_{i-1}} + A_i B_i C_{i-1}$$

$$C_i = A_i B_i + A_i C_{i-1} + B_i C_{i-1}$$



集成全加器

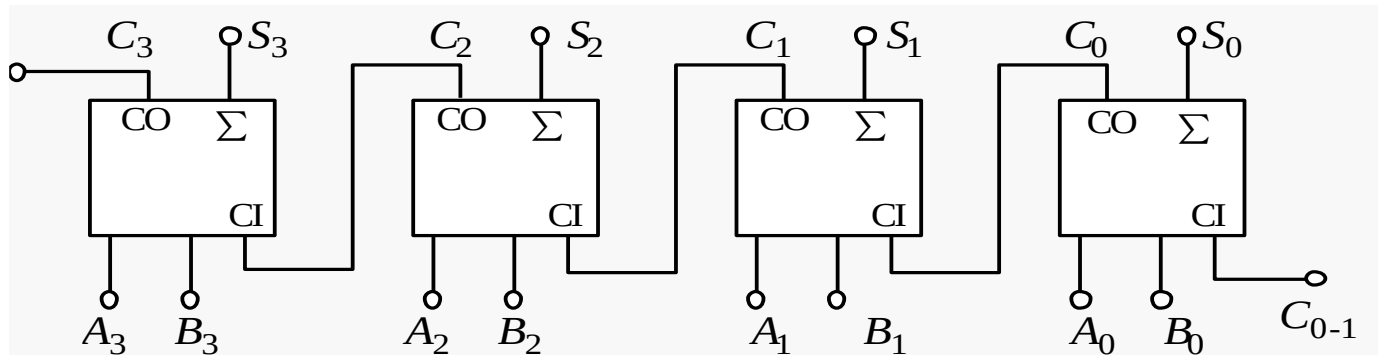


加法器

- 加法器：实现多位二进制数相加的电路。

4位串行进位加法器

构成：把 4 个全加器串联起来，低位全加器的进位输出连接到相邻的高位全加器的进位输入。



特点：进位信号是由低位向高位逐级传递的，速度不高。

超前进位加法器（并行进位加法器）

超前进位加法器: 在作加法运算时, **进位输入信号由二进制加数直接产生**, 而不需要等待低位加数相加后的进位, 这样的加法器称之为超前进位加法器。

以4位二进制加法器为例, 全加器的进位输入信号 C_i 的表达式为:

$$\begin{aligned}C_i &= m_3 + m_5 + A_i B_i = \bar{A}_i B_i C_{i-1} + A_i \bar{B}_i C_{i-1} + A_i B_i \\&= (\bar{A}_i B_i + A_i \bar{B}_i) C_{i-1} + A_i B_i = (A_i \oplus B_i) C_{i-1} + A_i B_i\end{aligned}$$

进位生成项

$$G_i = A_i B_i$$

低位进位传递条件

$$P_i = A_i \oplus B_i$$

进位表达式

$$C_i = A_i B_i + (A_i \oplus B_i) C_{i-1} = G_i + P_i C_{i-1}$$

和表达式

$$S_i = A_i \oplus B_i \oplus C_{i-1} = P_i \oplus C_{i-1}$$

进位生成项

$$G_i = A_i B_i$$

低位进位传递条件

$$P_i = A_i \oplus B_i$$

进位表达式

$$C_i = A_i B_i + (A_i \oplus B_i) C_{i-1} = G_i + P_i C_{i-1}$$

和表达式

$$S_i = A_i \oplus B_i \oplus C_{i-1} = P_i \oplus C_{i-1}$$

$$\begin{cases} S_0 = P_0 \oplus C_{0-1} \\ C_0 = G_0 + P_0 C_{0-1} \end{cases} \quad \text{4位超前进位加法器递推公式}$$
$$\begin{cases} S_1 = P_1 \oplus C_0 \\ C_1 = G_1 + P_1 C_0 = G_1 + P_1 G_0 + P_1 P_0 C_{0-1} \end{cases}$$
$$\begin{cases} S_2 = P_2 \oplus C_1 \\ C_2 = G_2 + P_2 C_1 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_{0-1} \end{cases}$$
$$\begin{cases} S_3 = P_3 \oplus C_2 \\ C_3 = G_3 + P_3 C_2 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_{0-1} \end{cases}$$

- 只要加数和最低位进位 C_{0-1} 给定，可以直接确定各进位 $C_3 C_2 C_1 C_0$ 的值。
- 高位的运算不需再等待低位的进位，极大提高运算速度。

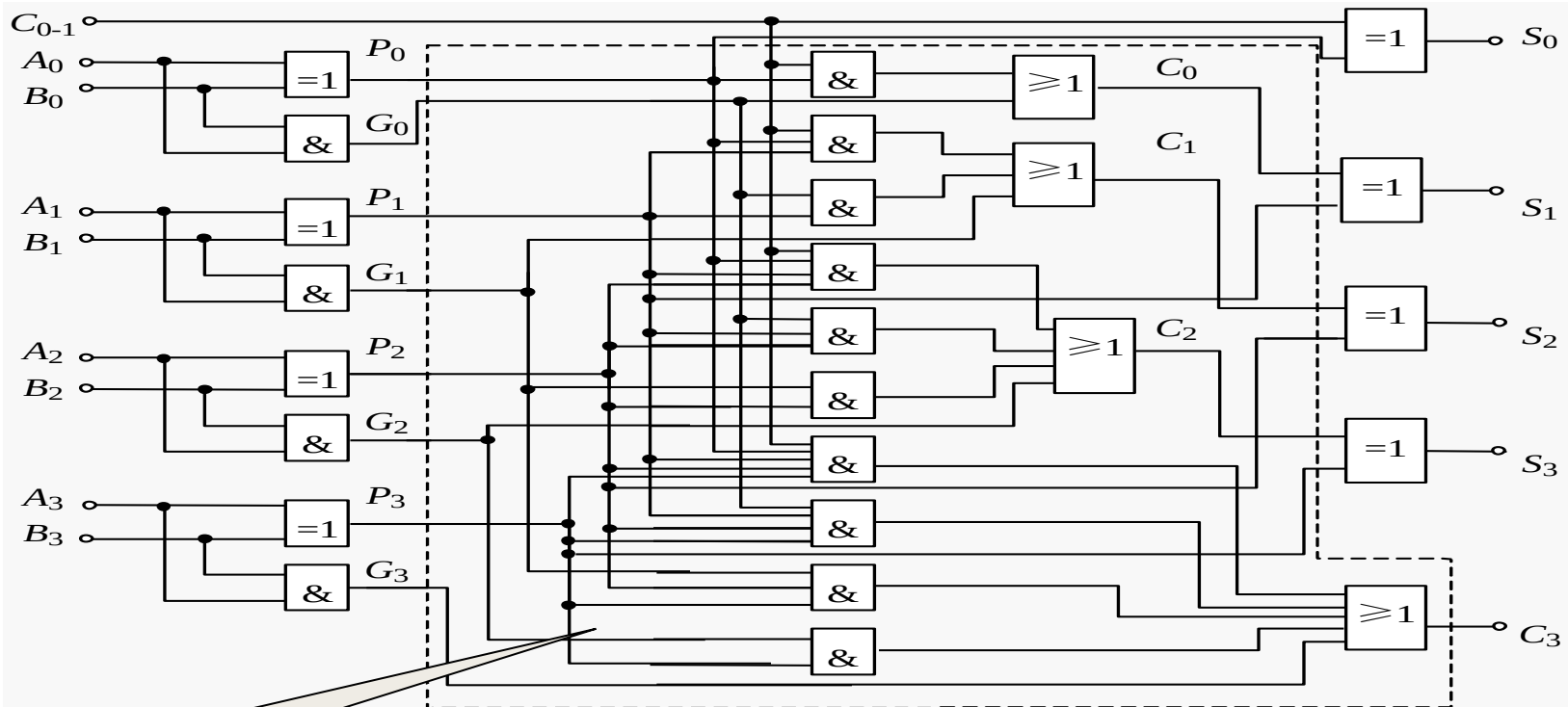
4位超前进位加法器

$$\begin{cases} S_0 = P_0 \oplus C_{0-1} \\ C_0 = G_0 + P_0 C_{0-1} \end{cases}$$

$$\begin{cases} S_1 = P_1 \oplus C_0 \\ C_1 = G_1 + P_1 C_0 = G_1 + P_1 G_0 + P_1 P_0 C_{0-1} \end{cases}$$

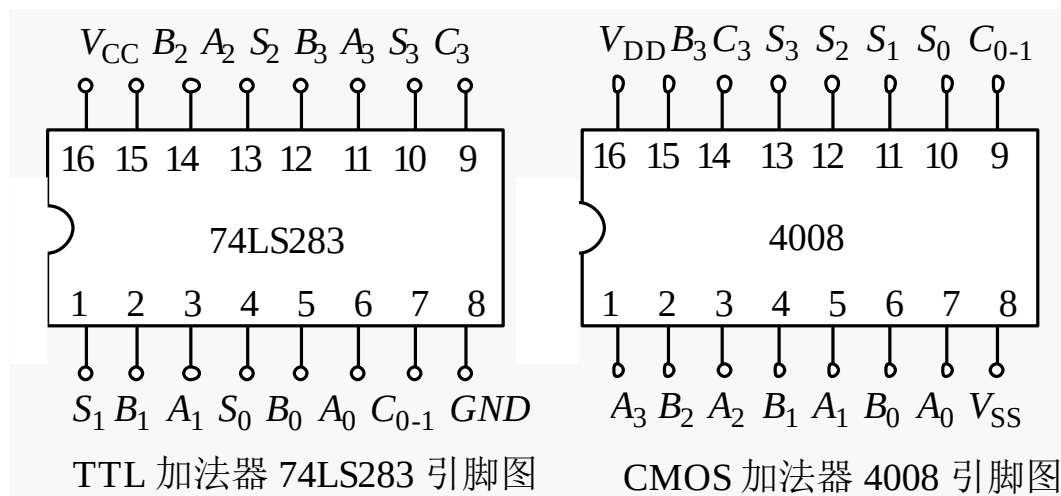
$$\begin{cases} S_2 = P_2 \oplus C_1 \\ C_2 = G_2 + P_2 C_1 = G_2 + P_2 G_1 + P_2 P_1 G_0 + P_2 P_1 P_0 C_{0-1} \end{cases}$$

$$\begin{cases} S_3 = P_3 \oplus C_2 \\ C_3 = G_3 + P_3 C_2 = G_3 + P_3 G_2 + P_3 P_2 G_1 + P_3 P_2 P_1 G_0 + P_3 P_2 P_1 P_0 C_{0-1} \end{cases}$$

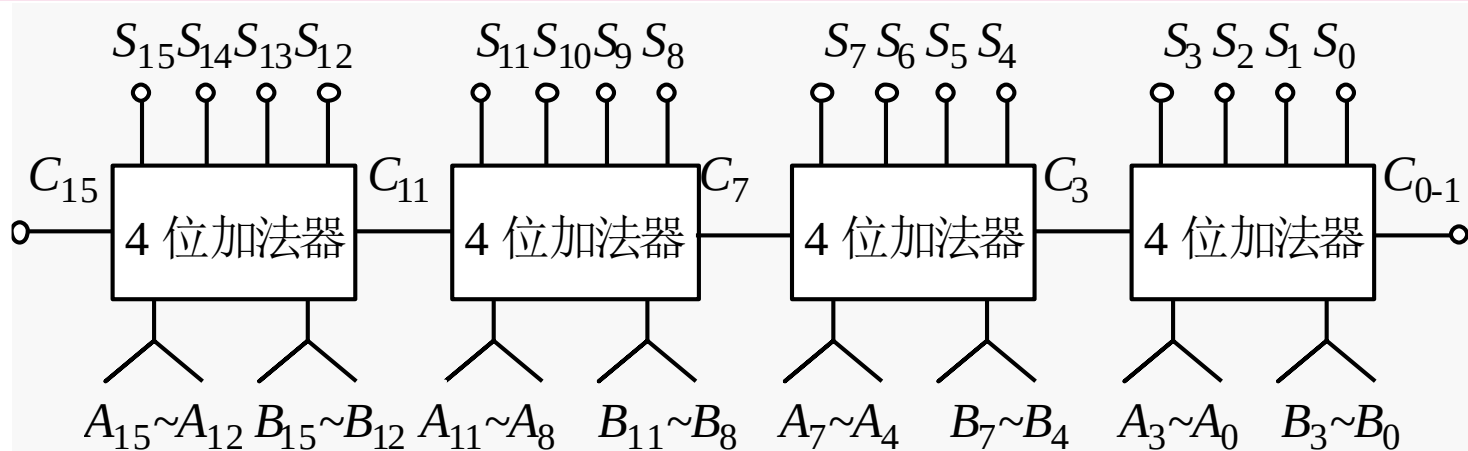


超前进位电路

集成二进制4位超前进位加法器

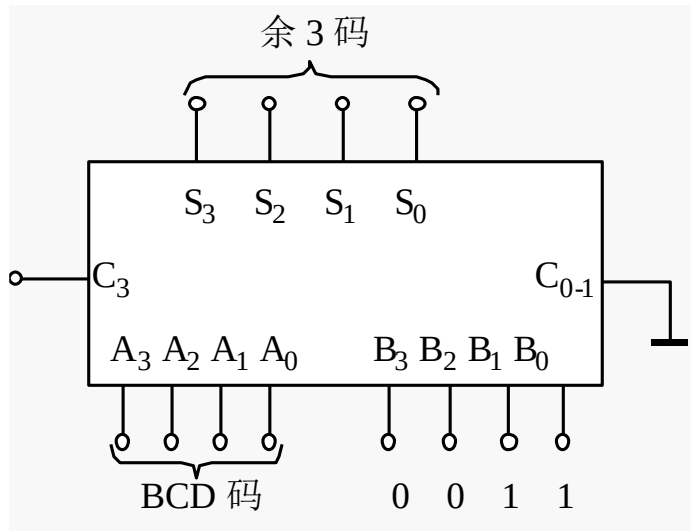


加法器的级连可以构成16位超前进位加法器



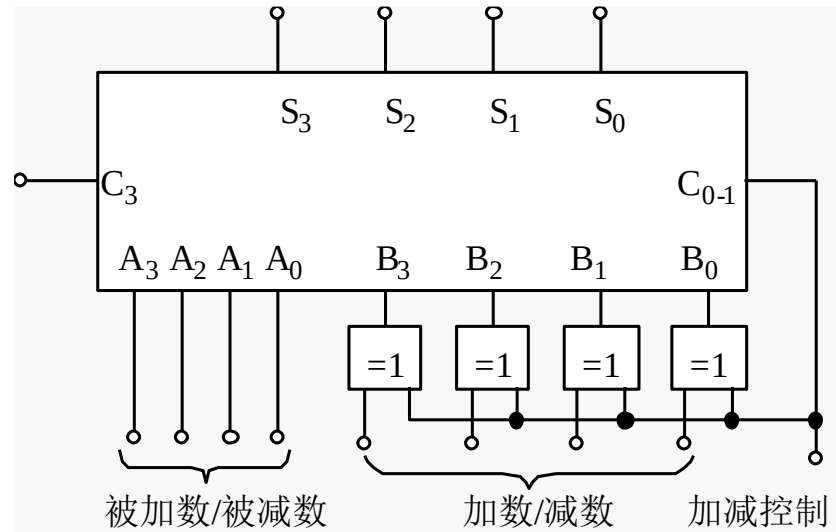
加法器的应用

8421 BCD码转换为余3码



BCD码+0011=余3码

二进制并行加法/减法器



$C_{0-1} = 0$ 时: $B \oplus 0 = B$, 执行 $A+B$ 运算

$C_{0-1} = 1$ 时: $B \oplus 1 = \bar{B}$, 执行 $A-B = A + \bar{B} + 1$ 运算

计算机中的算术逻辑单元 (ALU)

- **算术逻辑单元**：简称ALU，是一种功能较强的组合逻辑电路，能进行多种算术运算和逻辑运算。
- ALU的基本逻辑结构是超前进位加法器，它是通过改变加法器的Pi和Gi来获得多种运算能力的。

SN74181型四位ALU

- 能执行16种算术运算和16种逻辑运算；
- M：状态控制端，当M=H，执行逻辑运算；M=L执行算术运算；
- S_0-S_3 ：运算选择控制端，它决定电路执行哪种算术运算或哪种逻辑运算；
- A_3-A_0 , B_3-B_0 ：参加运算的两个数；

SN74181型ALU功能表

功 能 表				
工作方式选择 输入 $S_3S_2S_1S_0$	负逻辑输入或输出		正逻辑输入或输出	
	逻辑运算 ($M=1$)	算术运算 ($M=0$) ($C_{-1}=0$)	逻辑运算 ($M=1$)	算术运算 ($M=0$) ($C_{-1}=1$)
0000	$\overline{A}$	A 减 1	$\overline{A}$	A
0001	$\overline{AB}$	AB 减 1	$\overline{A+B}$	$A+B$
0010	$\overline{A+B}$	$A\overline{B}$ 减 1	$\overline{AB}$	$A+\overline{B}$
0011	逻辑 1	减 1	逻辑 0	减 1
0100	$\overline{A+B}$	A 加 ($A+\overline{B}$)	$\overline{AB}$	A 加 $A\overline{B}$
0101	$\overline{B}$	AB 加 ($A+\overline{B}$)	$\overline{B}$	($A+B$) 加 $A\overline{B}$
0110	$\overline{A\oplus B}$	A 减 B 减 1	$A\oplus B$	A 减 B 减 1
0111	$A+\overline{B}$	$A+\overline{B}$	$A\overline{B}$	$A\overline{B}$ 减 1
1000	$\overline{AB}$	A 加 ($A+B$)	$\overline{A+B}$	A 加 AB
1001	$A\oplus B$	A 加 B	$\overline{A\oplus B}$	A 加 B
1010	B	$A\overline{B}$ 加 ($A+B$)	B	($A+\overline{B}$) 加 AB
1011	$A+B$	$A+B$	AB	AB 减 1
1100	逻辑 0	A 加 A^*	逻辑 1	A 加 A^*
1101	$A\overline{B}$	AB 加 A	$A+\overline{B}$	($A+B$) 加 A
1110	AB	$A\overline{B}$ 加 A	$A+B$	($A+\overline{B}$) 加 A
1111	A	A	A	A 减 1

(1) 1=高电平; 0=低电平; (2) *表示每一位均移到下一个更高位, 即 $A^*=2A$

3.2.2 数值比较器

- 数值比较器：用来完成两个二进制数的大小比较的逻辑电路。

1位数值比较器

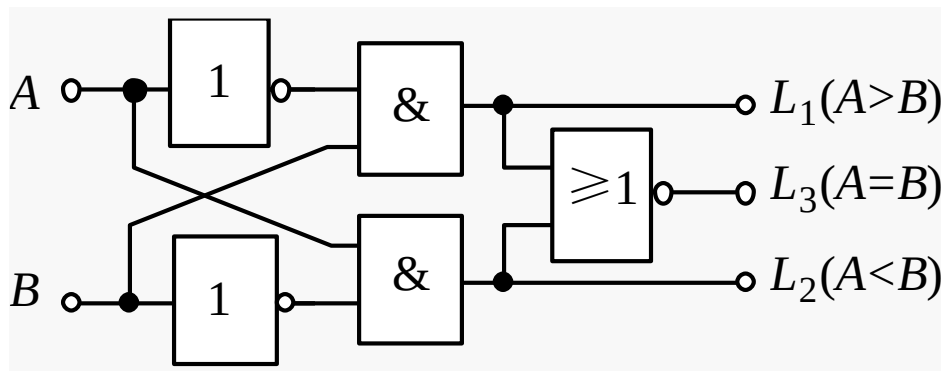
真值表 ($A > B$ 时 $L_1 = 1$; $A < B$ 时 $L_2 = 1$; $A = B$ 时 $L_3 = 1$)

A	B	$L_1(A > B)$	$L_2(A < B)$	$L_3(A = B)$
0	0	0	0	1
0	1	0	1	0
1	0	1	0	0
1	1	0	0	1

逻辑表达式

$$\begin{cases} L_1 = A\bar{B} \\ L_2 = \bar{A}B \\ L_3 = \bar{A}\bar{B} + AB = \overline{\bar{A}B + A\bar{B}} \end{cases}$$

逻辑图



4位数值比较器

- 两个4位二进制数 $A_3A_2A_1A_0$ 、 $B_3B_2B_1B_0$ 进行比较，从最高位开始比较，依次逐位进行，直到比较出结果为止。
- A' 与 B' 供扩展用，可与其它比较器连接，组成更多位数的数值比较器。

比 较 输 入				级 联 输 入			输 出		
$A_3 B_3$	$A_2 B_2$	$A_1 B_1$	$A_0 B_0$	$A'>B'$	$A'<B'$	$A'=B'$	$A>B$	$A<B$	$A=B$
$A_3>B_3$	×	×	×	×	×	×	1	0	0
$A_3<B_3$	×	×	×	×	×	×	0	1	0
$A_3=B_3$	$A_2>B_2$	×	×	×	×	×	1	0	0
$A_3=B_3$	$A_2<B_2$	×	×	×	×	×	0	1	0
$A_3=B_3$	$A_2=B_2$	$A_1>B_1$	×	×	×	×	1	0	0
$A_3=B_3$	$A_2=B_2$	$A_1<B_1$	×	×	×	×	0	1	0
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	$A_0>B_0$	×	×	×	1	0	0
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	$A_0<B_0$	×	×	×	0	1	0
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	$A_0=B_0$	1	0	0	1	0	0
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	$A_0=B_0$	0	1	0	0	1	0
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	$A_0=B_0$	0	0	1	0	0	1

比 较 输 入				级 联 输 入			输 出		
$A_3 B_3$	$A_2 B_2$	$A_1 B_1$	$A_0 B_0$	$A'>B' A'<B' A'=B'$			$A>B A<B A=B$		
$A_3>B_3$	×	×	×	×	×	×	1	0	0
$A_3<B_3$	×	×	×	×	×	×	0	1	0
$A_3=B_3$	$A_2>B_2$	×	×	×	×	×	1	0	0
$A_3=B_3$	$A_2<B_2$	×	×	×	×	×	0	1	0
$A_3=B_3$	$A_2=B_2$	$A_1>B_1$	×	×	×	×	1	0	0
$A_3=B_3$	$A_2=B_2$	$A_1<B_1$	×	×	×	×	0	1	0
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	$A_0>B_0$	×	×	×	1	0	0
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	$A_0<B_0$	×	×	×	0	1	0
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	$A_0=B_0$	1	0	0	1	0	0
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	$A_0=B_0$	0	1	0	0	1	0
$A_3=B_3$	$A_2=B_2$	$A_1=B_1$	$A_0=B_0$	0	0	1	0	0	1

● 3个输出信号： $L_1(A > B)$ 、 $L_2(A < B)$ 、和 $L_3(A = B)$

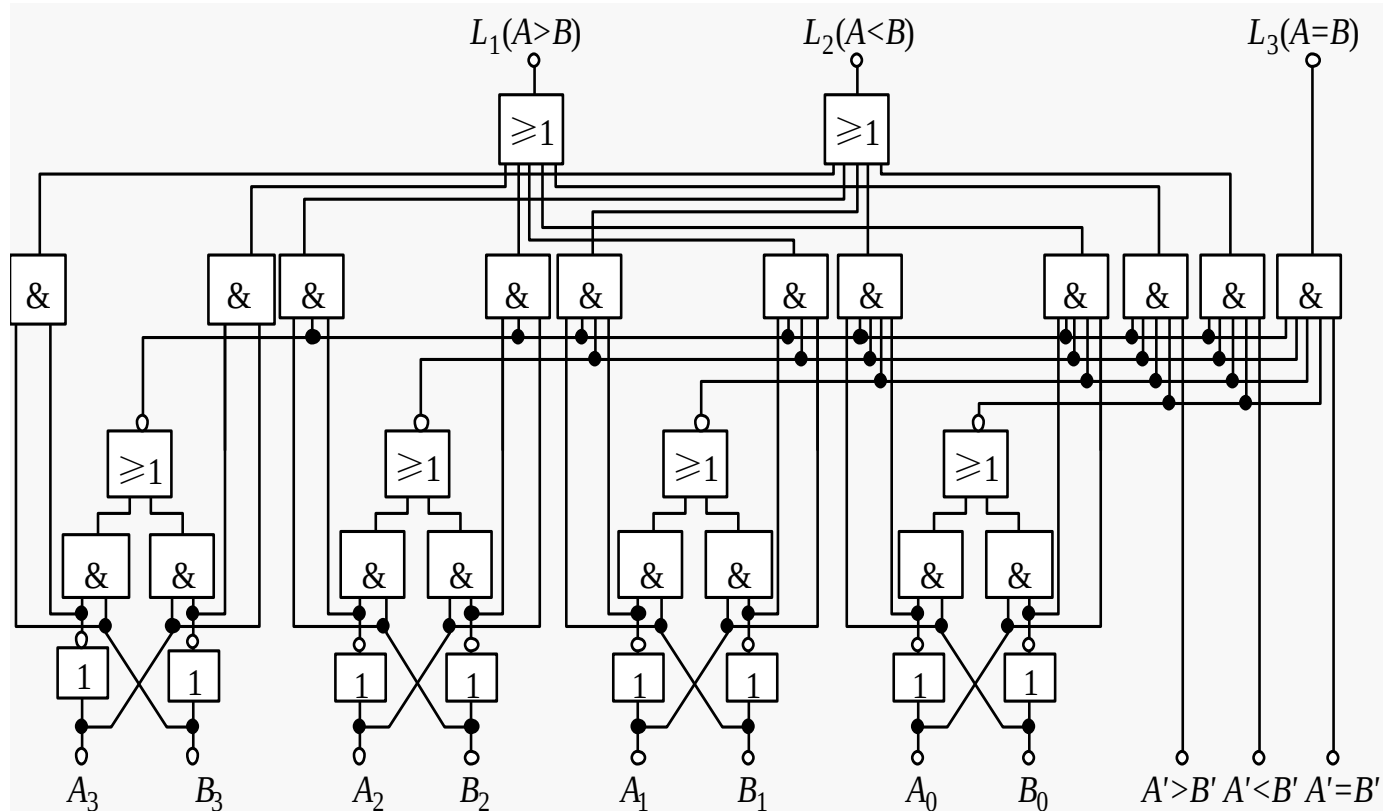
设 $L'_1 = (A' > B')$, $L'_2 = (A' < B')$, $L'_3 = (A' = B')$,

$L_{31} = A_3 \bar{B}_3 = (A_3 > B_3)$, $L_{32} = \bar{A}_3 B_3 = (A_3 < B_3)$,

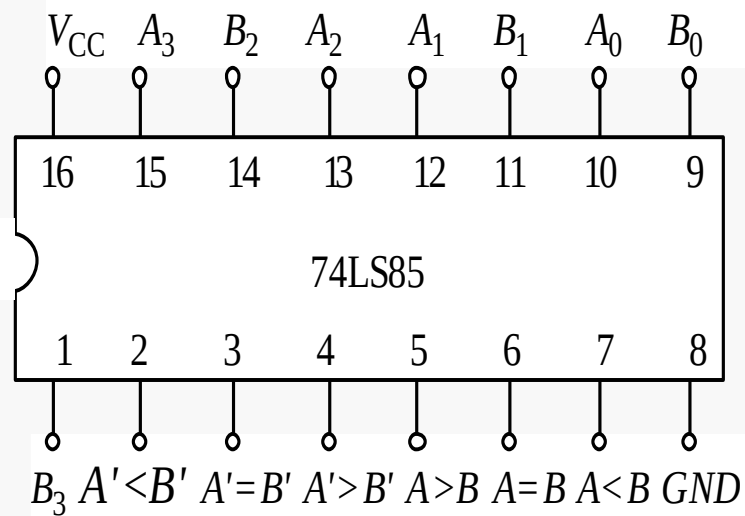
$L_{33} = \bar{\bar{A}_3 B_3 + A_3 \bar{B}_3} = (A_3 = B_3)$, 余类推。由真值表可得：

$$\begin{cases} L_1 = L_{31} + L_{33}L_{21} + L_{33}L_{23}L_{11} + L_{33}L_{23}L_{13}L_{01} + L_{33}L_{23}L_{13}L_{03}L'_1 \\ L_2 = L_{32} + L_{33}L_{22} + L_{33}L_{23}L_{12} + L_{33}L_{23}L_{13}L_{02} + L_{33}L_{23}L_{13}L_{03}L'_2 \\ L_3 = L_{33}L_{23}L_{13}L_{03}L'_3 \end{cases}$$

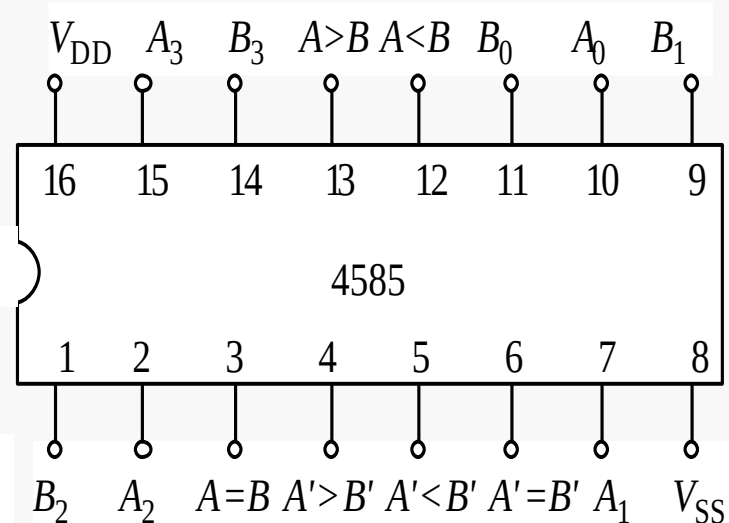
逻辑图



集成数值比较器

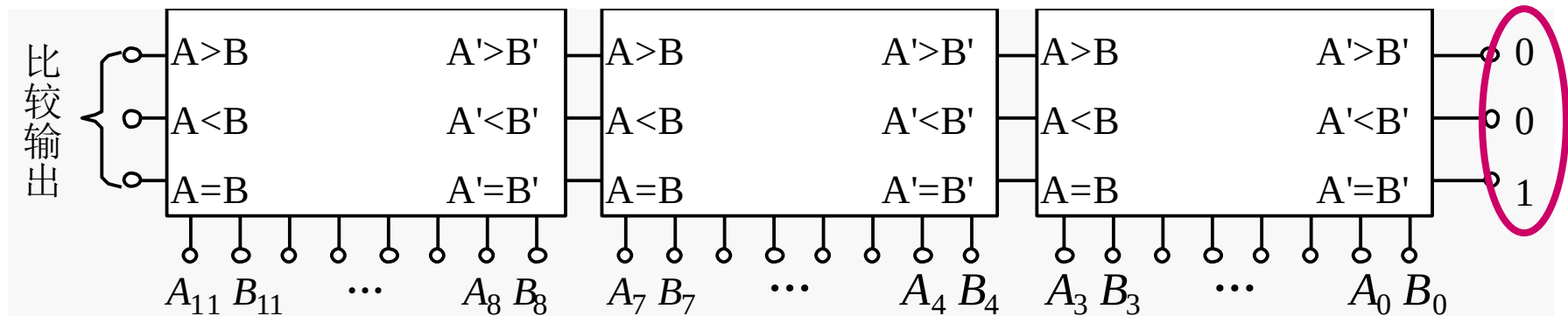


(a) TTL 数值比较器引脚图

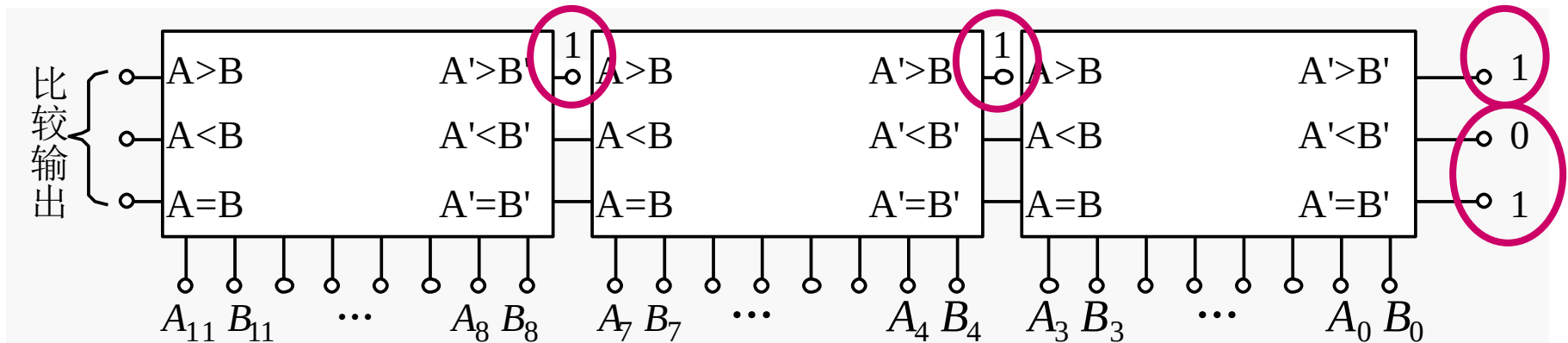


(b) CMOS 数值比较器引脚图

串联扩展



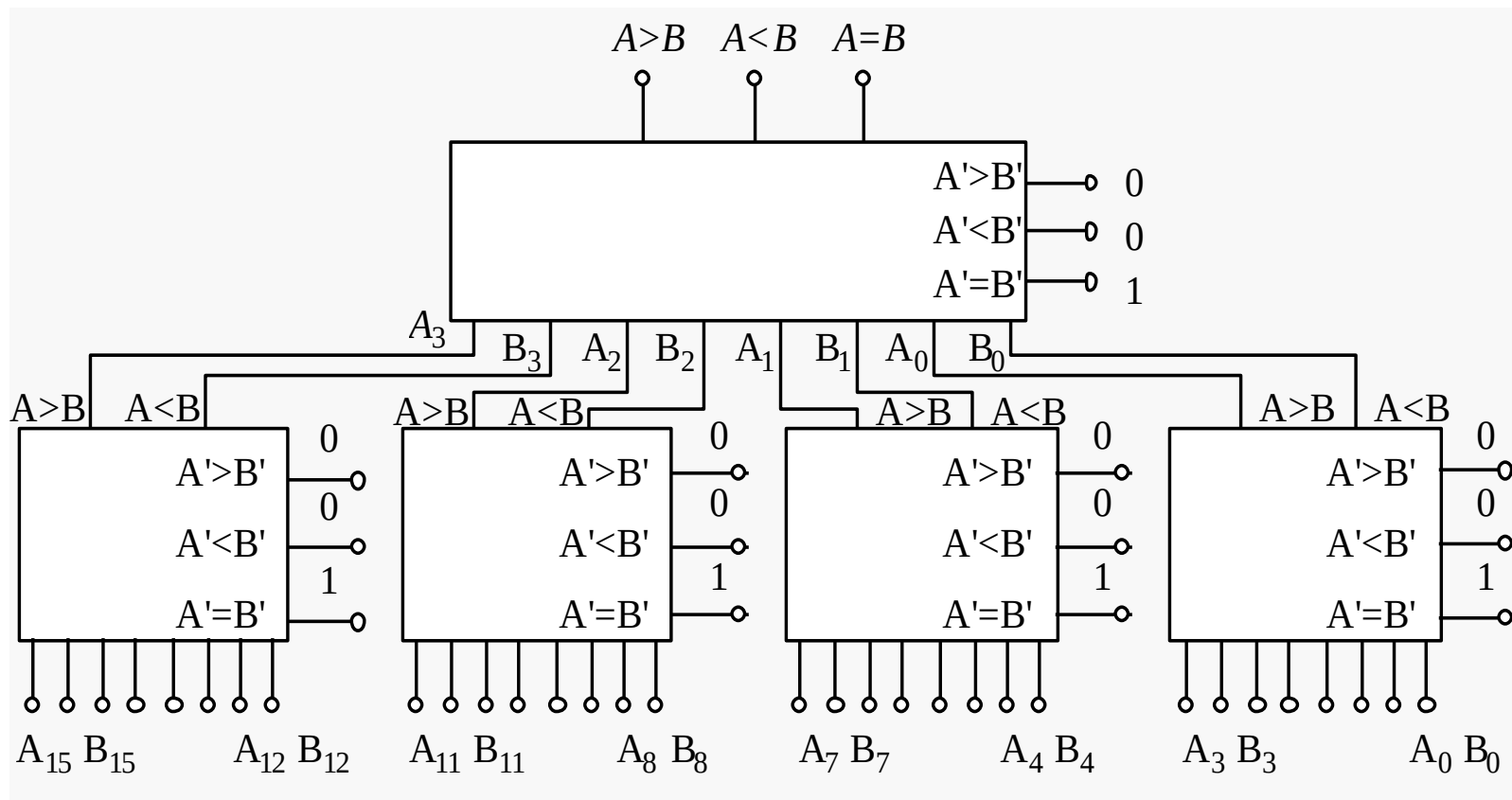
TTL电路：最低4位的级联输入端 $A' > B'$ 、 $A' < B'$ 和 $A' = B'$ 必须预置为0、0、1。



CMOS电路：各级的级联输入端 $A' > B'$ 必须预先预置为1。

- COMS集成4位数值比较器中，为了使电路简化，实现 $A > B$ 的方式是先求出 $A < B$ 和 $A = B$ ，再将两者进行或非运算而求得 $A > B$ ， $A > B$ 与 $A' > B'$ 无关；COMS集成4位数值比较器中 $A' > B'$ 形如虚设。
- TTL集成4位数值比较器中，是由各位数码的比较结果及级联输入端直接求得 $A > B$ 的。

并联扩展



TTL16位并联数值比较器，从数据输入到稳定输出只需2倍的4位比较延时时间，若采用串联方式则需要4倍的4位比较器的延时时间。

3.3 编码器和译码器

3.3.1 编码器

3.3.2 译码器

3.3.1 编码器

编码器：实现编码操作的电路。

3位二进制编码器

真值表

输入	输 出		
	Y_2	Y_1	Y_0
I_0	0	0	0
I_1	0	0	1
I_2	0	1	0
I_3	0	1	1
I_4	1	0	0
I_5	1	0	1
I_6	1	1	0
I_7	1	1	1

编码器在任何时刻只能对一个输入信号进行编码，即不允许有一个以上的输入信号同时有效。

逻辑表达式

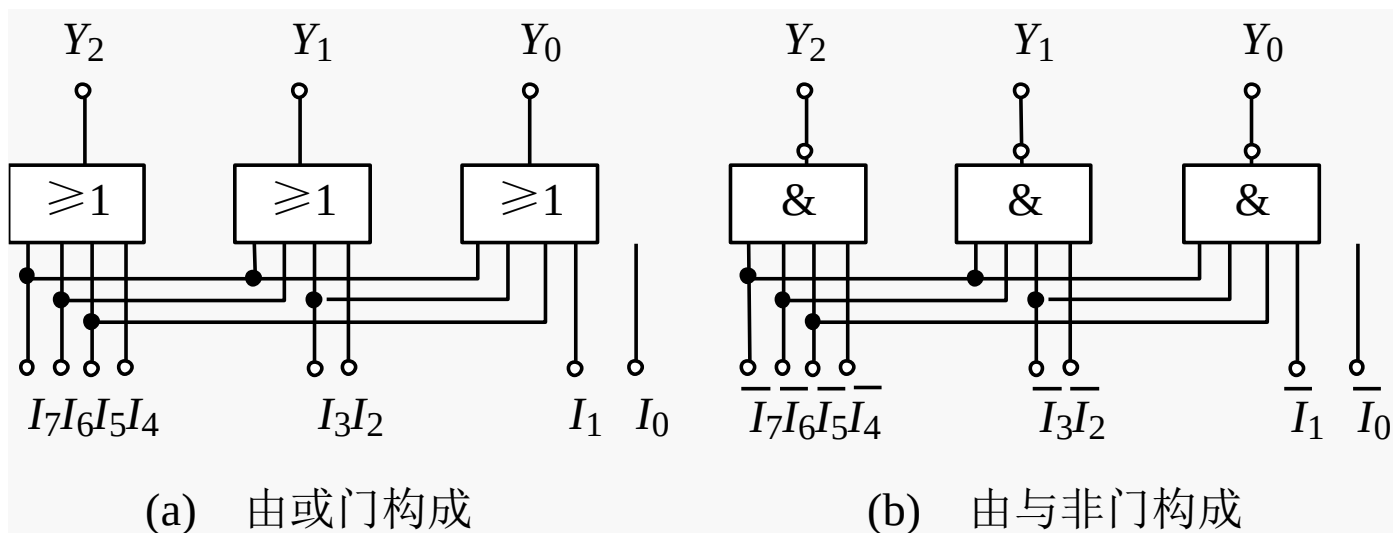
$$Y_2 = I_4 + I_5 + I_6 + I_7 = \overline{\overline{I_4} \overline{I_5} \overline{I_6} \overline{I_7}}$$

$$Y_1 = I_2 + I_3 + I_6 + I_7 = \overline{\overline{I_2} \overline{I_3} \overline{I_6} \overline{I_7}}$$

$$Y_0 = I_1 + I_3 + I_5 + I_7 = \overline{\overline{I_1} \overline{I_3} \overline{I_5} \overline{I_7}}$$

输入	输 出		
	Y_2	Y_1	Y_0
I_0	0	0	0
I_1	0	0	1
I_2	0	1	0
I_3	0	1	1
I_4	1	0	0
I_5	1	0	1
I_6	1	1	0
I_7	1	1	1

逻辑图



3位二进制优先编码器

普通编码器：输入信号是互相排斥的，即每一时刻只能有一个输入信号有效。

优先编码器：允许几个信号同时有效(或同时输入)，但只对优先级别最高的输入信号进行编码，不理睬级别低的信号。

设 I_7 的优先级别最高， I_6 次之，依此类推， I_0 最低。表中1=有效；0=无效；X=任意

真值表

输 入								输 出		
I_7	I_6	I_5	I_4	I_3	I_2	I_1	I_0	Y_2	Y_1	Y_0
1	X	X	X	X	X	X	X	1	1	1
0	1	X	X	X	X	X	X	1	1	0
0	0	1	X	X	X	X	X	1	0	1
0	0	0	1	X	X	X	X	1	0	0
0	0	0	0	1	X	X	X	0	1	1
0	0	0	0	0	1	X	X	0	1	0
0	0	0	0	0	0	1	X	0	0	1
0	0	0	0	0	0	0	1	0	0	0

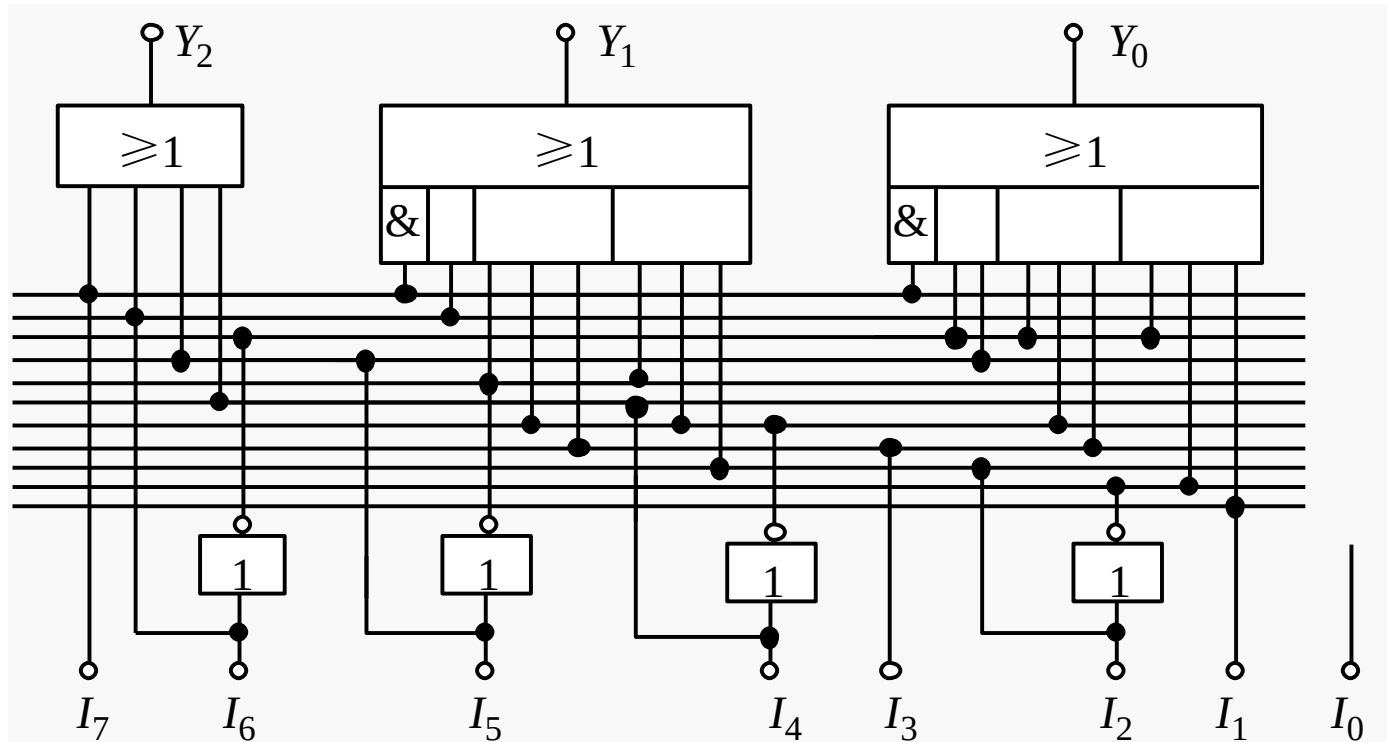
输 入								输 出		
I_7	I_6	I_5	I_4	I_3	I_2	I_1	I_0	Y_2	Y_1	Y_0
1	×	×	×	×	×	×	×	1	1	1
0	1	×	×	×	×	×	×	1	1	0
0	0	1	×	×	×	×	×	1	0	1
0	0	0	1	×	×	×	×	1	0	0
0	0	0	0	1	×	×	×	0	1	1
0	0	0	0	0	1	×	×	0	1	0
0	0	0	0	0	0	1	×	0	0	1
0	0	0	0	0	0	0	1	0	0	0

逻辑表达式

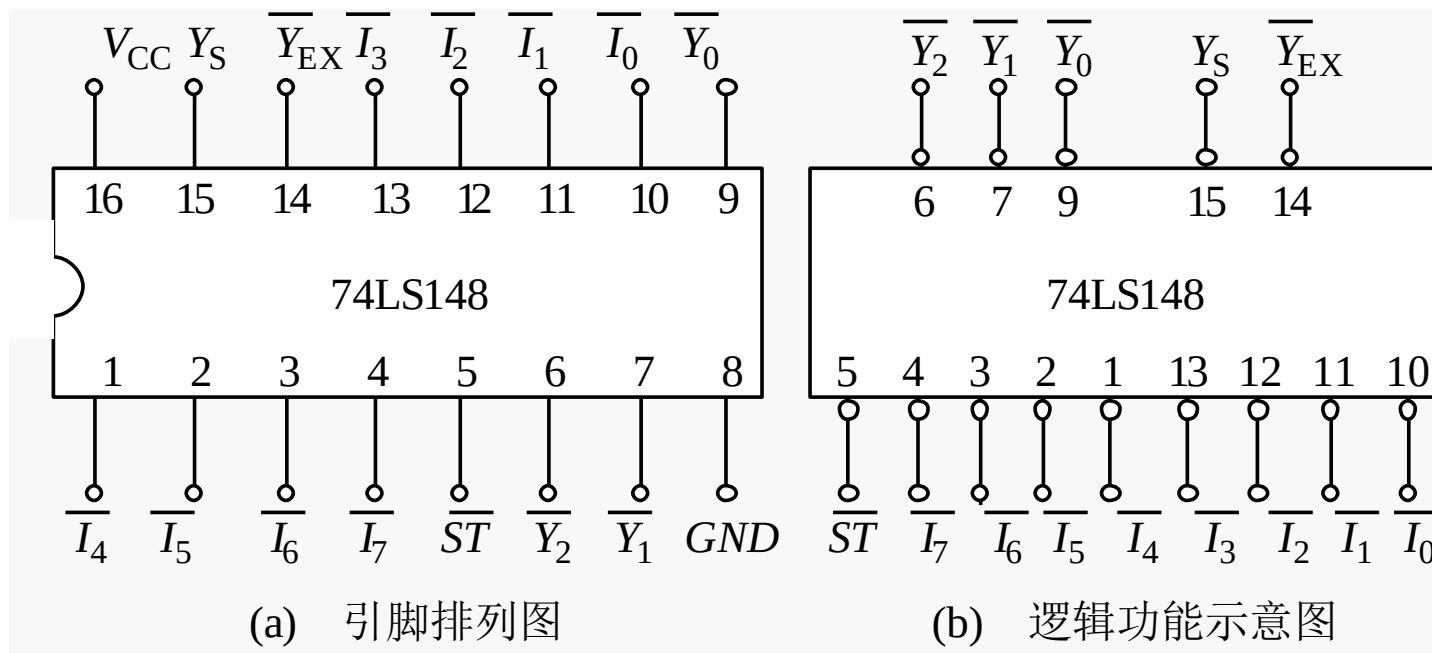
$$\begin{cases}
 Y_2 = I_7 + \bar{I}_7 I_6 + \bar{I}_7 \bar{I}_6 I_5 + \bar{I}_7 \bar{I}_6 \bar{I}_5 I_4 \\
 \quad = I_7 + I_6 + I_5 + I_4 \\
 Y_1 = I_7 + \bar{I}_7 I_6 + \bar{I}_7 \bar{I}_6 \bar{I}_5 \bar{I}_4 I_3 + \bar{I}_7 \bar{I}_6 \bar{I}_5 \bar{I}_4 \bar{I}_3 I_2 \\
 \quad = I_7 + I_6 + \bar{I}_5 \bar{I}_4 I_3 + \bar{I}_5 \bar{I}_4 I_2 \\
 Y_0 = I_7 + \bar{I}_7 \bar{I}_6 I_5 + \bar{I}_7 \bar{I}_6 \bar{I}_5 \bar{I}_4 I_3 + \bar{I}_7 \bar{I}_6 \bar{I}_5 \bar{I}_4 \bar{I}_3 \bar{I}_2 I_1 \\
 \quad = I_7 + \bar{I}_6 I_5 + \bar{I}_6 \bar{I}_4 I_3 + \bar{I}_6 \bar{I}_4 \bar{I}_2 I_1
 \end{cases}$$

公式16
 $A + \bar{A}B = A + B$

8线-3线优先编码器逻辑图



集成3位二进制优先编码器74LS148



$\overline{ST}$ 为使能输入端，低电平有效。 Y_S 为使能输出端，通常接至低位芯片的使能输入端。 Y_S 和 $\overline{ST}$ 配合可以实现多级编码器之间的优先级别的控制。 $\overline{Y}_{EX}$ 为扩展输出端，是控制标志。 $\overline{Y}_{EX} = 0$ 表示是编码输出； $\overline{Y}_{EX} = 1$ 表示不是编码输出。

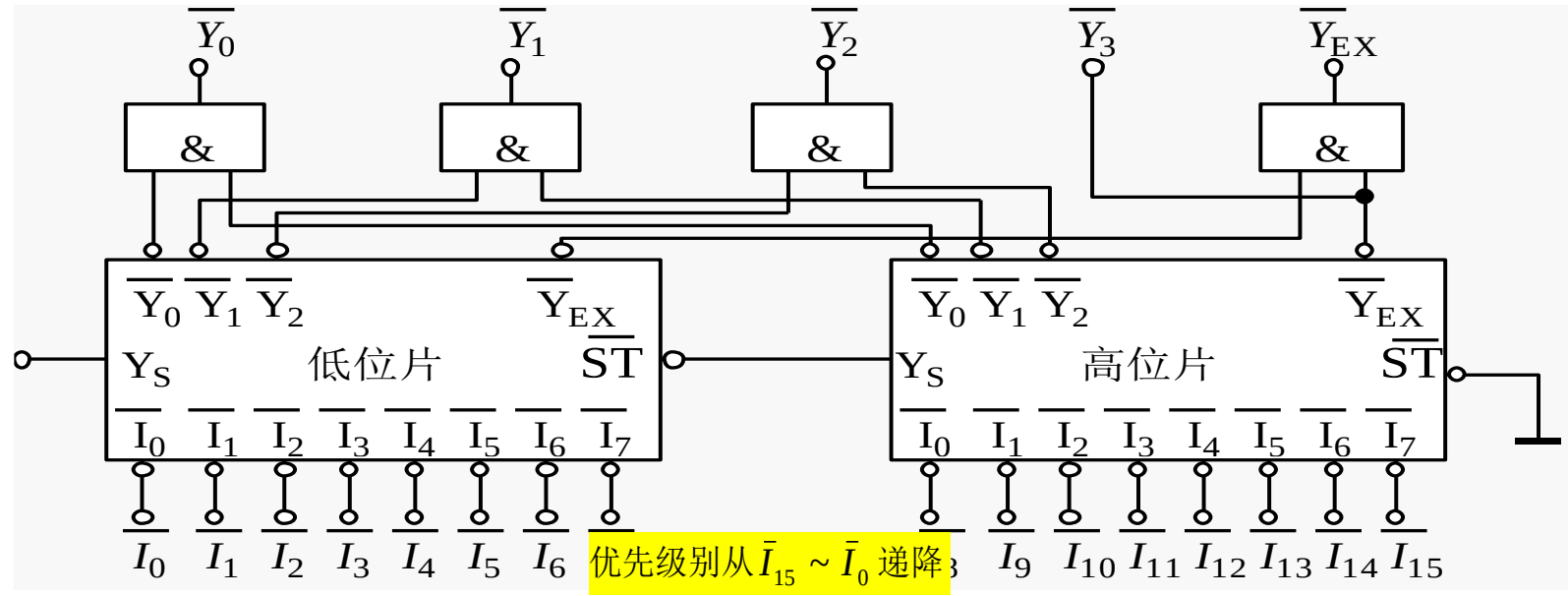
集成8线-3线优先编码器74LS148的真值表

输 入									输 出				
$\overline{ST}$	$\overline{I}_7$	$\overline{I}_6$	$\overline{I}_5$	$\overline{I}_4$	$\overline{I}_3$	$\overline{I}_2$	$\overline{I}_1$	$\overline{I}_0$	$\overline{Y}_2$	$\overline{Y}_1$	$\overline{Y}_0$	$\overline{Y}_{EX}$	Y_S
1	×	×	×	×	×	×	×	×	1	1	1	1	1
0	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	×	×	×	×	×	×	×	0	0	0	0	1
0	1	0	×	×	×	×	×	×	0	0	1	0	1
0	1	1	0	×	×	×	×	×	0	1	0	0	1
0	1	1	1	0	×	×	×	×	0	1	1	0	1
0	1	1	1	1	0	×	×	×	1	0	0	0	1
0	1	1	1	1	1	0	×	×	1	0	1	0	1
0	1	1	1	1	1	1	0	×	1	1	0	0	1
0	1	1	1	1	1	1	1	0	1	1	1	0	1

输入：逻辑0(低电平) 有效

输出：逻辑0(低电平) 有效

级联应用举例

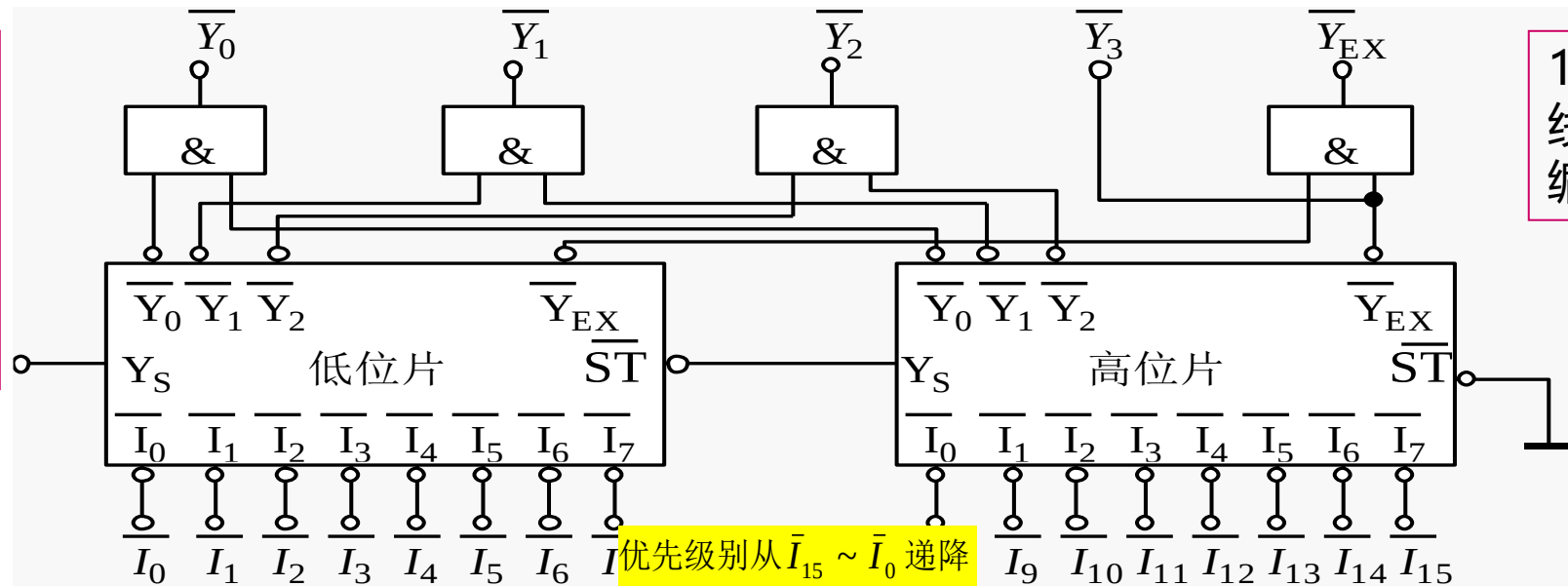


16线-4线
优先
编码器

输 入									输 出					
$\overline{ST}$	$\bar{I}_7$	$\bar{I}_6$	$\bar{I}_5$	$\bar{I}_4$	$\bar{I}_3$	$\bar{I}_2$	$\bar{I}_1$	$\bar{I}_0$	$\bar{Y}_3$	$\bar{Y}_2$	$\bar{Y}_1$	$\bar{Y}_0$	$\bar{Y}_{EX}$	Y_S
1	×	×	×	×	×	×	×	×	1	1	1	1	1	1
0	1	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	×	×	×	×	×	×	×	0	0	0	0	0	1
0	1	0	×	×	×	×	×	×	0	0	0	1	0	1
0	1	1	0	×	×	×	×	×	0	0	1	0	0	1
0	1	1	1	0	×	×	×	×	0	0	1	1	0	1
0	1	1	1	1	0	×	×	×	0	1	0	0	0	1
0	1	1	1	1	1	0	×	×	0	1	0	1	0	1
0	1	1	1	1	1	1	0	×	0	1	1	0	0	1
0	1	1	1	1	1	1	1	0	0	1	1	1	0	1

级联应用举例

16线-4线优先编码器



输 入									输				出	
$\overline{ST}$	$\bar{I}_7$	$\bar{I}_6$	$\bar{I}_5$	$\bar{I}_4$	$\bar{I}_3$	$\bar{I}_2$	$\bar{I}_1$	$\bar{I}_0$	$\overline{Y}_3$	$\overline{Y}_2$	$\overline{Y}_1$	$\overline{Y}_0$	$\overline{Y}_{EX}$	Y_S
1	×	×	×	×	×	×	×	×	1	1	1	1	1	1
0	1	1	1	1	1	1	1	1	1	1	1	1	1	0
0	0	×	×	×	×	×	×	×	1	0	0	0	0	1
0	1	0	×	×	×	×	×	×	1	0	0	1	0	1
0	1	1	0	×	×	×	×	×	1	0	1	0	0	1
0	1	1	1	0	×	×	×	×	1	0	1	1	0	1
0	1	1	1	1	0	×	×	×	1	1	0	0	0	1
0	1	1	1	1	1	0	×	×	1	1	0	1	0	1
0	1	1	1	1	1	1	0	×	1	1	1	0	0	1
0	1	1	1	1	1	1	1	0	1	1	1	1	0	1

8421 BCD码编码器

二-十进制编码器真值表

输 入 I	输 出			
	Y_3	Y_2	Y_1	Y_0
0(I_0)	0	0	0	0
1(I_1)	0	0	0	1
2(I_2)	0	0	1	0
3(I_3)	0	0	1	1
4(I_4)	0	1	0	0
5(I_5)	0	1	0	1
6(I_6)	0	1	1	0
7(I_7)	0	1	1	1
8(I_8)	1	0	0	0
9(I_9)	1	0	0	1

输入10个互斥的数码

输出4位二进制代码

逻辑表达式

$$Y_3 = I_8 + I_9$$

$$= \overline{\overline{I_8} \overline{I_9}}$$

$$Y_2 = I_4 + I_5 + I_6 + I_7$$

$$= \overline{\overline{I_4} \overline{I_5} \overline{I_6} \overline{I_7}}$$

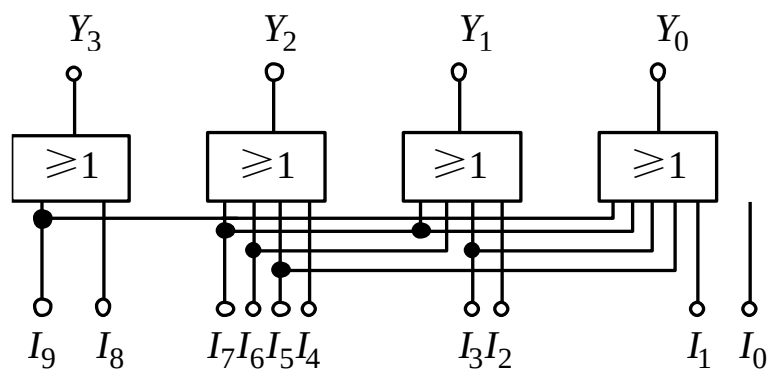
$$Y_1 = I_2 + I_3 + I_6 + I_7$$

$$= \overline{\overline{I_2} \overline{I_3} \overline{I_6} \overline{I_7}}$$

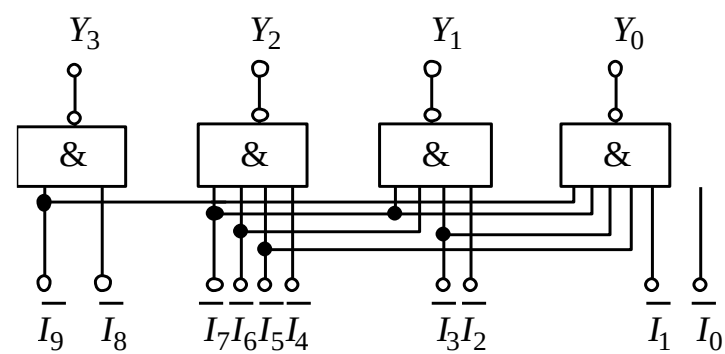
$$Y_0 = I_1 + I_3 + I_5 + I_7 + I_9$$

$$= \overline{\overline{I_1} \overline{I_3} \overline{I_5} \overline{I_7} \overline{I_9}}$$

逻辑图



(a) 由或门构成



(b) 由与非门构成

8421 BCD码优先编码器

真值表

I_9	I_8	I_7	I_6	I_5	I_4	I_3	I_2	I_1	I_0	Y_3	Y_2	Y_1	Y_0
1	×	×	×	×	×	×	×	×	×	1	0	0	1
0	1	×	×	×	×	×	×	×	×	1	0	0	0
0	0	1	×	×	×	×	×	×	×	0	1	1	1
0	0	0	1	×	×	×	×	×	×	0	1	1	0
0	0	0	0	1	×	×	×	×	×	0	1	0	1
0	0	0	0	0	1	×	×	×	×	0	1	0	0
0	0	0	0	0	0	1	×	×	×	0	0	1	1
0	0	0	0	0	0	0	1	×	×	0	0	1	0
0	0	0	0	0	0	0	0	1	×	0	0	0	1
0	0	0	0	0	0	0	0	0	1	0	0	0	0

优先级别从 I_9 至 I_0 递降

I_9	I_8	I_7	I_6	I_5	I_4	I_3	I_2	I_1	I_0	Y_3	Y_2	Y_1	Y_0
1	×	×	×	×	×	×	×	×	×	1	0	0	1
0	1	×	×	×	×	×	×	×	×	1	0	0	0
0	0	1	×	×	×	×	×	×	×	0	1	1	1
0	0	0	1	×	×	×	×	×	×	0	1	1	0
0	0	0	0	1	×	×	×	×	×	0	1	0	1
0	0	0	0	0	1	×	×	×	×	0	1	0	0
0	0	0	0	0	0	1	×	×	×	0	0	1	1
0	0	0	0	0	0	0	1	×	×	0	0	1	0
0	0	0	0	0	0	0	0	1	×	0	0	0	1
0	0	0	0	0	0	0	0	0	1	0	0	0	0

逻辑表达式

$$\begin{cases}
 Y_3 = I_9 + \bar{I}_9 I_8 = I_9 + I_8 \\
 Y_2 = \bar{I}_9 \bar{I}_8 I_7 + \bar{I}_9 \bar{I}_8 \bar{I}_7 I_6 + \bar{I}_9 \bar{I}_8 \bar{I}_7 \bar{I}_6 I_5 + \bar{I}_9 \bar{I}_8 \bar{I}_7 \bar{I}_6 \bar{I}_5 I_4 \\
 \quad = \bar{I}_9 \bar{I}_8 I_7 + \bar{I}_9 \bar{I}_8 I_6 + \bar{I}_9 \bar{I}_8 I_5 + \bar{I}_9 \bar{I}_8 I_4 \\
 Y_1 = \bar{I}_9 \bar{I}_8 I_7 + \bar{I}_9 \bar{I}_8 \bar{I}_7 I_6 + \bar{I}_9 \bar{I}_8 \bar{I}_7 \bar{I}_6 \bar{I}_5 \bar{I}_4 I_3 + \bar{I}_9 \bar{I}_8 \bar{I}_7 \bar{I}_6 \bar{I}_5 \bar{I}_4 \bar{I}_3 I_2 \\
 \quad = \bar{I}_9 \bar{I}_8 I_7 + \bar{I}_9 \bar{I}_8 I_6 + \bar{I}_9 \bar{I}_8 \bar{I}_5 \bar{I}_4 I_3 + \bar{I}_9 \bar{I}_8 \bar{I}_5 \bar{I}_4 I_2 \\
 Y_0 = I_9 + \bar{I}_9 \bar{I}_8 I_7 + \bar{I}_9 \bar{I}_8 \bar{I}_7 \bar{I}_6 I_5 + \bar{I}_9 \bar{I}_8 \bar{I}_7 \bar{I}_6 \bar{I}_5 \bar{I}_4 I_3 + \bar{I}_9 \bar{I}_8 \bar{I}_7 \bar{I}_6 \bar{I}_5 \bar{I}_4 \bar{I}_3 \bar{I}_2 I_1 \\
 \quad = I_9 + \bar{I}_8 I_7 + \bar{I}_8 \bar{I}_6 I_5 + \bar{I}_8 \bar{I}_6 \bar{I}_4 I_3 + \bar{I}_8 \bar{I}_6 \bar{I}_4 \bar{I}_2 I_1
 \end{cases}$$

3.3.2 译码器

译码器：实现把代码状态的特定含义翻译出来的电路。

- 二进制译码器：输入端为 n 个，输出端为 2^n 个，对应于输入代码的每一种状态， 2^n 个输出中只有一个为1（或为0），其余全为0（或为1）。
- 二进制译码器可以译出输入变量的全部状态，故又称为**变量译码器**。

3位二进制译码器真值表

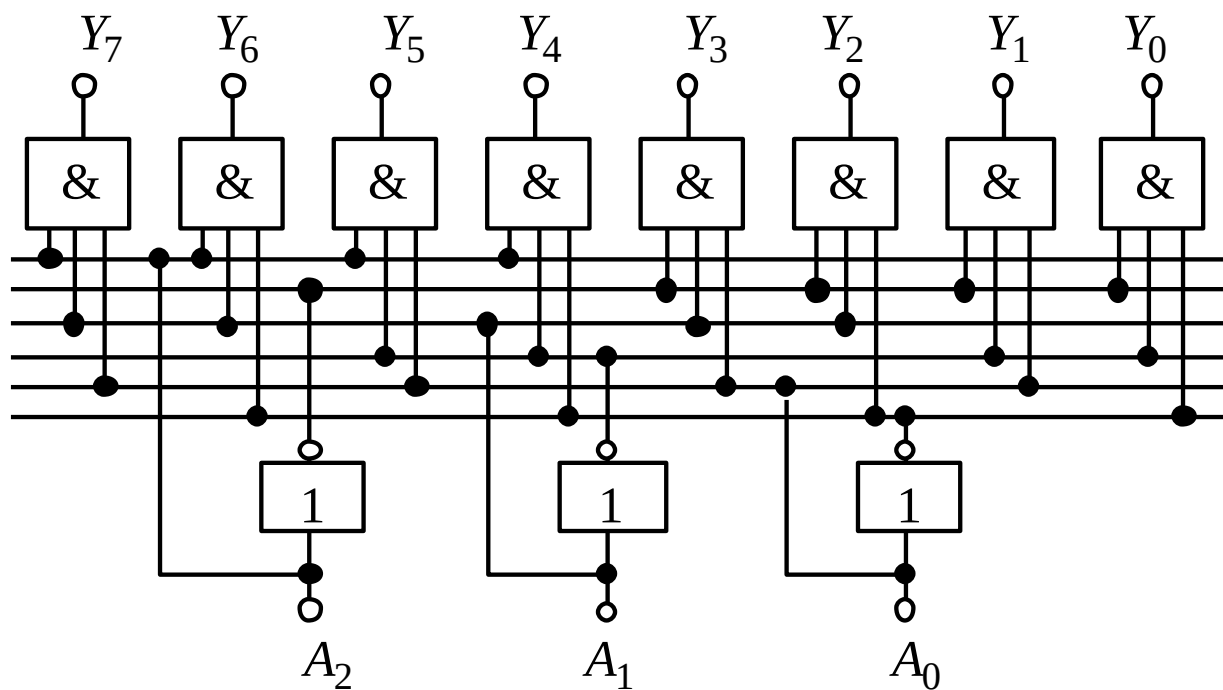
A_2	A_1	A_0	Y_0	Y_1	Y_2	Y_3	Y_4	Y_5	Y_6	Y_7
0	0	0	1	0	0	0	0	0	0	0
0	0	1	0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	1	0	0	0	0
1	0	0	0	0	0	0	1	0	0	0
1	0	1	0	0	0	0	0	1	0	0
1	1	0	0	0	0	0	0	0	1	0
1	1	1	0	0	0	0	0	0	0	1

输入：3位二进制代码，输出：8个互斥的信号

逻辑表达式

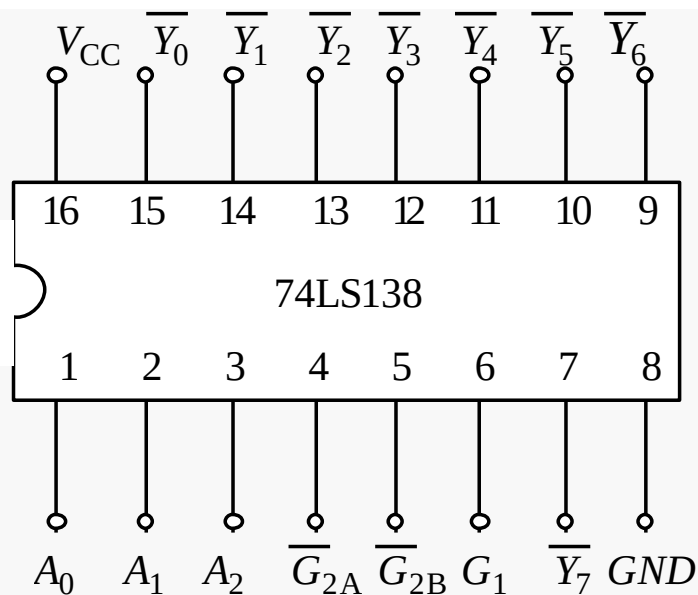
$$\left\{ \begin{array}{l} Y_0 = \bar{A}_2 \bar{A}_1 \bar{A}_0 \\ Y_1 = \bar{A}_2 \bar{A}_1 A_0 \\ Y_2 = \bar{A}_2 A_1 \bar{A}_0 \\ Y_3 = \bar{A}_2 A_1 A_0 \\ Y_4 = A_2 \bar{A}_1 \bar{A}_0 \\ Y_5 = A_2 \bar{A}_1 A_0 \\ Y_6 = A_2 A_1 \bar{A}_0 \\ Y_7 = A_2 A_1 A_0 \end{array} \right.$$

3-8译码器逻辑图

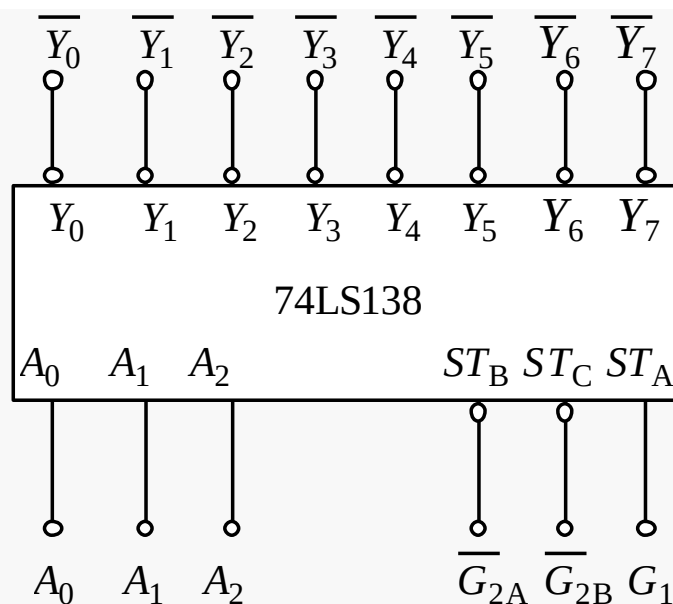


电路特点：与门组成的阵列

集成3线-8线译码器



(a) 引脚排列图



(b) 逻辑功能示意图

A_2 、 A_1 、 A_0 为二进制译码输入端， $\overline{Y}_7 \sim \overline{Y}_0$ 为译码输出端（低电平有效）， G_1 、 $\overline{G}_{2A}$ 、 $\overline{G}_{2B}$ 为选通控制端。当 $G_1 = 1$ 或 $\overline{G}_{2A} + \overline{G}_{2B} = 0$ 时，译码器处于工作状态；当 $G_1 = 0$ 、 $\overline{G}_{2A} + \overline{G}_{2B} = 1$ 时，译码器处于禁止状态。

$$\overline{G}_2 = \overline{G}_{2A} + \overline{G}_{2B}$$

真值表

输 入		输 出										
使 能	选 择											
G_1	$\overline{G}_2$	A_2	A_1	A_0	$\overline{Y}_7$	$\overline{Y}_6$	$\overline{Y}_5$	$\overline{Y}_4$	$\overline{Y}_3$	$\overline{Y}_2$	$\overline{Y}_1$	$\overline{Y}_0$
×	1	×	×	×	1	1	1	1	1	1	1	1
0	×	×	×	×	1	1	1	1	1	1	1	1
1	0	0	0	0	1	1	1	1	1	1	1	0
1	0	0	0	1	1	1	1	1	1	1	0	1
1	0	0	1	0	1	1	1	1	1	0	1	1
1	0	0	1	1	1	1	1	1	0	1	1	1
1	0	1	0	0	1	1	1	0	1	1	1	1
1	0	1	0	1	1	1	0	1	1	1	1	1
1	0	1	1	0	1	0	1	1	1	1	1	1
1	0	1	1	1	0	1	1	1	1	1	1	1

输入：自然二进制码；输出：低电平有效

变量译码器的应用

- 实现逻辑函数：译码器的每个输出端分别与一个最小项相对应，辅以适当的门电路，可实现任何组合逻辑函数。

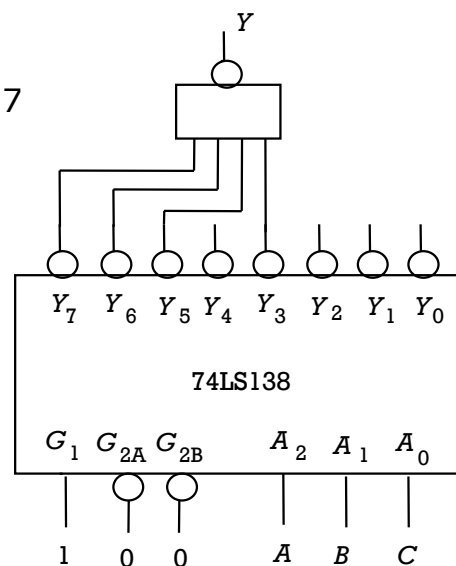
例. 试用译码器和门电路实现逻辑函数

$$Y = AB + BC + AC$$

解：（1）将逻辑函数转换成最小项表达式，再转换成与非—与非形式。

$$\begin{aligned} Y &= \overline{A}BC + A\overline{B}C + AB\overline{C} + ABC = m_3 + m_5 + m_6 + m_7 \\ &= \overline{m_3 \cdot m_5 \cdot m_6 \cdot m_7} \end{aligned}$$

（2）该函数有三个变量，所以选用3线—8线译码器74LS138。用一片74LS138加一个与非门就可实现逻辑函数Y。

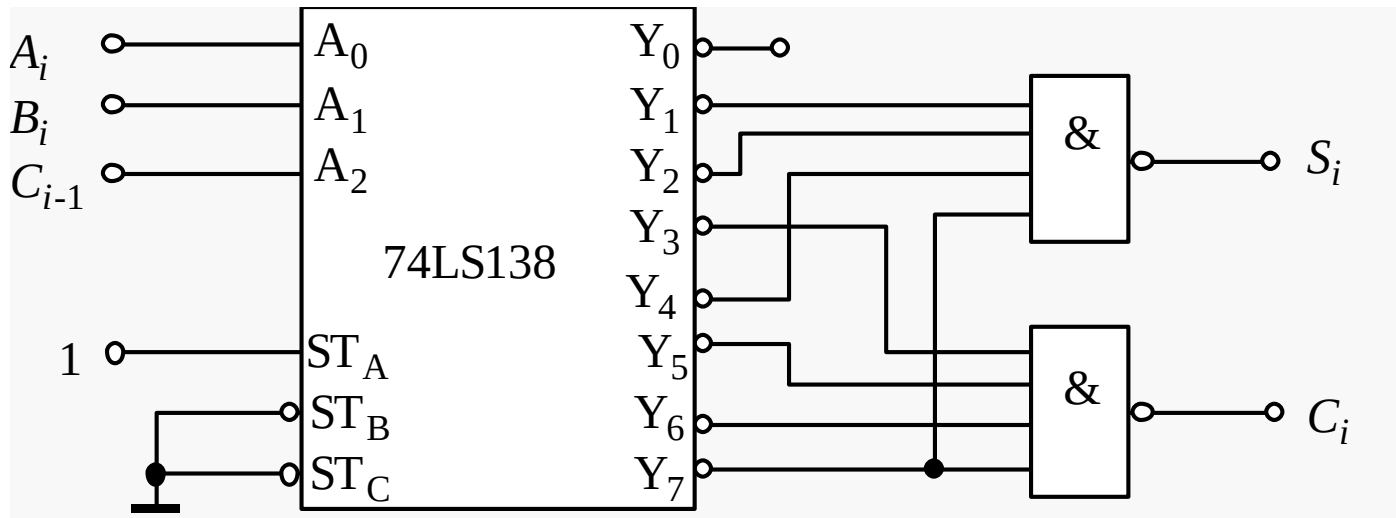


用二进制译码器实现逻辑函数

①写出函数的标准与或表达式，并变换为与非-与非形式。

$$\begin{cases} S_i(A_i, B_i, C_{i-1}) = \sum m(1, 2, 4, 7) = \overline{m_1} \overline{m_2} \overline{m_4} \overline{m_7} \\ C_i(A_i, B_i, C_{i-1}) = \sum m(3, 5, 6, 7) = \overline{m_3} \overline{m_5} \overline{m_6} \overline{m_7} \end{cases}$$

②画出用二进制译码器和与非门实现这些函数的接线图。



某组合逻辑电路的真值表如表所示，试用译码器和门电路设计该逻辑电路。

输 入			输 出		
A	B	C	L	F	G
0	0	0	0	0	1
0	0	1	1	0	0
0	1	0	1	0	1
0	1	1	0	1	0
1	0	0	1	0	1
1	0	1	0	1	0
1	1	0	0	1	1
1	1	1	1	0	0

解：写出各输出的最小项表达式，再转换成与非—与非形式：

$$L = \overline{\overline{A}}\overline{\overline{B}}\overline{\overline{C}} + \overline{\overline{A}}\overline{\overline{B}}\overline{\overline{C}} + \overline{\overline{A}}\overline{\overline{B}}\overline{\overline{C}} + \overline{\overline{A}}\overline{\overline{B}}\overline{\overline{C}} = m_1 + m_2 + m_4 + m_7 = \overline{\overline{\overline{m_1} \cdot m_2 \cdot m_4 \cdot m_7}}$$

$$F = \overline{\overline{A}}\overline{\overline{B}}\overline{\overline{C}} + \overline{\overline{A}}\overline{\overline{B}}\overline{\overline{C}} + \overline{\overline{A}}\overline{\overline{B}}\overline{\overline{C}} = m_3 + m_5 + m_6 = \overline{\overline{\overline{m_3} \cdot m_5 \cdot m_6}}$$

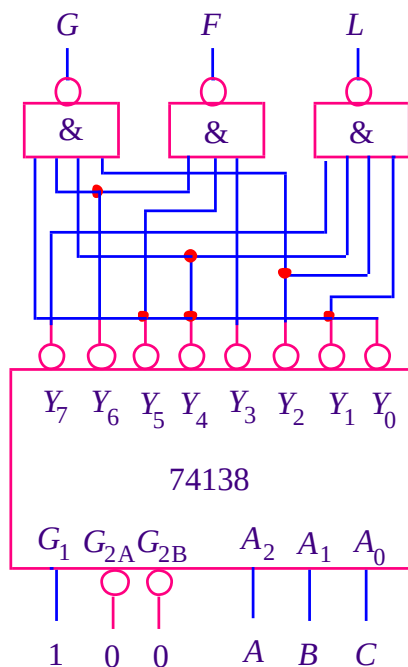
$$G = \overline{\overline{A}}\overline{\overline{B}}\overline{\overline{C}} + \overline{\overline{A}}\overline{\overline{B}}\overline{\overline{C}} + \overline{\overline{A}}\overline{\overline{B}}\overline{\overline{C}} + \overline{\overline{A}}\overline{\overline{B}}\overline{\overline{C}} = m_0 + m_2 + m_4 + m_6 = \overline{\overline{\overline{m_0} \cdot m_2 \cdot m_4 \cdot m_6}}$$

$$L = \overline{\overline{A}}\overline{\overline{B}}\overline{\overline{C}} + \overline{\overline{A}}\overline{\overline{B}}\overline{\overline{C}} + \overline{\overline{A}}\overline{\overline{B}}\overline{\overline{C}} + \overline{\overline{A}}\overline{\overline{B}}\overline{\overline{C}} = m_1 + m_2 + m_4 + m_7 = \overline{\overline{m_1 \cdot m_2 \cdot m_4 \cdot m_7}}$$

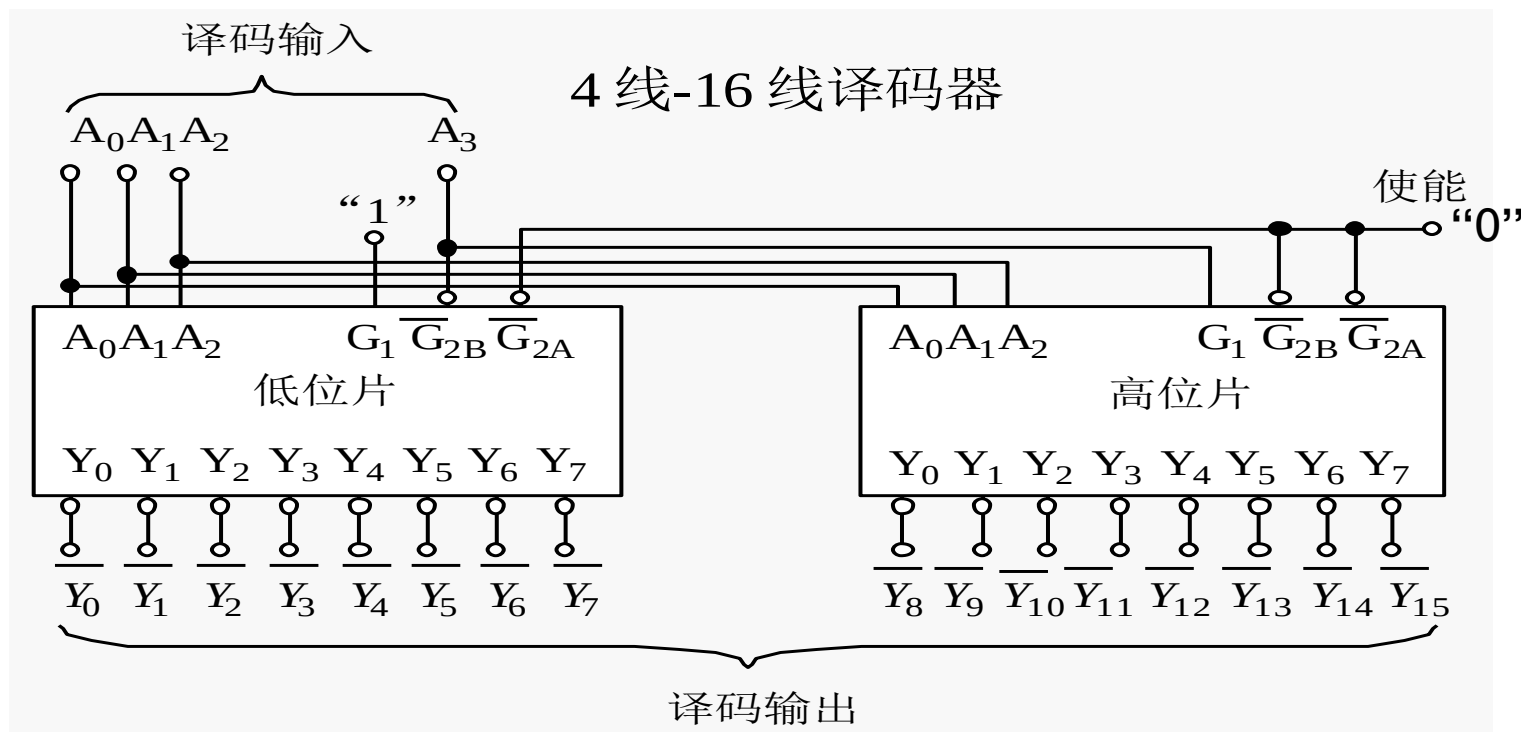
$$F = \overline{\overline{A}}\overline{\overline{B}}\overline{\overline{C}} + \overline{\overline{A}}\overline{\overline{B}}\overline{\overline{C}} + \overline{\overline{A}}\overline{\overline{B}}\overline{\overline{C}} = m_3 + m_5 + m_6 = \overline{\overline{m_3 \cdot m_5 \cdot m_6}}$$

$$G = \overline{\overline{A}}\overline{\overline{B}}\overline{\overline{C}} + \overline{\overline{A}}\overline{\overline{B}}\overline{\overline{C}} + \overline{\overline{A}}\overline{\overline{B}}\overline{\overline{C}} + \overline{\overline{A}}\overline{\overline{B}}\overline{\overline{C}} = m_0 + m_2 + m_4 + m_6 = \overline{\overline{m_0 \cdot m_2 \cdot m_4 \cdot m_6}}$$

用一片74138加三个与非门就可实现该组合逻辑电路。



二进制译码器的级联



输入4位二进制代码中，当高位 $A_3=0$ 时，低位片 $\overline{G_{2A}} + \overline{G_{2B}} = 0$ ， $G_1=1$ 工作；高位片 $G_1=0$ ，禁止。输出 $Y_0 \sim Y_7$ 是 $0A_2A_1A_0$ 的译码；

输入4位二进制代码中，当高位 $A_3=1$ 时，低位片 $\overline{G_{2B}} = 1$ ，禁止；高位片 $G_1=1$ ， $\overline{G_{2A}} = \overline{G_{2B}} = 0$ 工作。输出 $Y_8 \sim Y_{15}$ 是 $1A_2A_1A_0$ 的译码；

8421 BCD码译码器

- **二-十进制译码器**：把二-十进制BCD代码翻译成10个十进制数字信号的电路。
- 二-十进制译码器有4根输入线，10根输出线，又称为4线-10线译码器。

真值表

A_3	A_2	A_1	A_0	Y_9	Y_8	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0
0	0	0	0	0	0	0	0	0	0	0	0	0	1
0	0	0	1	0	0	0	0	0	0	0	0	1	0
0	0	1	0	0	0	0	0	0	0	0	1	0	0
0	0	1	1	0	0	0	0	0	0	1	0	0	0
0	1	0	0	0	0	0	0	0	1	0	0	0	0
0	1	0	1	0	0	0	0	1	0	0	0	0	0
0	1	1	0	0	0	0	1	0	0	0	0	0	0
0	1	1	1	0	0	1	0	0	0	0	0	0	0
1	0	0	0	0	1	0	0	0	0	0	0	0	0
1	0	0	1	1	0	0	0	0	0	0	0	0	0

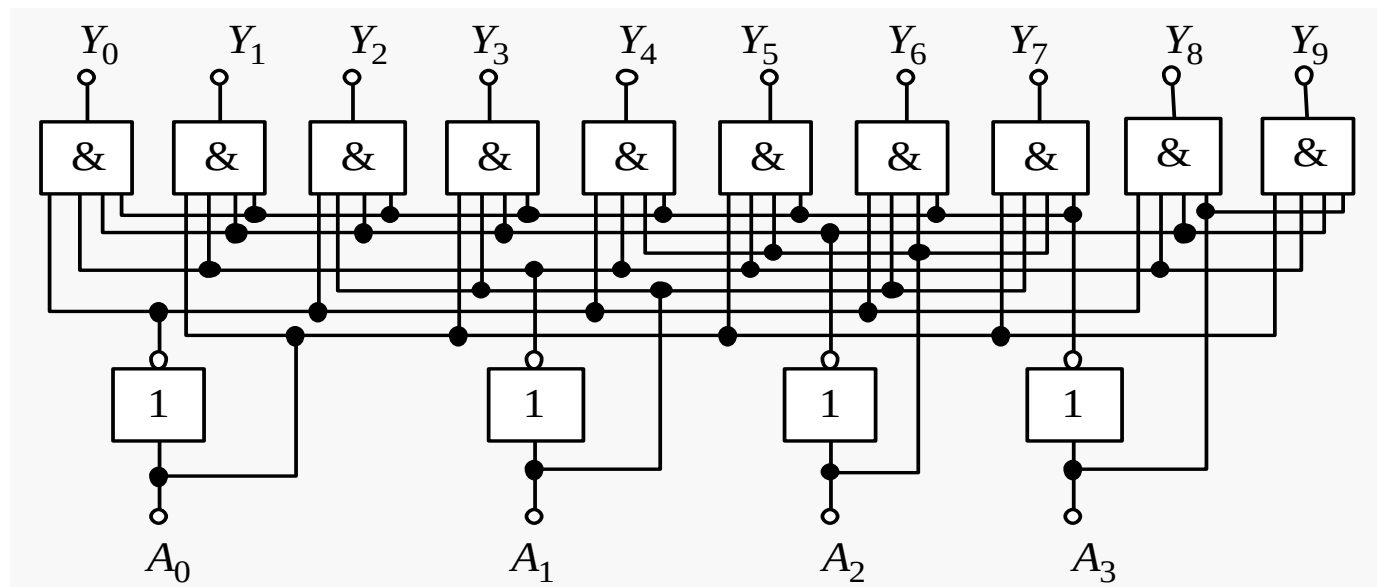
- 1010~1111这六种状态（**伪码**）没有使用，是无效状态，在正常工作状态下不会出现，化简时可以作为约束项处理。对1010~1111这6种状态作为约束项处理后，可以简化电路，但输入信号 $A_3A_2A_1A_0$ 必须遵守约束条件，否则就会出现不全为0的错误输出。
- 在二-十进制译码器中，对BCD码采用了完全译码方案，即输出函数不利用约束项化简。这样，输入端代码出现无效状态时，译码器不予响应，各输入信号之间没有约束。

逻辑表达式

采用完全译码方案

$$\begin{aligned}
 Y_0 &= \bar{A}_3 \bar{A}_2 \bar{A}_1 \bar{A}_0 & Y_1 &= \bar{A}_3 \bar{A}_2 \bar{A}_1 A_0 & Y_2 &= \bar{A}_3 \bar{A}_2 A_1 \bar{A}_0 & Y_3 &= \bar{A}_3 \bar{A}_2 A_1 A_0 \\
 Y_4 &= \bar{A}_3 A_2 \bar{A}_1 \bar{A}_0 & Y_5 &= \bar{A}_3 A_2 \bar{A}_1 A_0 & Y_6 &= \bar{A}_3 A_2 A_1 \bar{A}_0 & Y_7 &= \bar{A}_3 A_2 A_1 A_0 \\
 Y_8 &= A_3 \bar{A}_2 \bar{A}_1 \bar{A}_0 & Y_9 &= A_3 \bar{A}_2 \bar{A}_1 A_0
 \end{aligned}$$

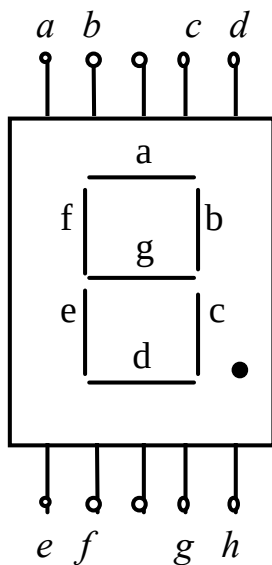
逻辑图



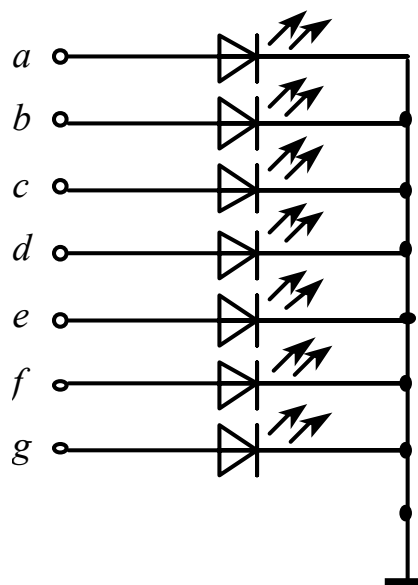
显示译码器

- 显示译码器：驱动各种显示器件，将用二进制代码表示的数字、文字、符号翻译成人们习惯的形式直观地显示出来的电路。

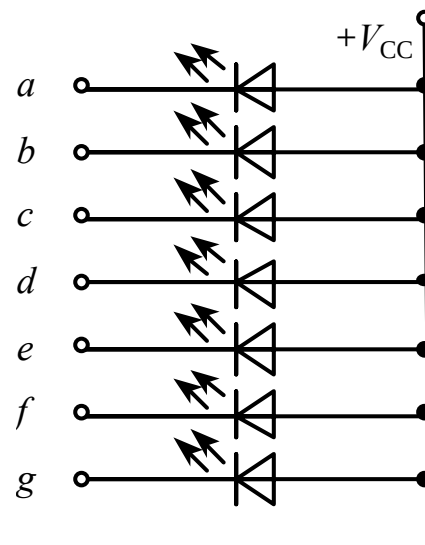
数码显示器



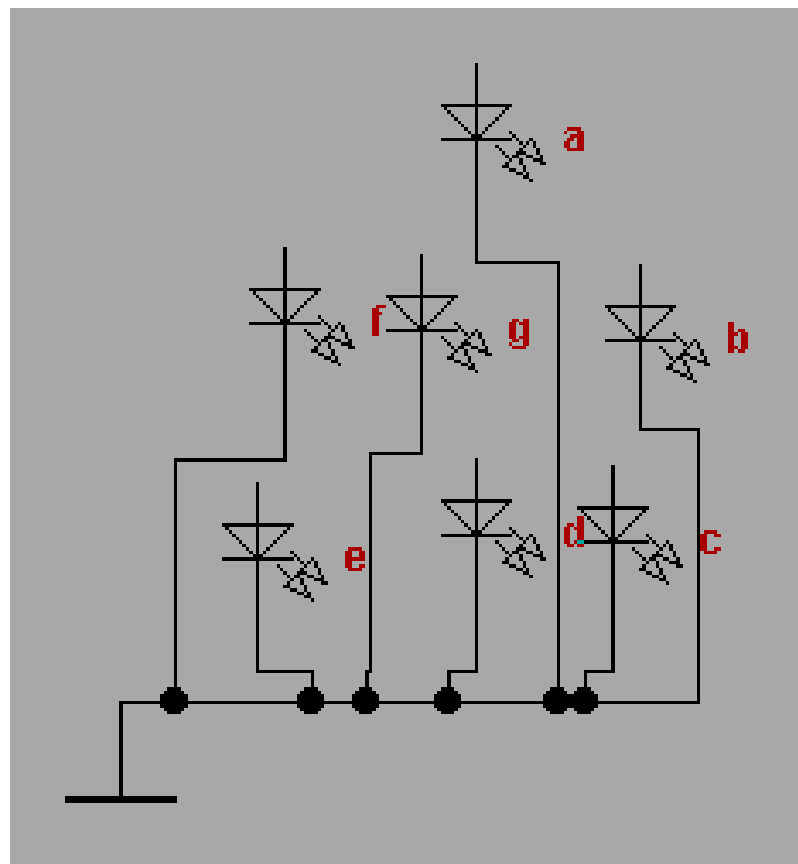
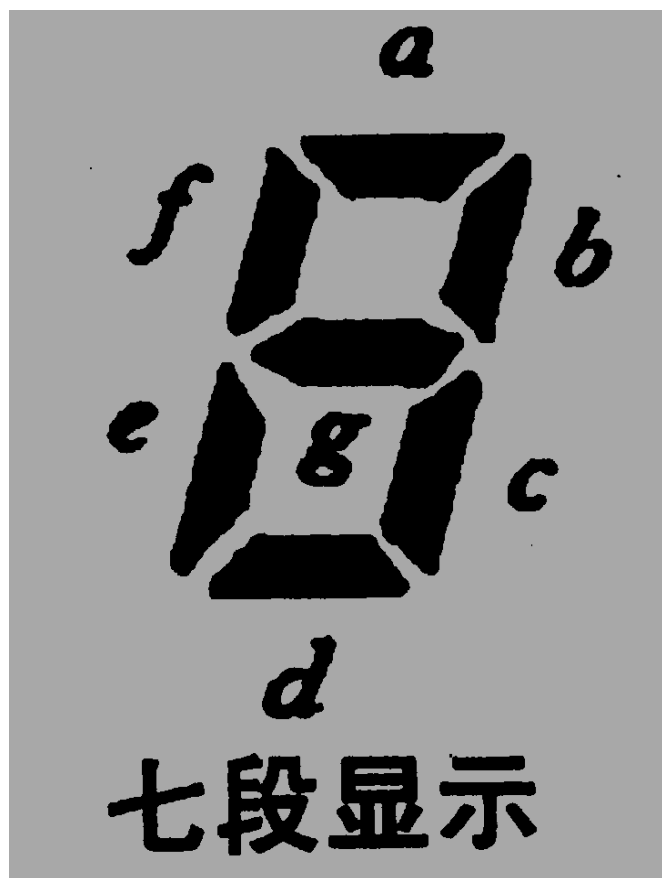
(a) 外形



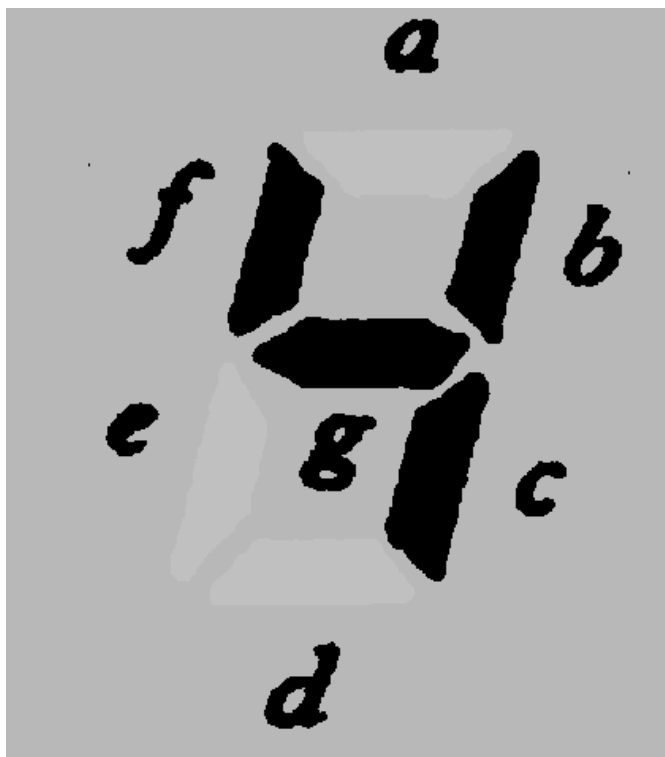
(b) 共阴极



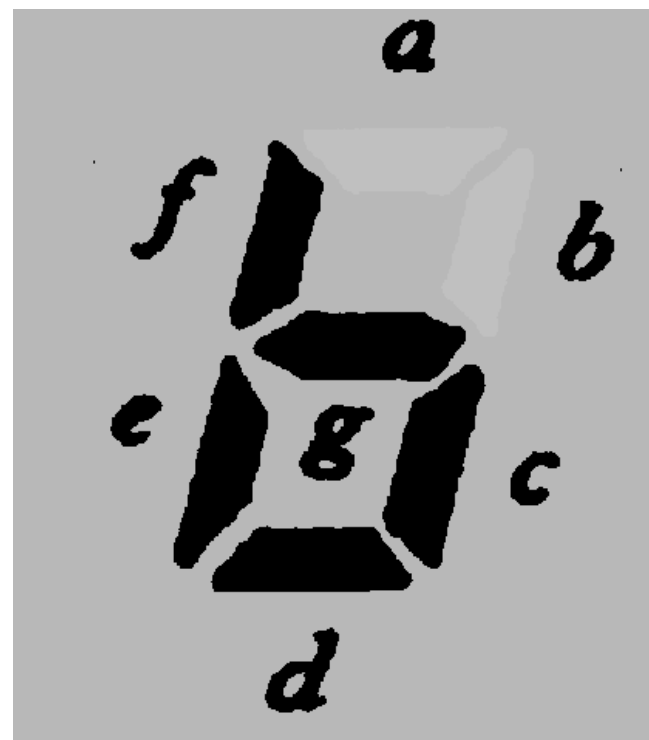
(c) 共阳极



共阴极



$b=c=f=g=1,$
 $a=d=e=0$ 时



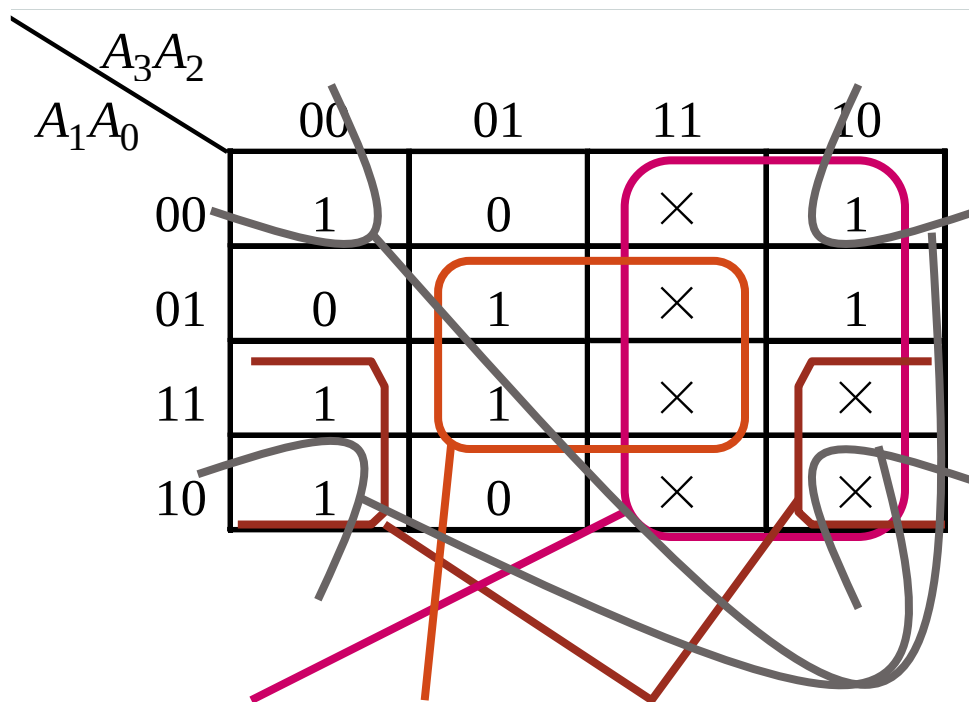
$c=d=e=f=g=1,$
 $a=b=0$ 时

显示译码器真值表

输 入				输 出							显示
A_3	A_2	A_1	A_0	a	b	c	d	e	f	g	字形
0	0	0	0	1	1	1	1	1	1	0	□
0	0	0	1	0	1	1	0	0	0	0	┐
0	0	1	0	1	1	0	1	1	0	1	┌
0	0	1	1	1	1	1	1	0	0	1	└
0	1	0	0	0	1	1	0	0	1	1	┐
0	1	0	1	1	0	1	1	0	1	1	┌
0	1	1	0	0	0	1	1	1	1	1	└
0	1	1	1	1	1	1	0	0	0	0	┐
1	0	0	0	1	1	1	1	1	1	1	┌
1	0	0	1	1	1	1	0	0	1	1	└

真值表仅适用于共阴极LED

a的卡诺图



$$a = A_3 + A_2A_0 + A_1\bar{A}_2 + \bar{A}_2\bar{A}_0$$

输 入				输 出							显示 字形
A_3	A_2	A_1	A_0	a	b	c	d	e	f	g	
0	0	0	0	1	1	1	1	1	1	0	0
0	0	0	1	0	1	1	0	0	0	0	1
0	0	1	0	1	1	0	1	1	0	1	2
0	0	1	1	1	1	1	1	0	0	1	3
0	1	0	0	0	1	1	0	0	1	1	4
0	1	0	1	1	0	1	1	0	1	1	5
0	1	1	0	0	0	1	1	1	1	1	6
0	1	1	1	1	1	1	0	0	0	0	7
1	0	0	0	1	1	1	1	1	1	1	8
1	0	0	1	1	1	1	0	0	1	1	9

输 入				输 出								显示 字形
A_3	A_2	A_1	A_0	a	b	c	d	e	f	g		
0	0	0	0	1	1	1	1	1	1	0	0	
0	0	0	1	0	1	1	0	0	0	0	1	
0	0	1	0	1	1	0	1	1	0	1	2	
0	0	1	1	1	1	1	1	0	0	1	3	
0	1	0	0	0	1	1	0	0	1	1	4	
0	1	0	1	1	0	1	1	0	1	1	5	
0	1	1	0	0	0	1	1	1	1	1	6	
0	1	1	1	1	1	1	0	0	0	0	7	
1	0	0	0	1	1	1	1	1	1	1	8	
1	0	0	1	1	1	1	0	0	1	1	9	

b的卡诺图

A_3A_2		A_1A_0			
		00	01	11	10
A_1A_0	00	1	1	×	1
	01	1	0	×	1
	11	1	1	×	×
	10	1	0	×	×

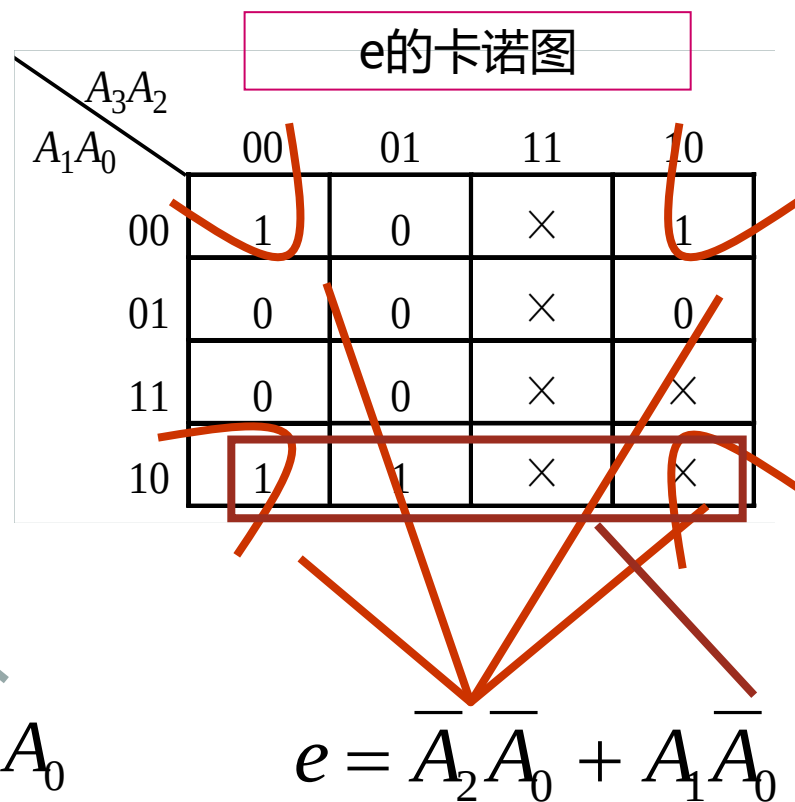
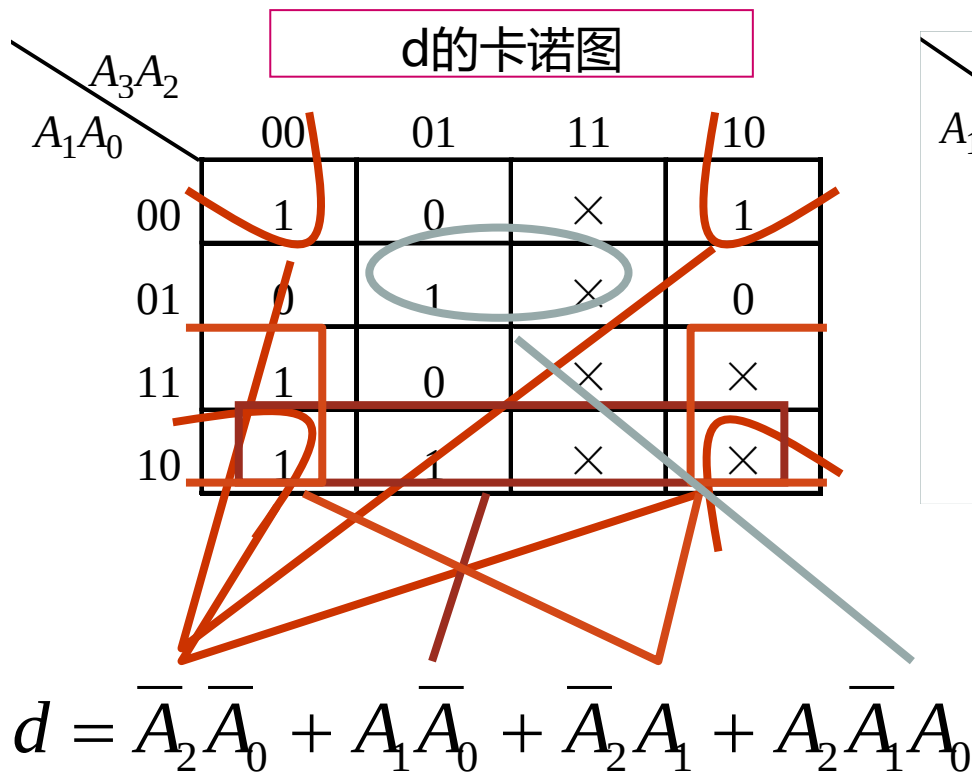
$$b = \bar{A}_2 + \bar{A}_1\bar{A}_0 + A_1A_0$$

c的卡诺图

A_3A_2		A_1A_0			
		00	01	11	10
A_1A_0	00	1	1	×	1
	01	1	1	×	1
	11	1	1	×	×
	10	0	1	×	×

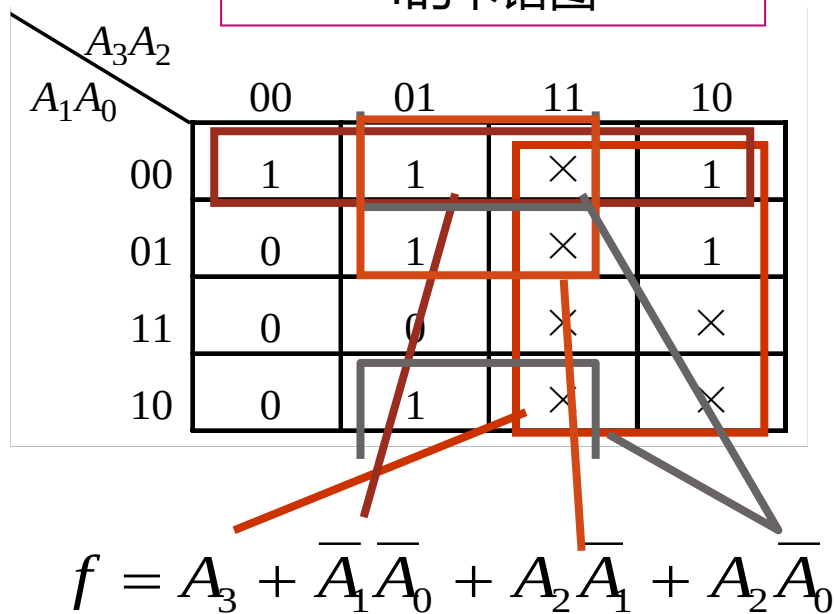
$$c = A_2 + \bar{A}_1 + A_0$$

输 入				输 出								显示 字形
A_3	A_2	A_1	A_0	a	b	c	d	e	f	g		
0	0	0	0	1	1	1	1	1	1	0		0
0	0	0	1	0	1	1	0	0	0	0		1
0	0	1	0	1	1	0	1	1	0	1		2
0	0	1	1	1	1	1	1	0	0	1		3
0	1	0	0	0	1	1	0	0	1	1		4
0	1	0	1	1	0	1	1	0	1	1		5
0	1	1	0	0	0	1	1	1	1	1		6
0	1	1	1	1	1	1	0	0	0	0		7
1	0	0	0	1	1	1	1	1	1	1		8
1	0	0	1	1	1	1	0	0	1	1		9

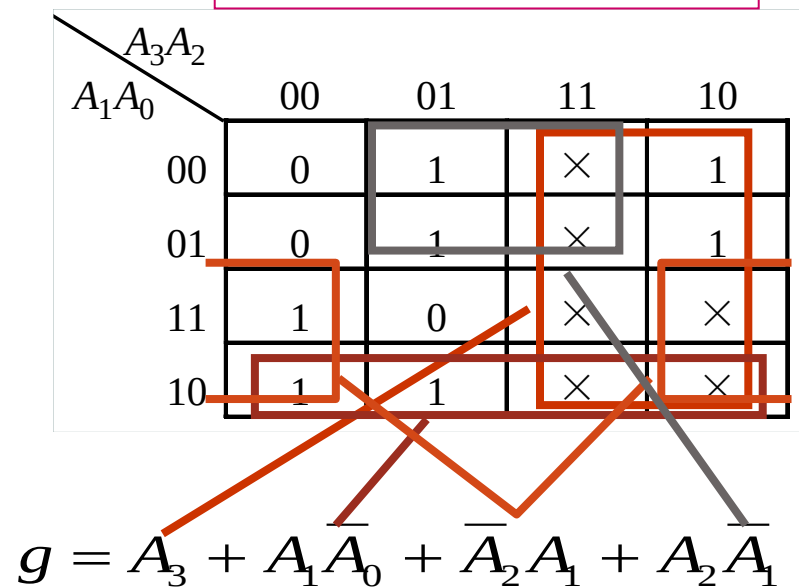


输 入				输 出							显示 字形
A_3	A_2	A_1	A_0	a	b	c	d	e	f	g	
0	0	0	0	1	1	1	1	1	1	0	0
0	0	0	1	0	1	1	0	0	0	0	1
0	0	1	0	1	1	0	1	1	0	1	2
0	0	1	1	1	1	1	1	0	0	1	3
0	1	0	0	0	1	1	0	0	1	1	4
0	1	0	1	1	0	1	1	0	1	1	5
0	1	1	0	0	0	1	1	1	1	1	6
0	1	1	1	1	1	1	0	0	0	0	7
1	0	0	0	1	1	1	1	1	1	1	8
1	0	0	1	1	1	1	0	0	1	1	9

f的卡诺图



g的卡诺图



逻辑表达式

$$a = A_3 + A_2 A_0 + A_1 \bar{A}_2 + \bar{A}_2 \bar{A}_0$$

$$b = \bar{A}_2 + \bar{A}_1 \bar{A}_0 + A_1 A_0$$

$$c = A_2 + \bar{A}_1 + A_0$$

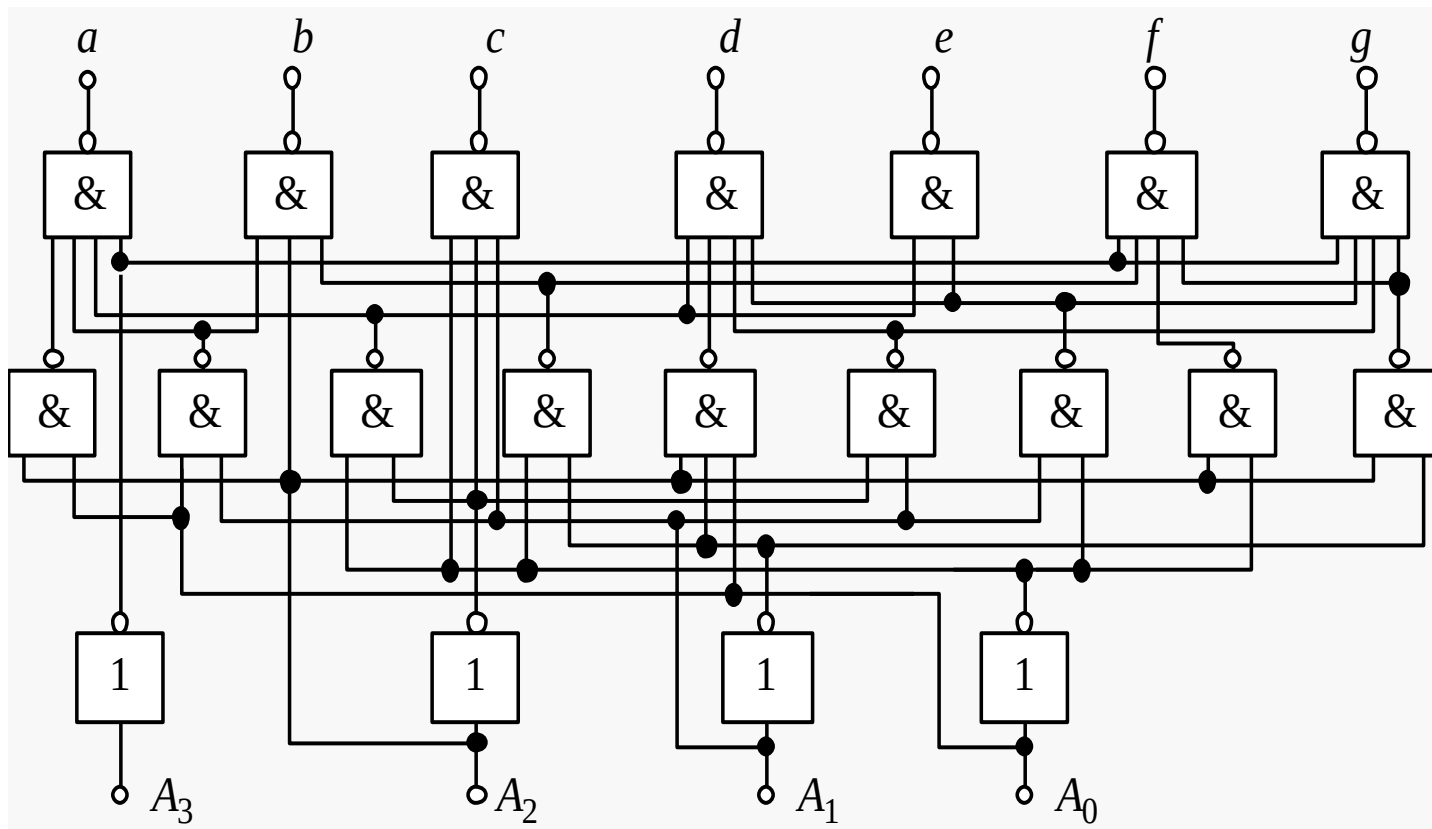
$$d = \bar{A}_2 \bar{A}_0 + A_1 \bar{A}_0 + \bar{A}_2 A_1 + A_2 \bar{A}_1 A_0$$

$$e = \bar{A}_2 \bar{A}_0 + A_1 \bar{A}_0$$

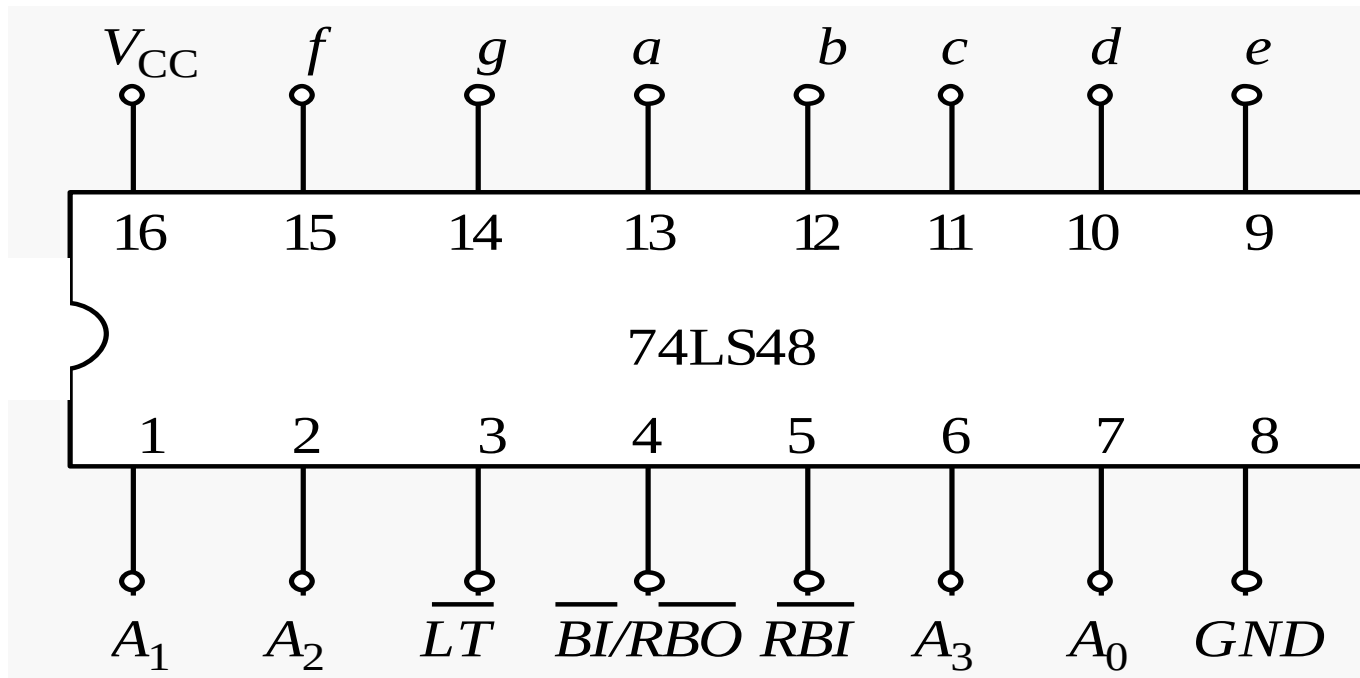
$$f = A_3 + \bar{A}_1 \bar{A}_0 + A_2 \bar{A}_1 + A_2 \bar{A}_0$$

$$g = A_3 + A_1 \bar{A}_0 + \bar{A}_2 A_1 + A_2 \bar{A}_1$$

逻辑图



集成7段显示译码器引脚排列图



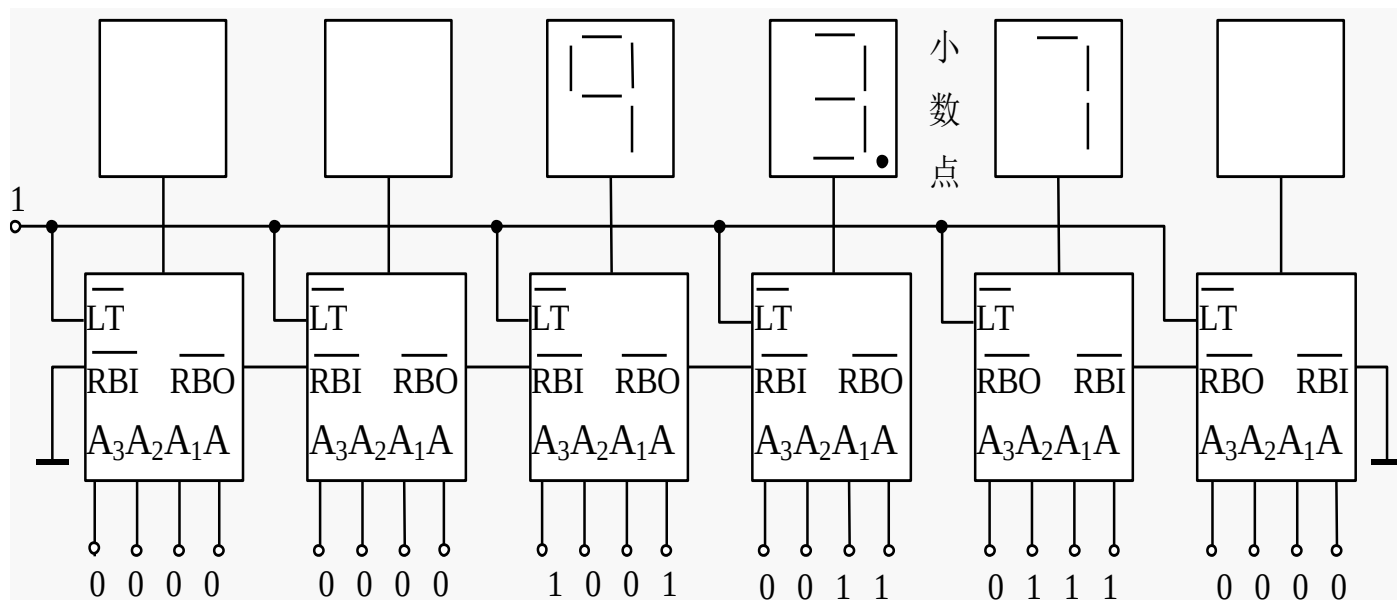
试灯输入端 $\overline{LT}$ ：当 $\overline{LT}=0$ 时，数码管的7段全亮，与输入的译码信号无关。该端用于测试数码管的好坏。

动态灭零输入端 $\overline{RBI}$ ：当 $\overline{LT}=1$ 、 $\overline{RBI}=0$ 、且译码输入全为0时，对应输出的0字型不显示，即0字被熄灭；当译码输入不全为0时，对应的输出字形正常显示。本输入端可以用于消隐无效的0。如数据0034.50可显示为34.5。

灭灯输入/动态灭零输出端 $\overline{BI} / \overline{RBO}$ ：这是一个特殊的端钮，灭灯时用作输入，动态灭零时用作输出。当作为输入且值为0时，数码管7段全灭，与译码输入无关。当作为输出使用时，受控于 $\overline{LT}$ 和 $\overline{RBI}$ 及译码输入：当 $\overline{LT}=1$ ， $\overline{RBI}=0$ 且译码输入为0000时，该端输出为0；其他情况该端输出为1。本端钮主要用于显示多位数字时，多个译码器之间的连接。

功能或 十进制数	输 入		输 出											
	$\overline{LT}$	$\overline{RBI}$	A_3	A_2	A_1	A_0	$\overline{BI} / \overline{RBO}$	a	b	c	d	e	f	g
$\overline{BI} / \overline{RBO}$ (灭灯)	×	×	×	×	×	×	0(输入)	0	0	0	0	0	0	0
$\overline{LT}$ (试灯)	0	×	×	×	×	×	1	1	1	1	1	1	1	1
$\overline{RBI}$ (动态灭零)	1	0	0	0	0	0	0 (输出)	0	0	0	0	0	0	0
0	1	1	0	0	0	0	1	1	1	1	1	1	0	0
1	1	×	0	0	0	1	1	0	1	1	0	0	0	0
2	1	×	0	0	1	0	1	1	1	0	1	1	0	1
3	1	×	0	0	1	1	1	1	1	1	1	0	0	1
4	1	×	0	1	0	0	1	0	1	1	0	0	1	1
5	1	×	0	1	0	1	1	1	0	1	1	0	1	1
6	1	×	0	1	1	0	1	0	0	1	1	1	1	1
7	1	×	0	1	1	1	1	1	1	1	0	0	0	0
8	1	×	1	0	0	0	1	1	1	1	1	1	1	1
9	1	×	1	0	0	1	1	1	1	1	0	0	1	1
10	1	×	1	0	1	0	1	0	0	0	1	1	0	1
11	1	×	1	0	1	1	1	0	0	1	1	0	0	1
12	1	×	1	1	0	0	1	0	1	0	0	0	1	1
13	1	×	1	1	0	1	1	1	0	0	1	0	1	1
14	1	×	1	1	1	0	1	0	0	0	1	1	1	1
15	1	×	1	1	1	1	1	0	0	0	0	0	0	0

例: 数码显示电路的动态灭零



整数部分：高位的 $\overline{BI} / \overline{RBO}$ 与低位的 $\overline{RBI}$ 相连

小数部分：低位的 $\overline{BI} / \overline{RBO}$ 与高位的 $\overline{RBI}$ 相连

动态灭零输入端 $\overline{RBI}$ ：当试灯输入端 $\overline{LT} = 1$ 、 $\overline{RBI} = 0$ 、且译码输入全为0时，对应输出的0字型不显示，即0字被熄灭；当译码输入不全为0时，对应的输出字形正常显示。本输入端可以用于消隐无效的0。如数据0093.70可显示为93.7。

灭灯输入/动态灭零输出端 $\overline{BI} / \overline{RBO}$ ：动态灭零时用作输出，使用时受控于 $\overline{LT}$ 和 $\overline{RBI}$ 及译码输入，当 $\overline{LT} = 1$ ， $\overline{RBI} = 0$ 且译码输入为0000时，该端输出为0；其他情况该端输出为1。

3.4 数据选择器和分配器

3.4.1 数据选择器

3.4.2 数据分配器

3.4.1 数据选择器

数据选择器：在多路数据传送过程中，能够根据需要将其任一路挑选出来的电路。

4选1数据选择器真值表

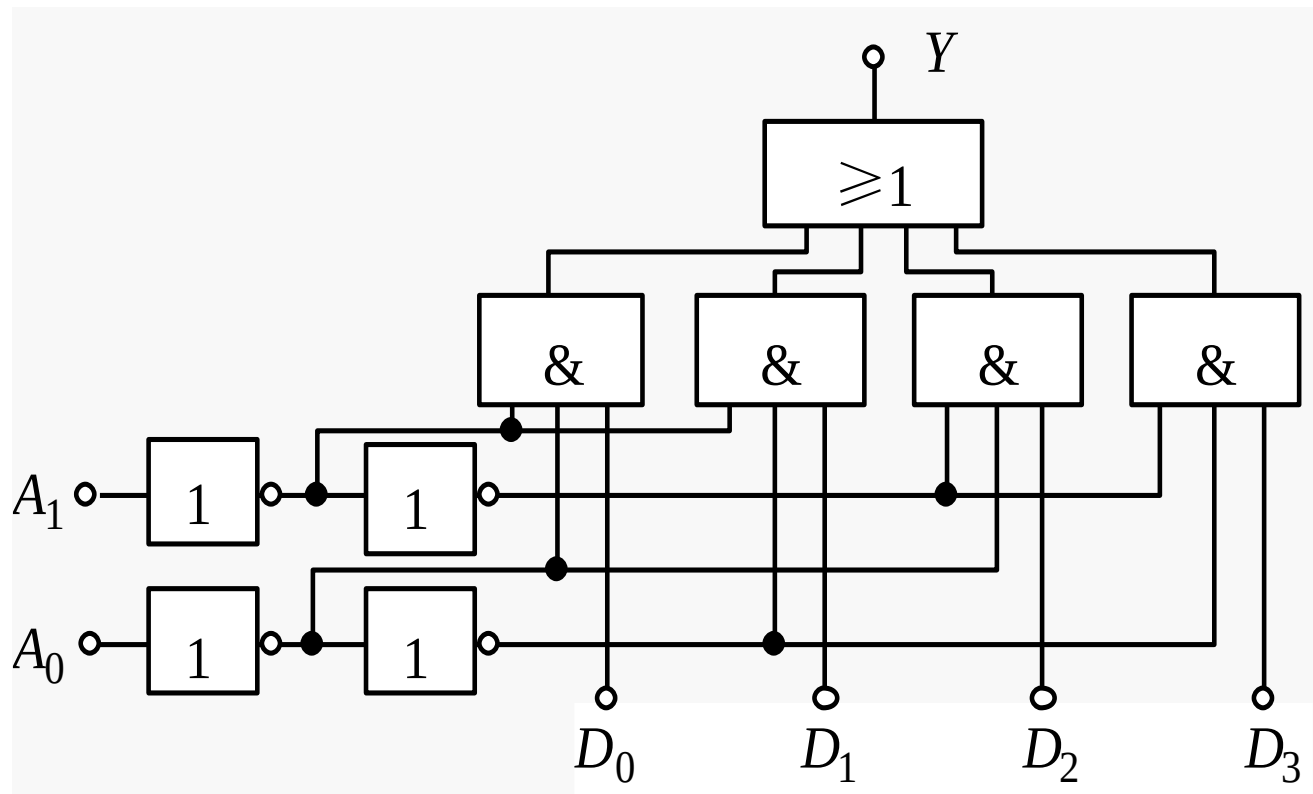
输入数据	输入		输出
	D	$A_1 A_0$	Y
	D_0	0 0	D_0
	D_1	0 1	D_1
	D_2	1 0	D_2
	D_3	1 1	D_3

由地址码决定从 4 路输入中选择哪 1 路输出。

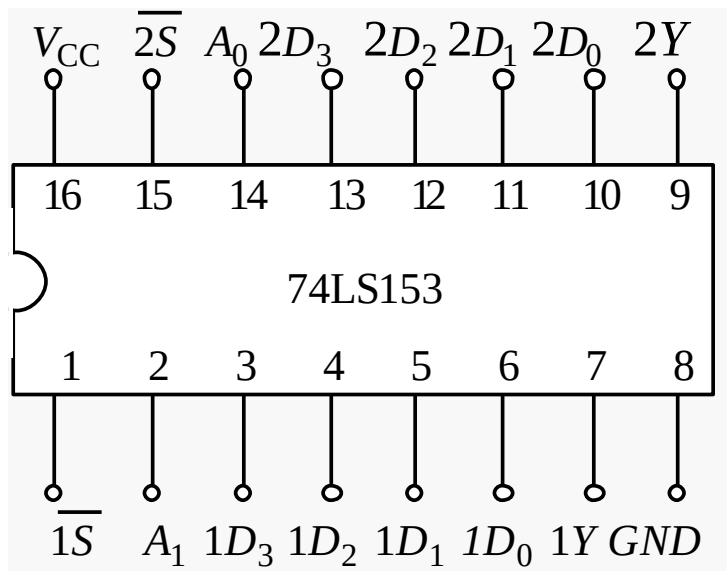
逻辑表达式

$$Y = D_0 \bar{A}_1 \bar{A}_0 + D_1 \bar{A}_1 A_0 + D_2 A_1 \bar{A}_0 + D_3 A_1 A_0 = \sum_{i=0}^3 D_i m_i$$

逻辑图



集成双4选1数据选择器74LS153



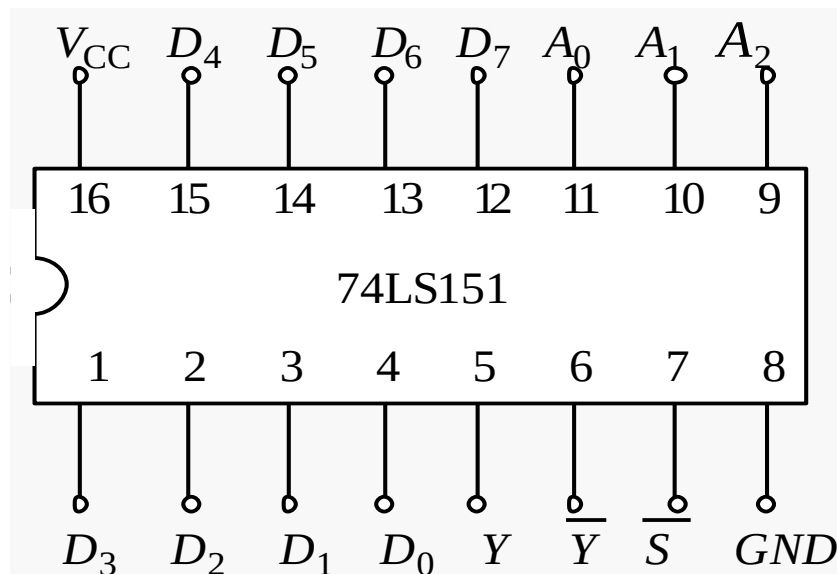
输 入				输 出
S	D	A_1	A_0	Y
1	×	×	×	0
0	D_0	0	0	D_0
0	D_1	0	1	D_1
0	D_2	1	0	D_2
0	D_3	1	1	D_3

选通控制端 $\bar{S}$ 为低电平有效，即 $\bar{S} = 0$ 时芯片被选中，处于工作状态；
 $\bar{S} = 1$ 时芯片被禁止， $Y \equiv 0$ 。

74LS151的真值表

输 入					输 出	
D	A_2	A_1	A_0	$\bar{S}$	Y	$\bar{Y}$
$\times$	$\times$	$\times$	$\times$	1	0	1
D_0	0	0	0	0	D_0	$\bar{D}_0$
D_1	0	0	1	0	D_1	$\bar{D}_1$
D_2	0	1	0	0	D_2	$\bar{D}_2$
D_3	0	1	1	0	D_3	$\bar{D}_3$
D_4	1	0	0	0	D_4	$\bar{D}_4$
D_5	1	0	1	0	D_5	$\bar{D}_5$
D_6	1	1	0	0	D_6	$\bar{D}_6$
D_7	1	1	1	0	D_7	$\bar{D}_7$

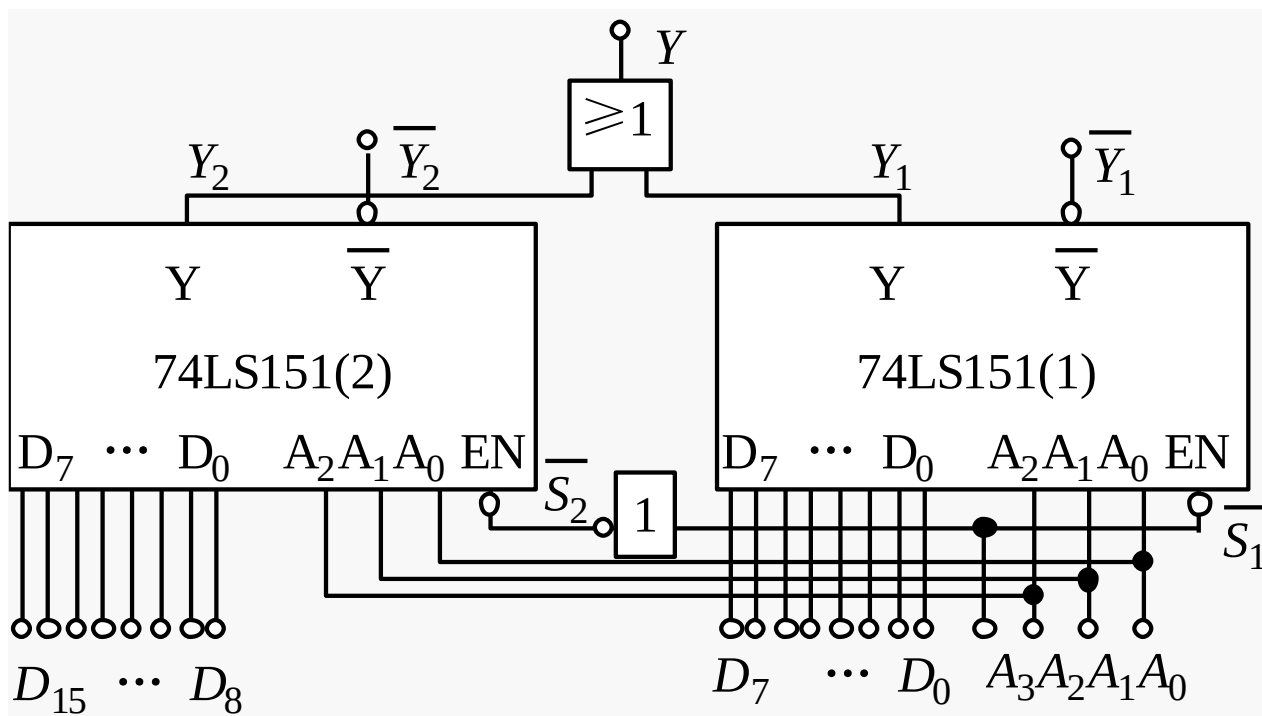
集成8选1数据选择器74LS151



$\bar{S} = 1$ 时，选择器被禁止，无论地址码是什么， Y 总是等于 0

$$\bar{S} = 0 \text{ 时 } \left\{ \begin{array}{l} Y = D_0 \bar{A}_2 \bar{A}_1 \bar{A}_0 + D_1 \bar{A}_2 \bar{A}_1 A_0 + \cdots + D_7 A_2 A_1 A_0 = \sum_{i=0}^7 D_i m_i \\ \bar{Y} = \bar{D}_0 \bar{A}_2 \bar{A}_1 \bar{A}_0 + \bar{D}_1 \bar{A}_2 \bar{A}_1 A_0 + \cdots + \bar{D}_7 A_2 A_1 A_0 = \sum_{i=0}^7 \bar{D}_i m_i \end{array} \right.$$

数据选择器的扩展



$A_3=0$ 时, $\overline{S}_1=0$ 、 $\overline{S}_2=1$, 片(2)禁止、片(1)工作

$A_3=1$ 时, $\overline{S}_1=1$ 、 $\overline{S}_2=0$, 片(1)禁止、片(2)工作

用数据选择器实现逻辑函数

- 数据选择器的主要特点：

(1) 具有n个地址变量的 2^n 选1数据选择器，输出信号Y为标准与或表达式的形式，即：

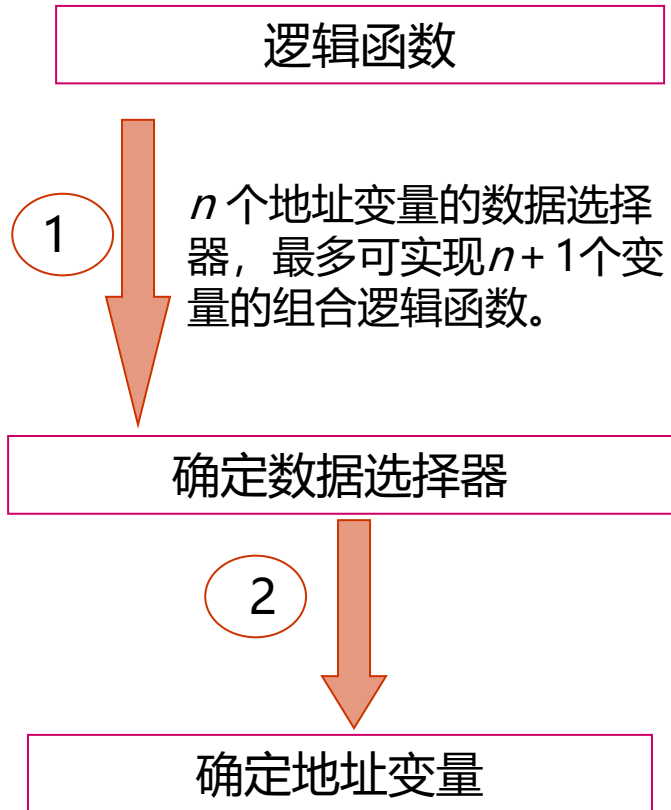
$$Y = \sum_{i=0}^{2^n-1} D_i m_i$$

(2) 表达式中提供了地址变量的全部最小项。

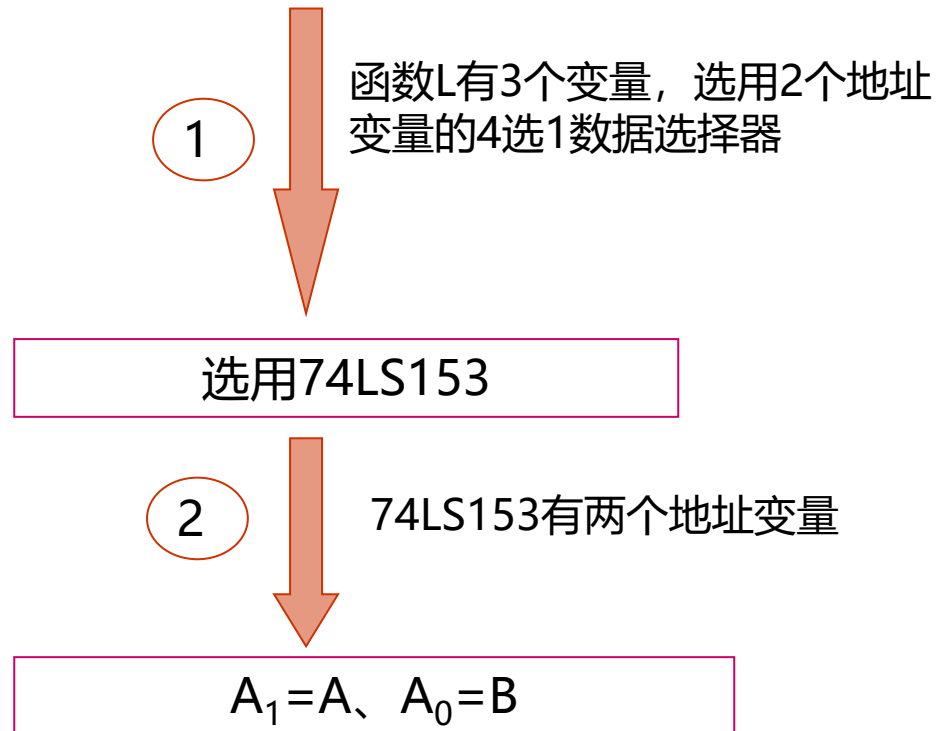
(3) 当 $D_i=1$ 时，对应的 m_i 在上式中出现，当 $D_i=0$ 时， m_i 不出现。

- 任何组合逻辑函数总可以用最小项之和的标准形式表示。
- 利用数据选择器的输入 D_i 来选择地址变量组成的最小项 m_i ，可以实现任何所需的组合逻辑函数。

基本步骤



$$L = \overline{A}\overline{B}C + \overline{A}B\overline{C} + AB$$



3

求Di

3

(1) 公式法

函数的标准与或表达式:

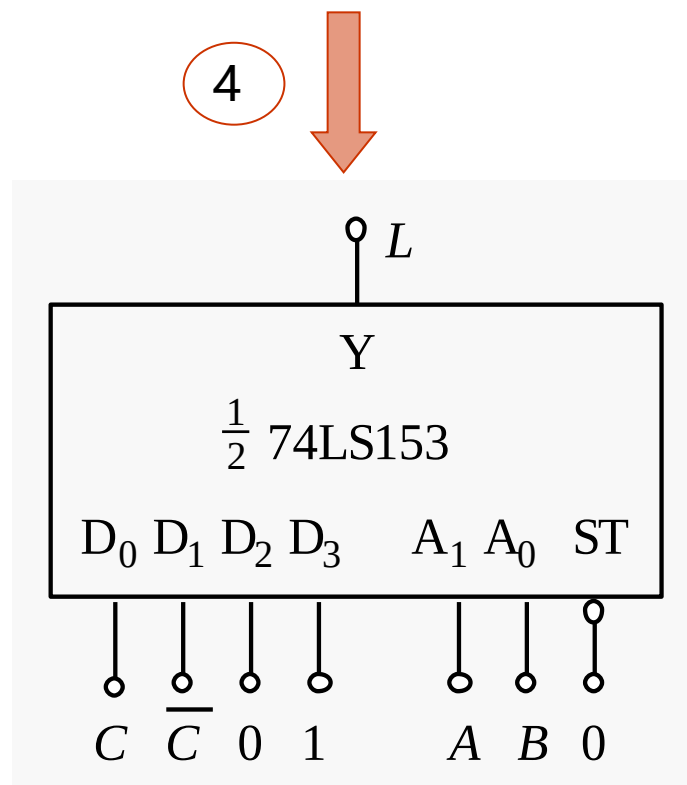
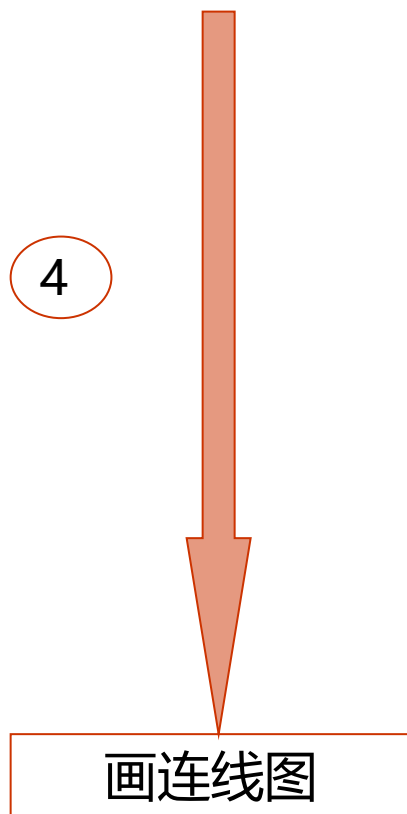
$$\begin{aligned} L &= \overline{A}\overline{B}C + \overline{A}B\overline{C} + AB \\ &= m_0C + m_1\overline{C} + m_2 \cdot 0 + m_3 \cdot 1 \end{aligned}$$

4选1数据选择器输出信号的表达式:

$$Y = m_0D_0 + m_1D_1 + m_2D_2 + m_3D_3$$

比较L和Y, 得:

$$D_0 = C, D_1 = \overline{C}, D_2 = 0, D_3 = 1$$



(2) 真值表法求 D_i

$$L = \overline{A}\overline{B}C + \overline{A}B\overline{C} + AB$$

m_i	A	B	C	L
m_0	0	0	0	0
	0	0	1	1
m_1	0	1	0	1
	0	1	1	0
m_2	1	0	0	0
	1	0	1	0
m_3	1	1	0	1
	1	1	1	1

$C=1$ 时 $L=1$, 故 $D_0=C$

$C=0$ 时 $L=1$, 故 $D_1=\overline{C}$

$L\equiv 0$, 故 $D_2=0$

$L\equiv 1$, 故 $D_3=1$

(3) 图形法求 D_i

AB		00	01	11	10
C					
0		0	1	1	0
1		1	0	1	0

Diagram illustrating the graphical method to find D_i from a truth table. The truth table is shown with columns for AB (00, 01, 11, 10) and rows for C (0, 1). The output values are circled in pink. Below the table, the output values are mapped to D_i boxes:

- D_0 (bottom-left) corresponds to the value 0 in the cell $(C=0, AB=00)$.
- D_1 (bottom-middle-left) corresponds to the value 1 in the cell $(C=0, AB=01)$.
- D_3 (bottom-middle-right) corresponds to the value 1 in the cell $(C=0, AB=11)$.
- D_2 (bottom-right) corresponds to the value 0 in the cell $(C=0, AB=10)$.

$$D_0 = C, D_1 = \bar{C}, D_2 = 0, D_3 = 1$$

例：用数据选择器实现函数：

$$L(A, B, C, D) = \sum m(0, 3, 4, 5, 9, 10, 11, 12, 13)$$

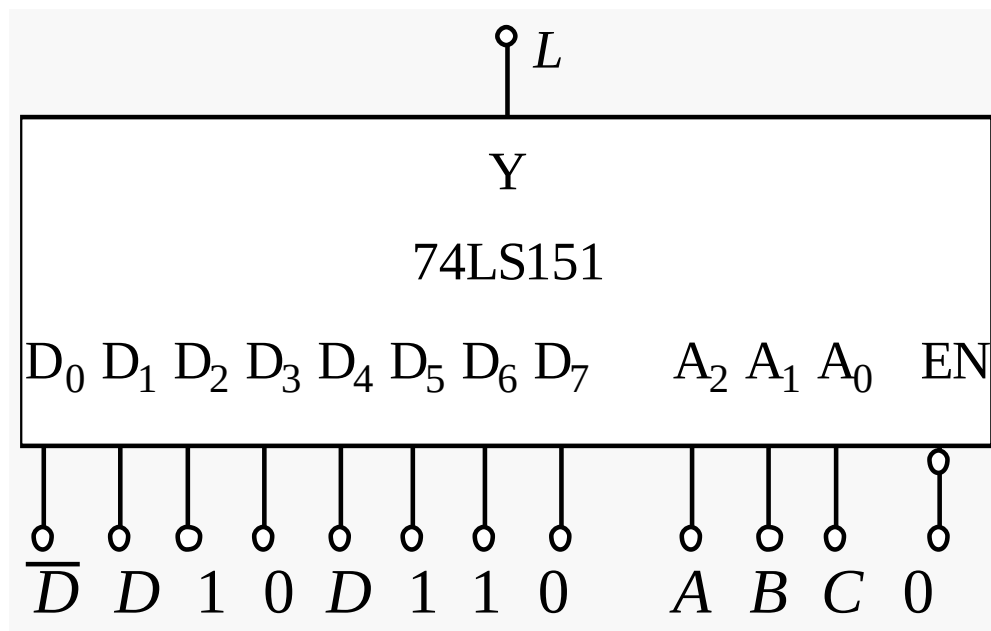
①因为L有4个变量，所以选用3地址变量的8选1数据选择器74LS151

②设 $A_2=A$ 、 $A_1=B$ 、 $A_0=C$

③用图形法求 D_i

		$D0 = \bar{D}$ $D2 = 1$ $D6 = 1$ $D4 = D$			
		00	01	11	10
CD	00	1	1	1	0
	01	0	1	1	1
	11	1	0	0	1
	10	0	0	0	1
		$D1 = D$	$D3 = 0$	$D7 = 0$	$D5 = 1$

④画连线图



3.4.2 数据分配器

数据分配器：能将1个输入数据，根据需要传送到m个输出端的任何一个输出端的电路。

1路-4路数据分配器真值表

输 入		输 出			
A_1	A_0	Y_0	Y_1	Y_2	Y_3
0	0	D	0	0	0
0	1	0	D	0	0
1	0	0	0	D	0
1	1	0	0	0	D

输入数据

D

地址码：决定将输入数据D送给哪1路输出。

逻辑表达式

$$Y_0 = D\bar{A}_1\bar{A}_0$$

$$Y_1 = D\bar{A}_1A_0$$

$$Y_2 = DA_1\bar{A}_0$$

$$Y_3 = DA_1A_0$$

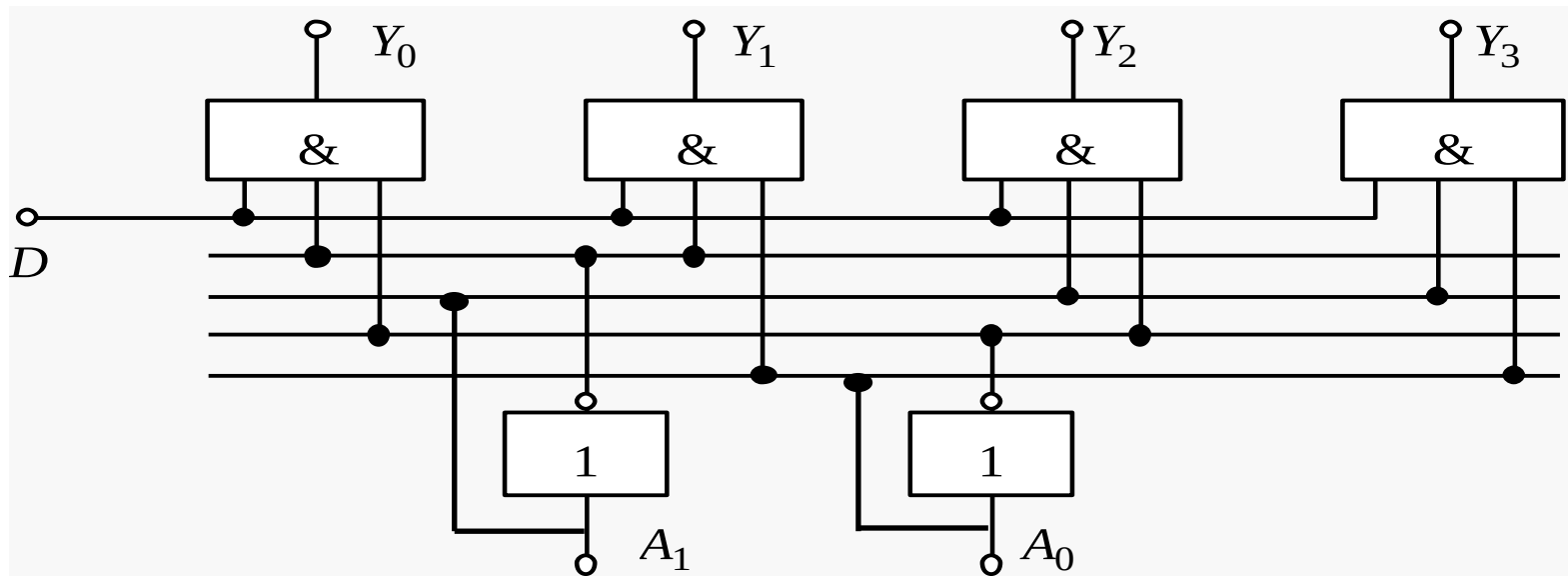
逻辑图

$$Y_0 = D\bar{A}_1\bar{A}_0$$

$$Y_1 = D\bar{A}_1A_0$$

$$Y_2 = DA_1\bar{A}_0$$

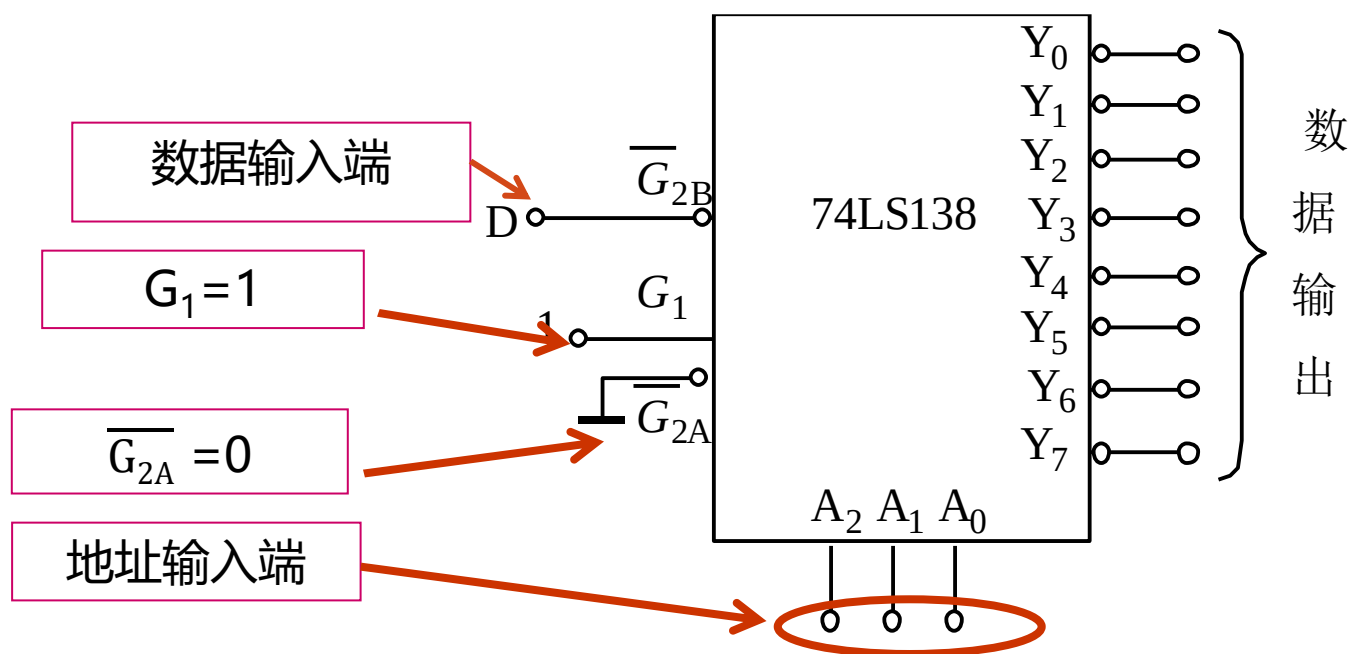
$$Y_3 = DA_1A_0$$



集成数据分配器

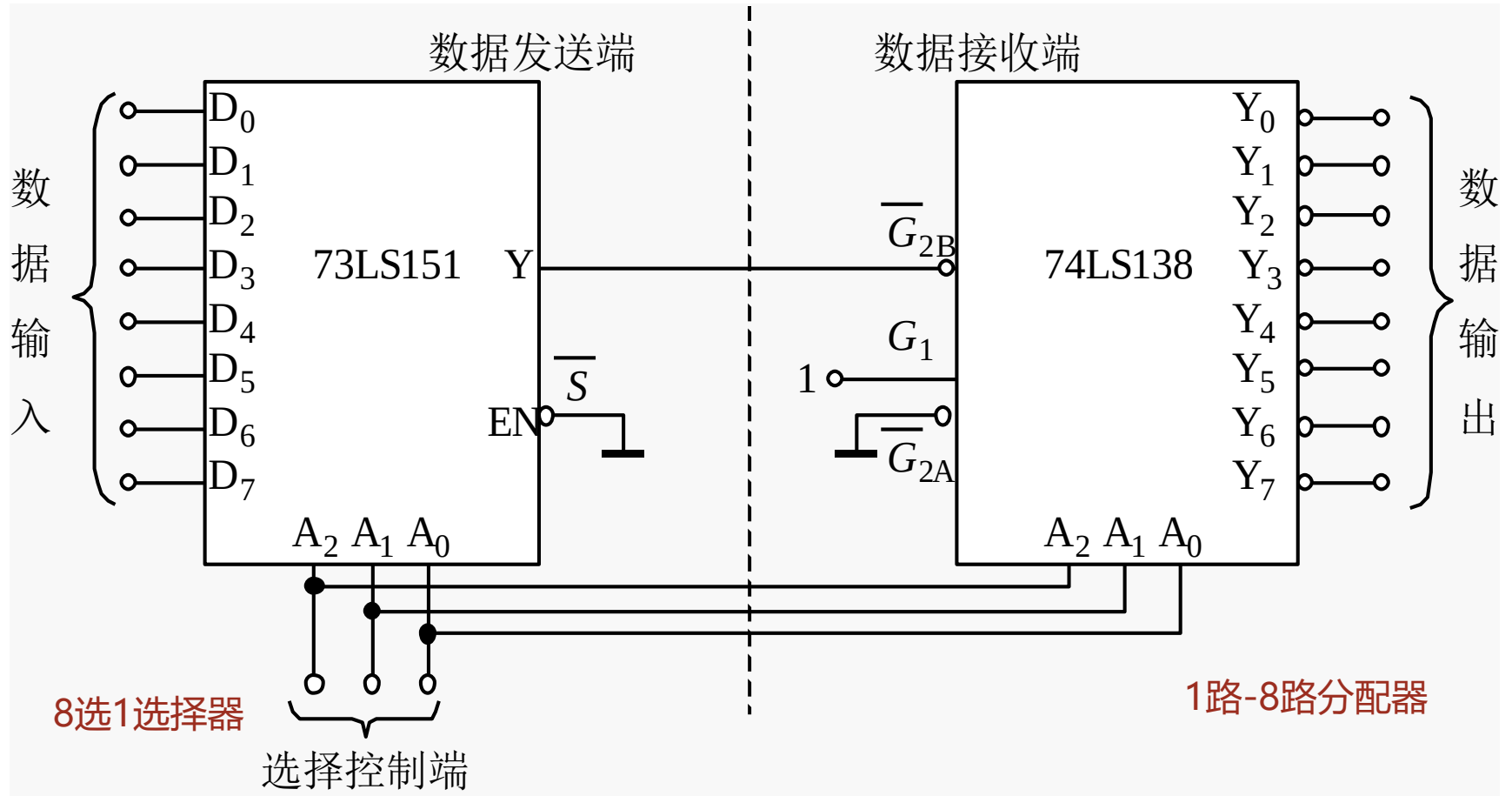
把二进制译码器的使能端作为数据输入端，二进制代码输入端作为地址码输入端，则带使能端的二进制译码器，就是能够传输 0 有效的数据分配器。

由74LS138构成的1路-8路数据分配器

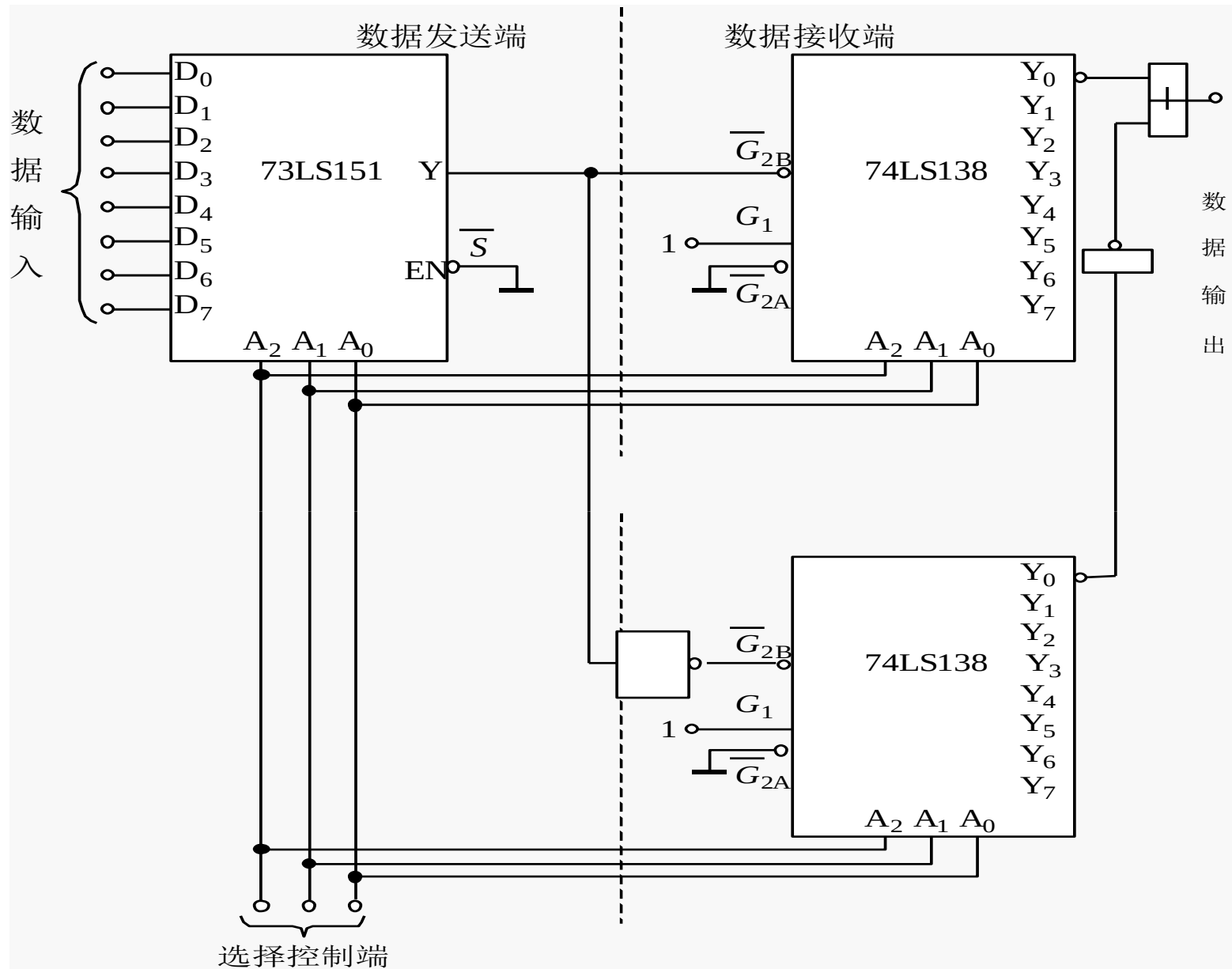


数据分配器的应用

数据分配器和数据选择器一起构成 0 有效数据分时传送系统



数据分时传送系统



3.5 只读存储器ROM

3.5.1 ROM的结构及工作原理

3.5.2 ROM的应用举例及容量扩展

存储器的分类：

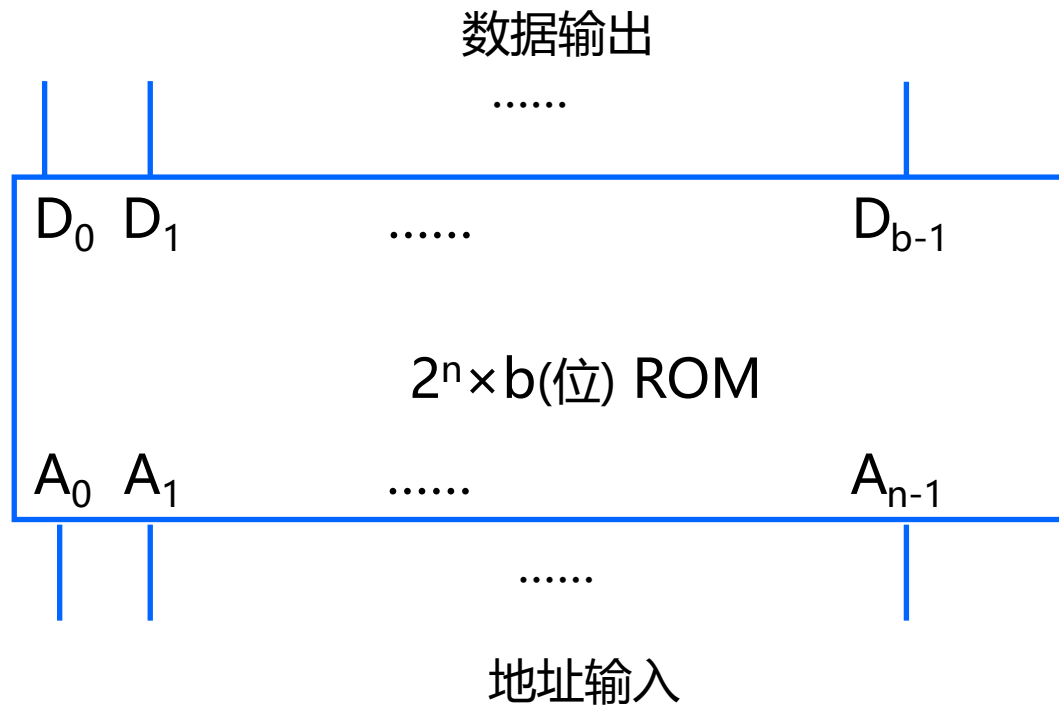
- RAM：在工作时既能从中读出（取出）信息，又能随时写入（存入）信息，但断电后所存信息消失。
- ROM：在工作时只能从中读出信息，不能写入信息，且断电后其所存信息仍能保持。

ROM的分类

- 掩膜ROM：不能改写。
- PROM：只能改写一次。
- EPROM：可以改写多次。

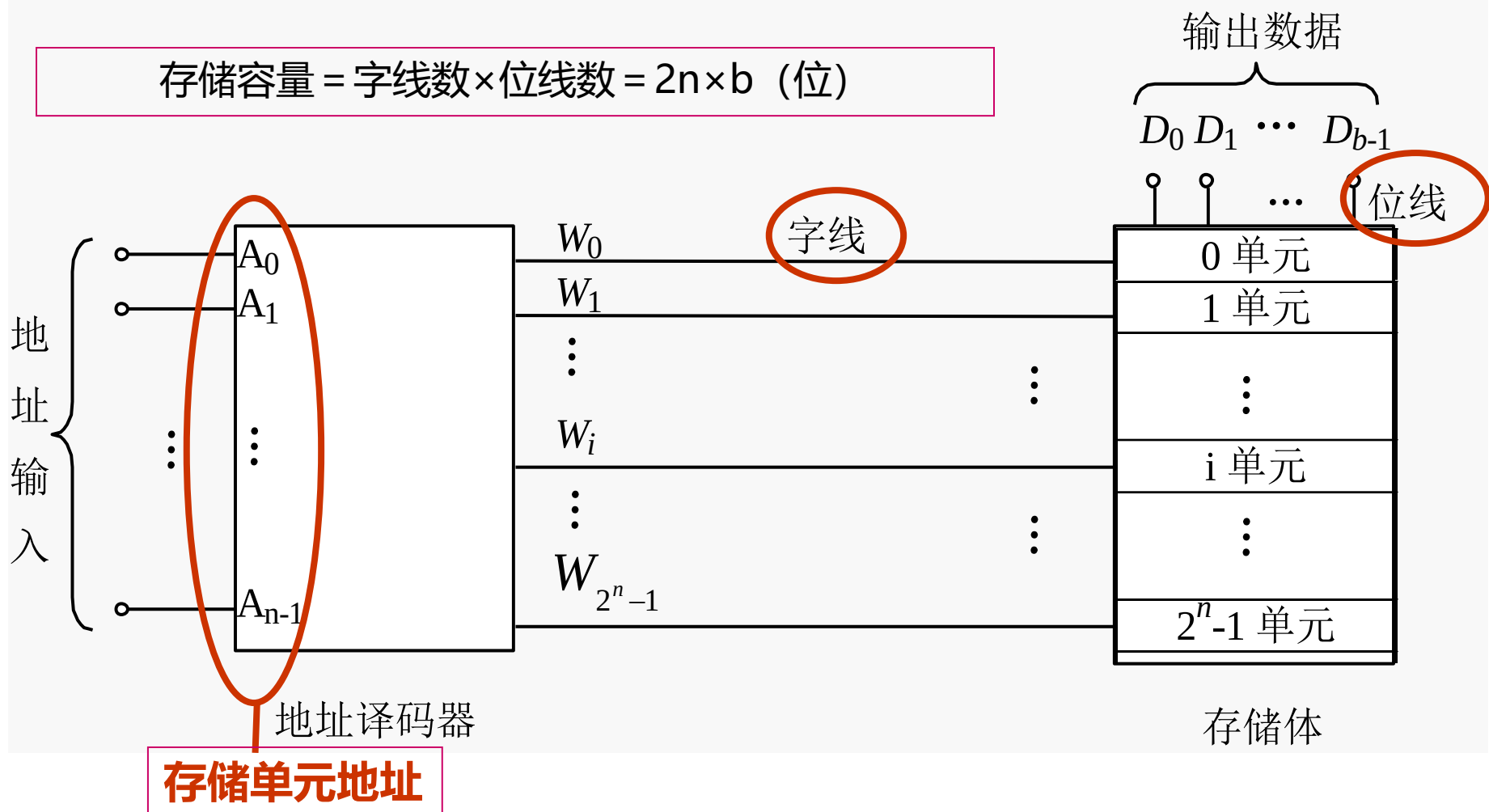
3.5.1 ROM的结构及工作原理

ROM的结构示意图



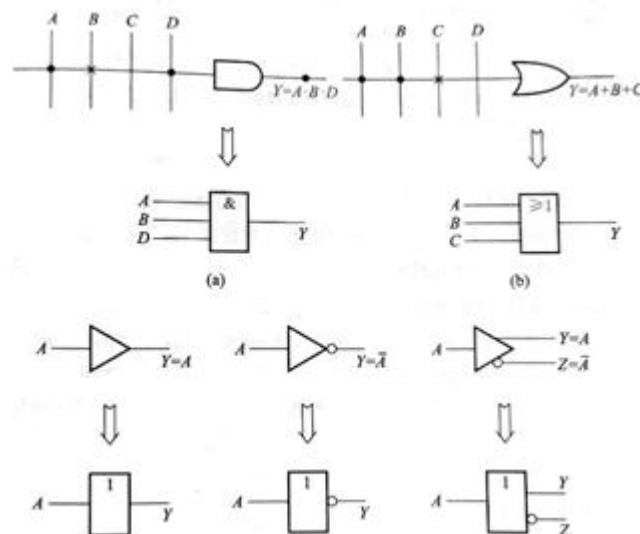
内部结构示意图

$$\text{存储容量} = \text{字线数} \times \text{位线数} = 2^n \times b \text{ (位)}$$

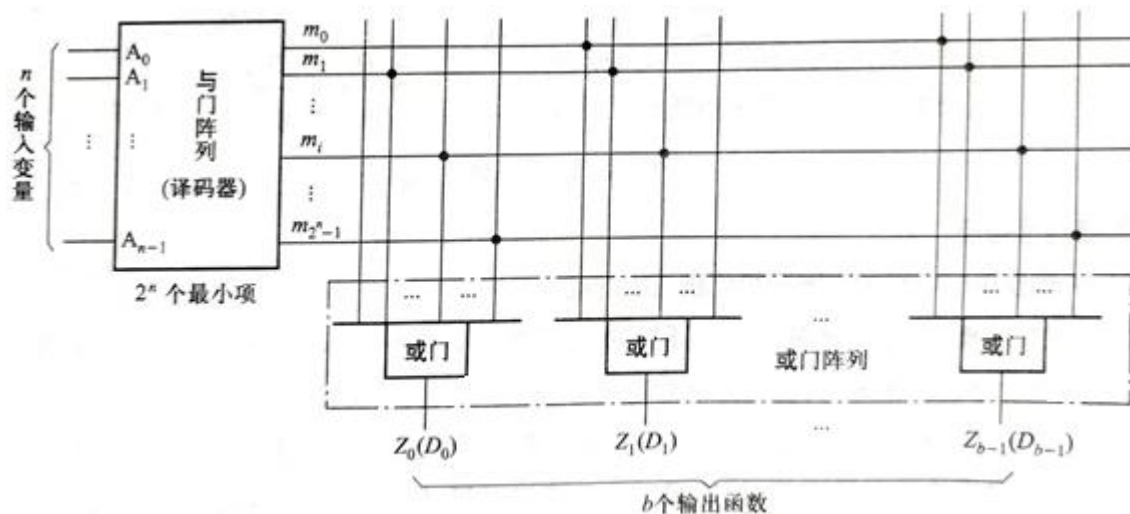


逻辑结构示意图

- 中、大规模集成电路中逻辑图简化画法的约定

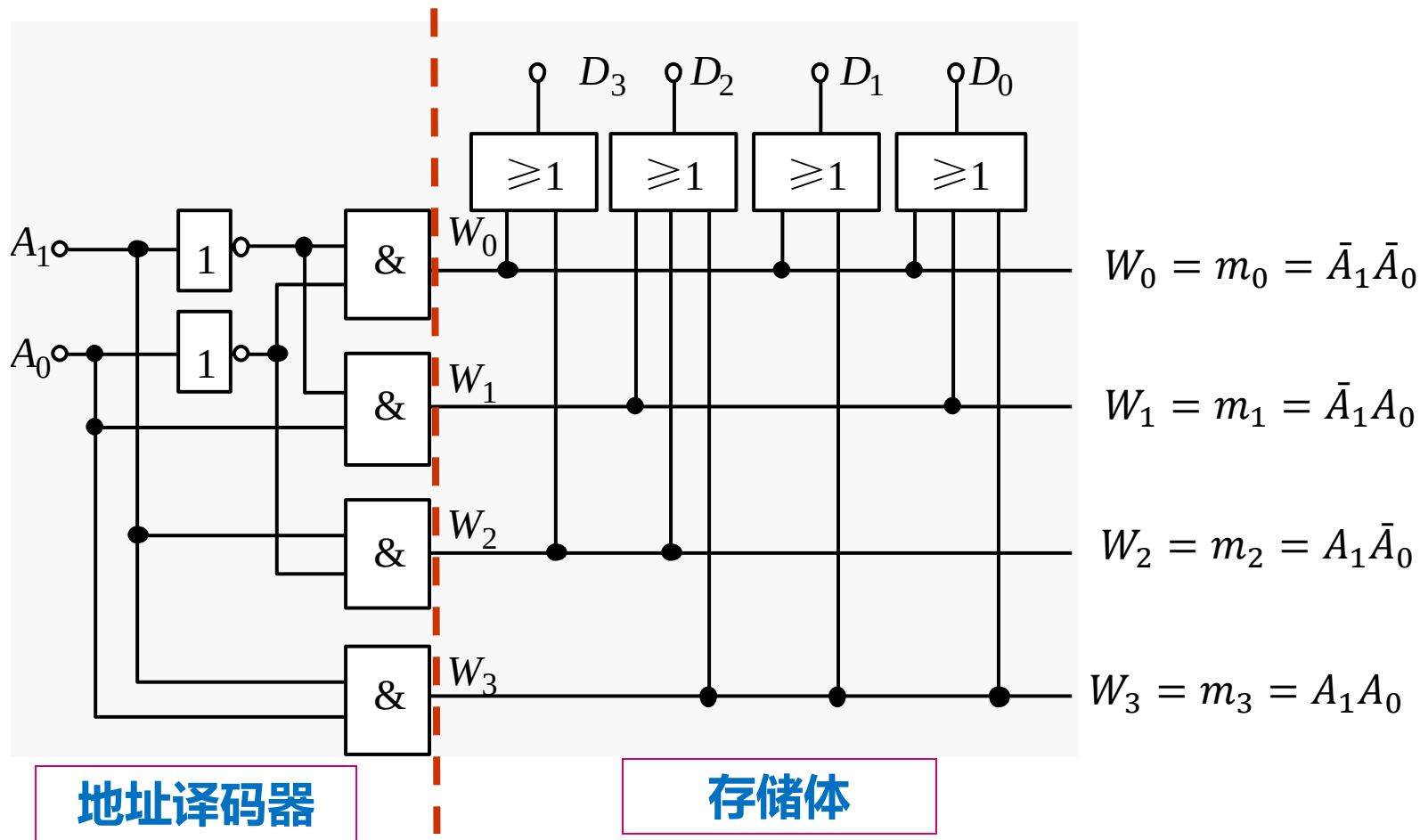


- 逻辑结构示意图



ROM的基本工作原理

4
×
4
位
R
O
M



$$D_3 = W_0 + W_2 = m_0 + m_2$$

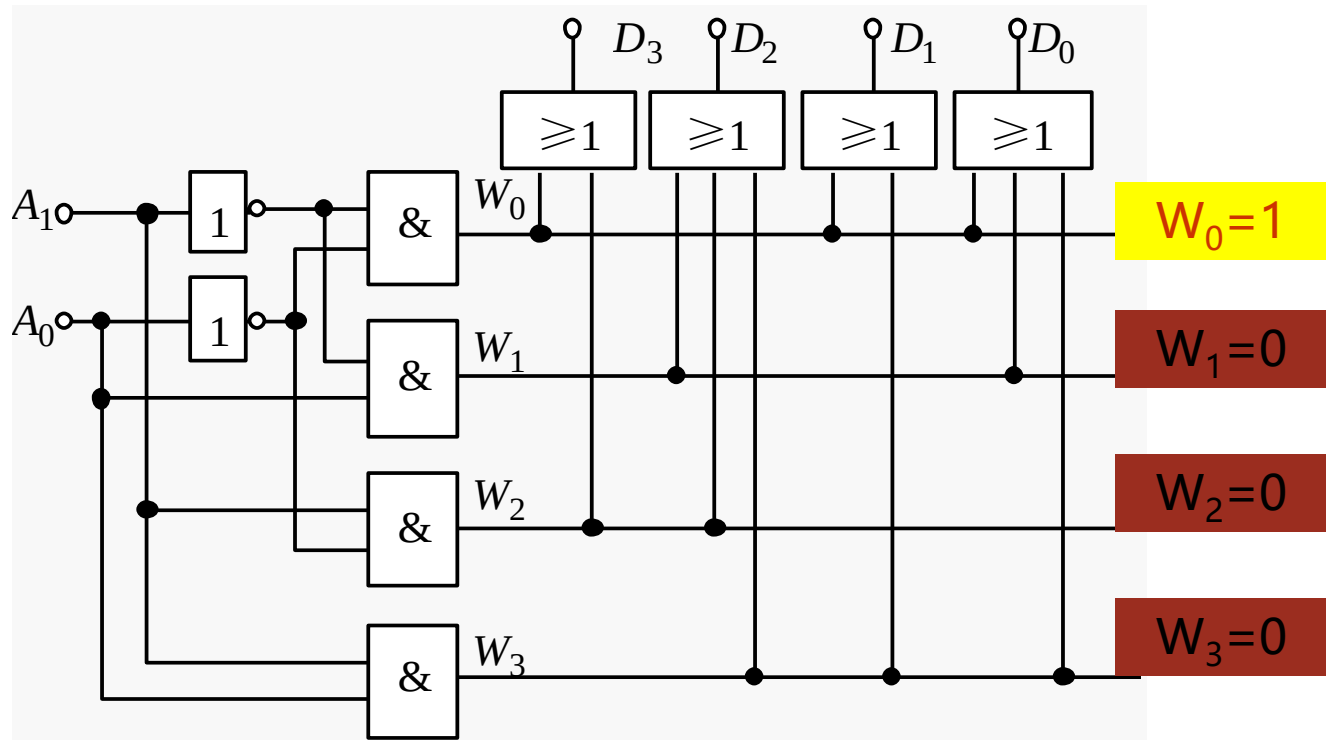
$$D_1 = W_0 + W_3 = m_0 + m_3$$

$$D_2 = W_1 + W_2 + W_3 = m_1 + m_2 + m_3$$

$$D_0 = W_0 + W_1 + W_3 = m_0 + m_1 + m_3$$

$D_3=1$
 $D_2=0$
 $D_1=1$
 $D_0=1$

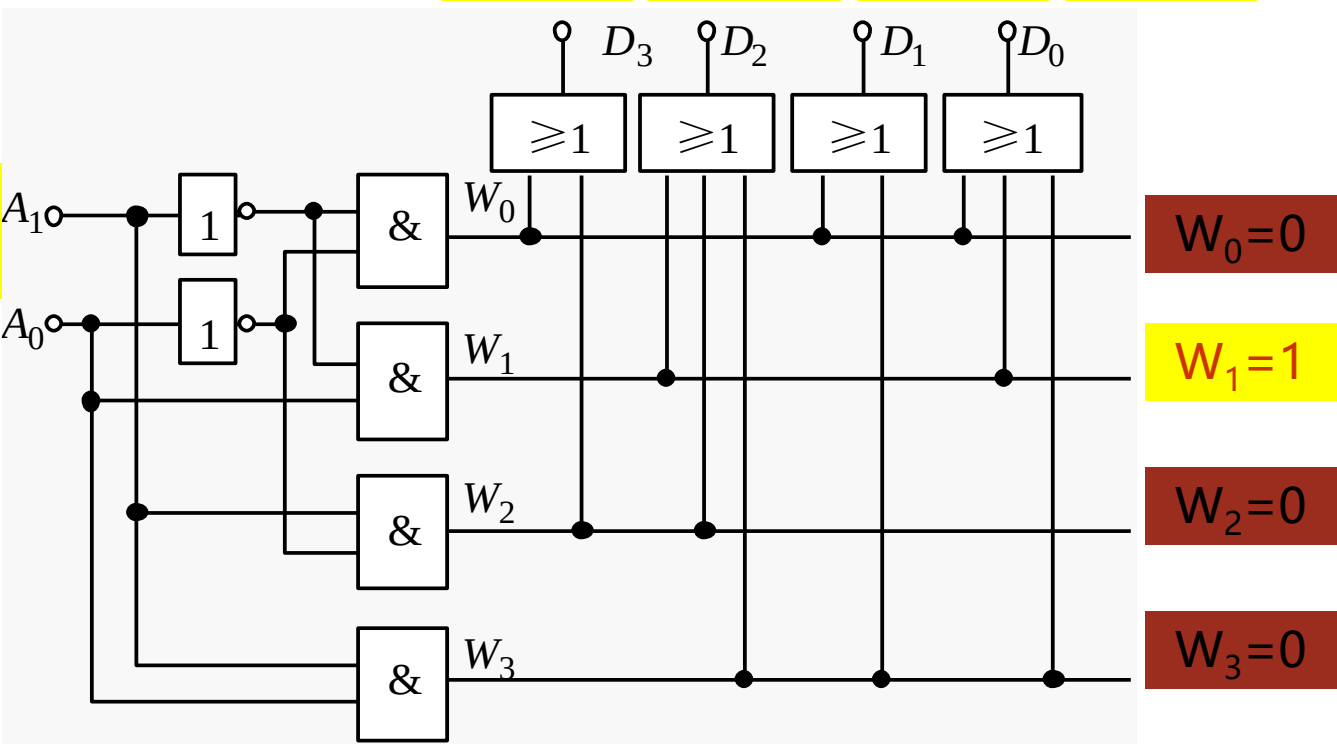
$A_1=0$
 $A_0=0$



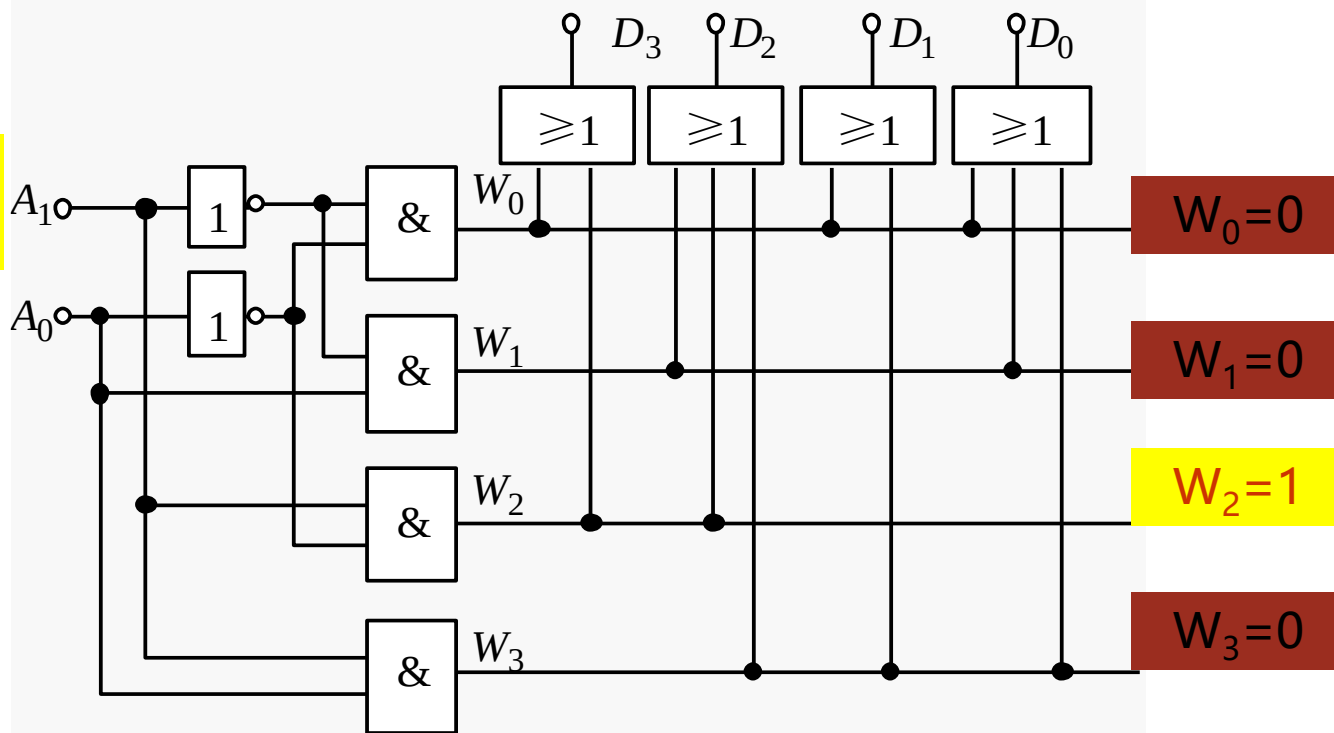
地 址		字 线				存 储 内 容			
A_1	A_0	W_0	W_1	W_2	W_3	D_3	D_2	D_1	D_0
0	0	1	0	0	0	1	0	1	1
0	1	0	1	0	0	0	1	0	1
1	0	0	0	1	0	1	1	0	0
1	1	0	0	0	1	0	1	1	1

$A_1=0$
 $A_0=1$

$D_3=0$ $D_2=1$ $D_1=0$ $D_0=1$



地 址		字 线				存 储 内 容			
A_1	A_0	W_0	W_1	W_2	W_3	D_3	D_2	D_1	D_0
0	0	1	0	0	0	1	0	1	1
0	1	0	1	0	0	0	1	0	1
1	0	0	0	1	0	1	1	0	0
1	1	0	0	0	1	0	1	1	1

$D_3=1$ $D_2=1$ $D_1=0$ $D_0=0$ $A_1=1$
 $A_0=0$ 

地 址		字 线				存 储 内 容			
A_1	A_0	W_0	W_1	W_2	W_3	D_3	D_2	D_1	D_0
0	0	1	0	0	0	1	0	1	1
0	1	0	1	0	0	0	1	0	1
1	0	0	0	1	0	1	1	0	0
1	1	0	0	0	1	0	1	1	1

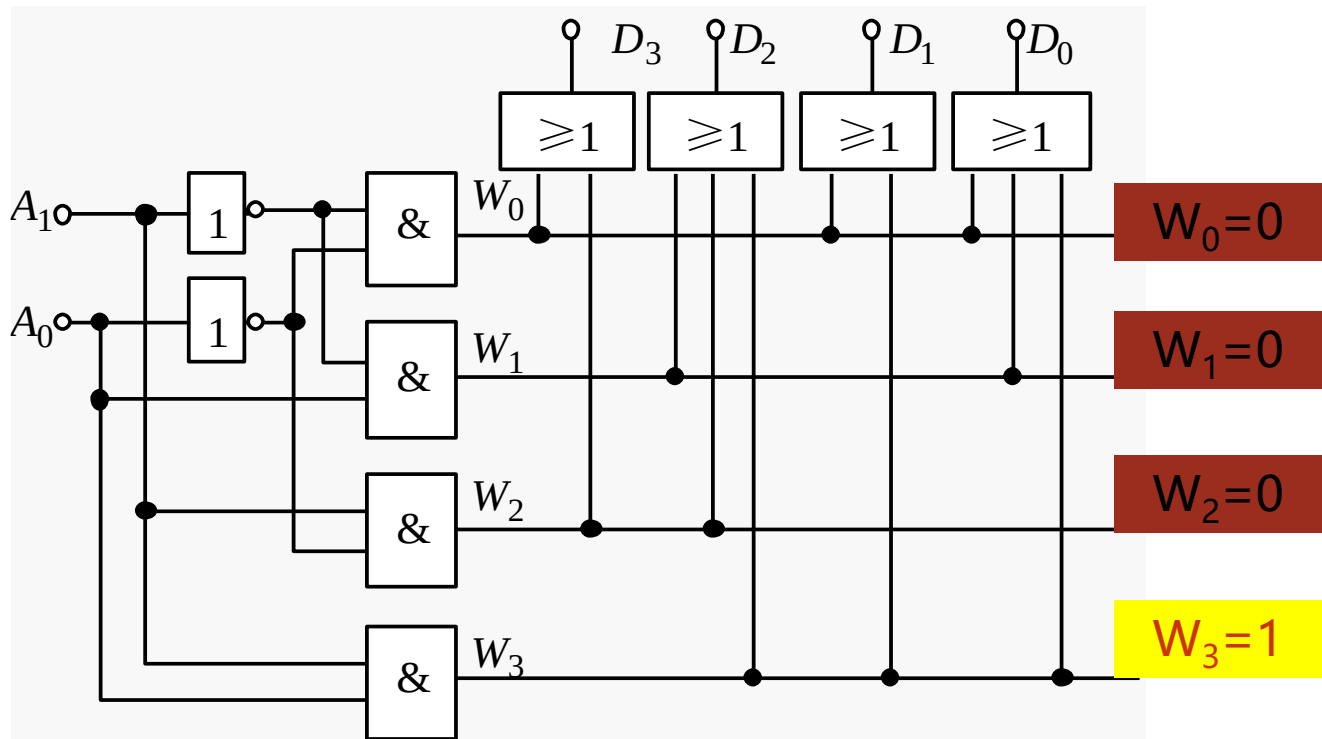
$A_1=1$
 $A_0=1$

$D_3=0$

$D_2=1$

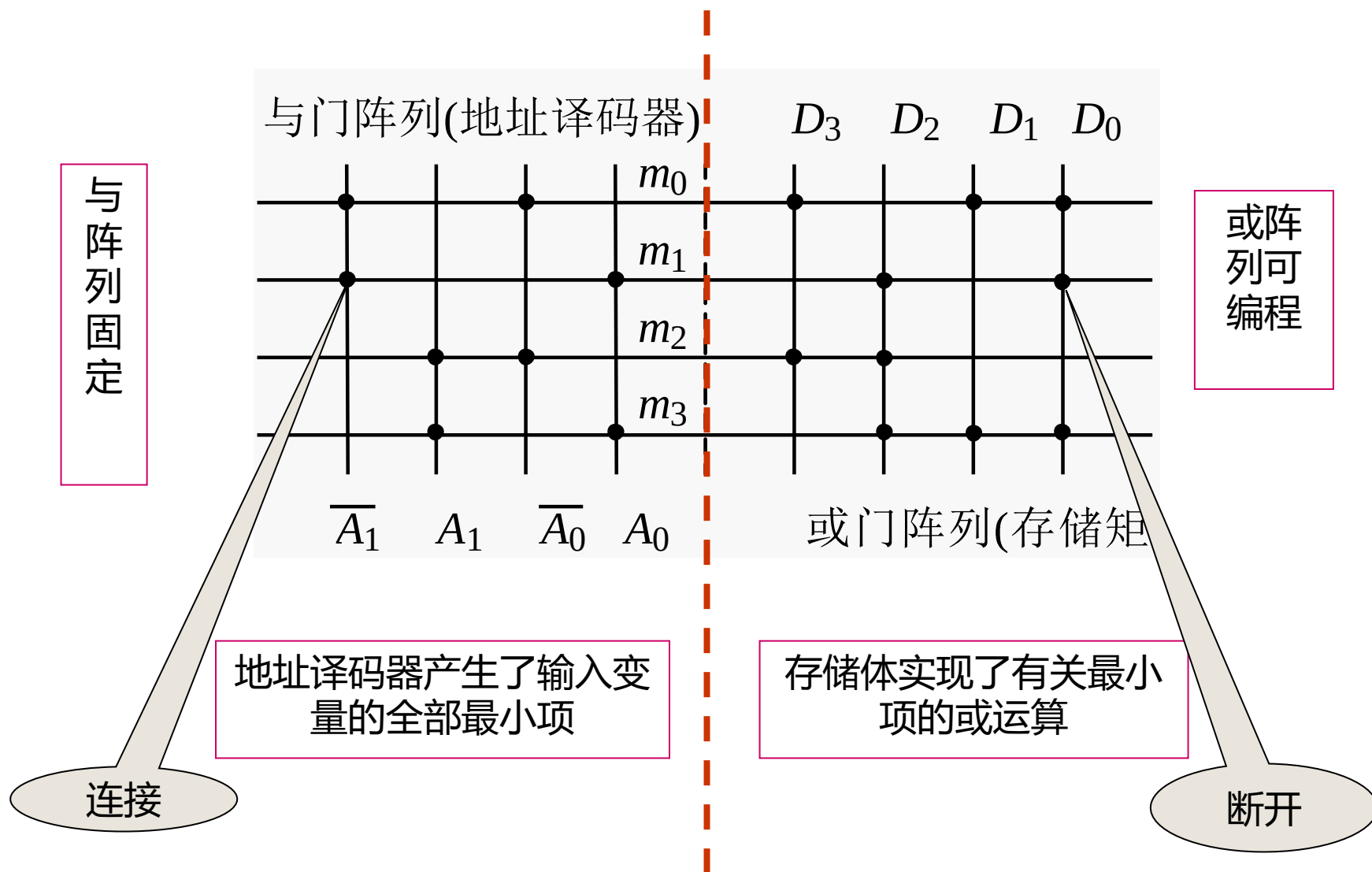
$D_1=1$

$D_0=1$



地 址		字 线				存 储 内 容			
A_1	A_0	W_0	W_1	W_2	W_3	D_3	D_2	D_1	D_0
0	0	1	0	0	0	1	0	1	1
0	1	0	1	0	0	0	1	0	1
1	0	0	0	1	0	1	1	0	0
1	1	0	0	0	1	0	1	1	1

ROM的简化画法



3.5.2 ROM的应用举例及容量扩展

例：用ROM构成能实现函数 $y = x^2$ 的运算表电路。

设 x 的取值范围为 $0 \sim 15$ 的正整数，则对应的是4位二进制正整数，用 $B = B_3B_2B_1B_0$ 表示。

根据 $y = x^2$ 可算出 y 的最大值是 $15^2 = 225$ ，可以用8位二进制数 $Y = Y_7Y_6Y_5Y_4Y_3Y_2Y_1Y_0$ 表示。

由此可列出 $Y = B^2$ 即 $y = x^2$ 的真值表。

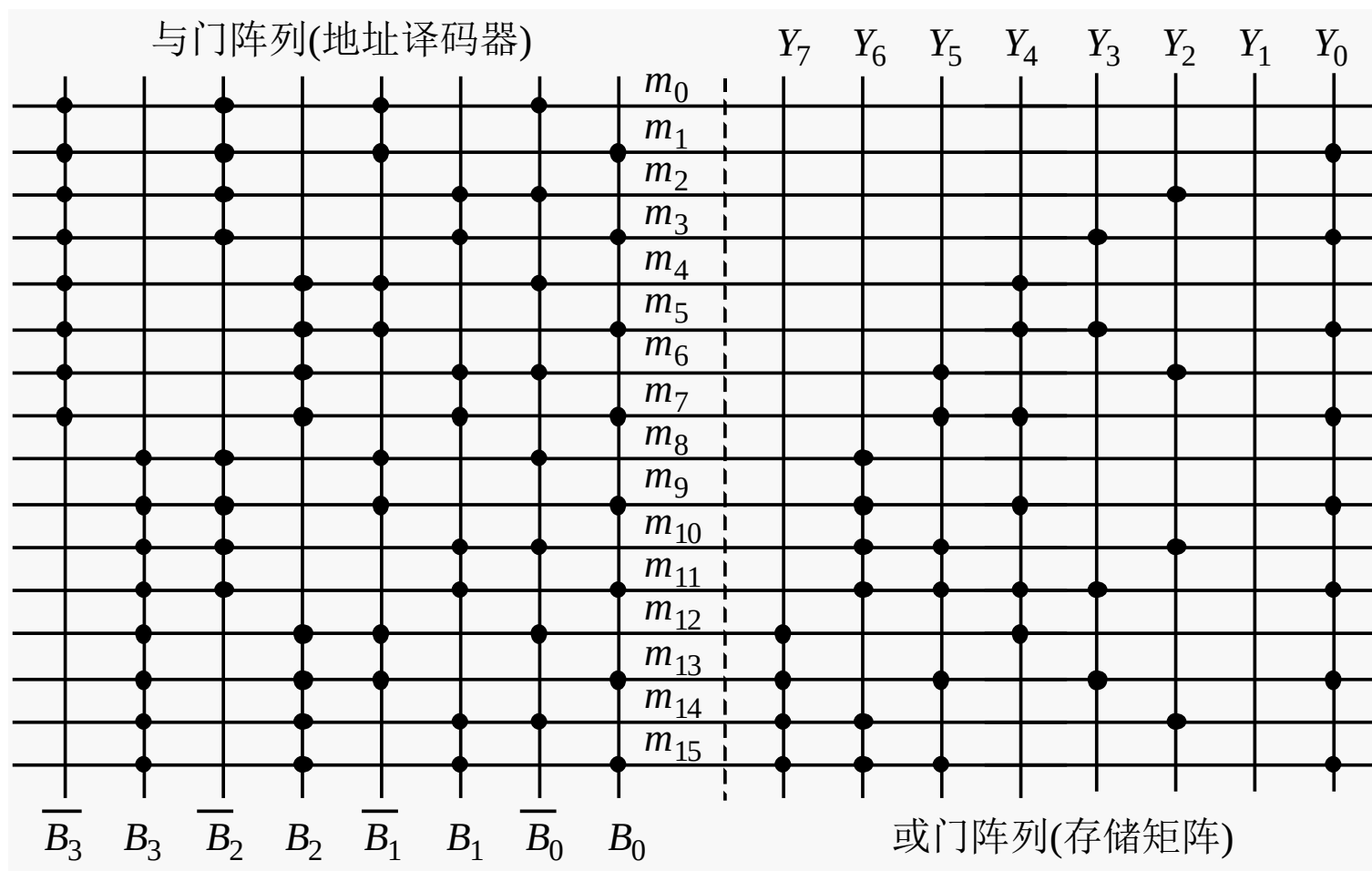
真值表

输 入				输 出								注
B_3	B_2	B_1	B_0	Y_7	Y_6	Y_5	Y_4	Y_3	Y_2	Y_1	Y_0	十进制数
0	0	0	0	0	0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0	0	0	0	1	1
0	0	1	0	0	0	0	0	0	1	0	0	4
0	0	1	1	0	0	0	0	1	0	0	1	9
0	1	0	0	0	0	0	1	0	0	0	0	16
0	1	0	1	0	0	0	1	1	0	0	1	25
0	1	1	0	0	0	1	0	0	1	0	0	36
0	1	1	1	0	0	1	1	0	0	0	1	49
1	0	0	0	0	1	0	0	0	0	0	0	64
1	0	0	1	0	1	0	1	0	0	0	1	81
1	0	1	0	0	1	1	0	0	1	0	0	100
1	0	1	1	0	1	1	1	1	0	0	1	121
1	1	0	0	1	0	0	1	0	0	0	0	144
1	1	0	1	1	0	1	0	1	0	0	1	169
1	1	1	0	1	1	0	0	0	1	0	0	196
1	1	1	1	1	1	1	0	0	0	0	1	225

逻辑表达式

$$\begin{cases}
 Y_7 = \sum m(12,13,14,15) \\
 Y_6 = \sum m(8,9,10,11,14,15) \\
 Y_5 = \sum m(6,7,10,11,13,15) \\
 Y_4 = \sum m(4,5,7,9,11,12) \\
 Y_3 = \sum m(3,5,11,13) \\
 Y_2 = \sum m(2,6,10,14) \\
 Y_1 = 0 \\
 Y_0 = \sum m(1,3,5,7,9,11,13,15)
 \end{cases}$$

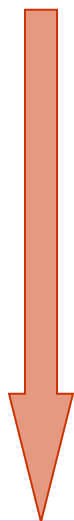
阵列图



实现任意组合逻辑函数

逻辑表达式

1



真值表或最小项
表达式

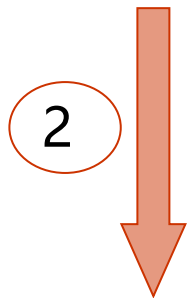
$$\begin{cases} Y_1 = A \oplus B \oplus C \\ Y_2 = AB + AC + BC \\ Y_3 = AB\bar{D} + BCD + \bar{B}\bar{C}D \\ Y_4 = \bar{A}\bar{C} + B\bar{C} + \bar{B}D + A\bar{B}C \end{cases}$$

1

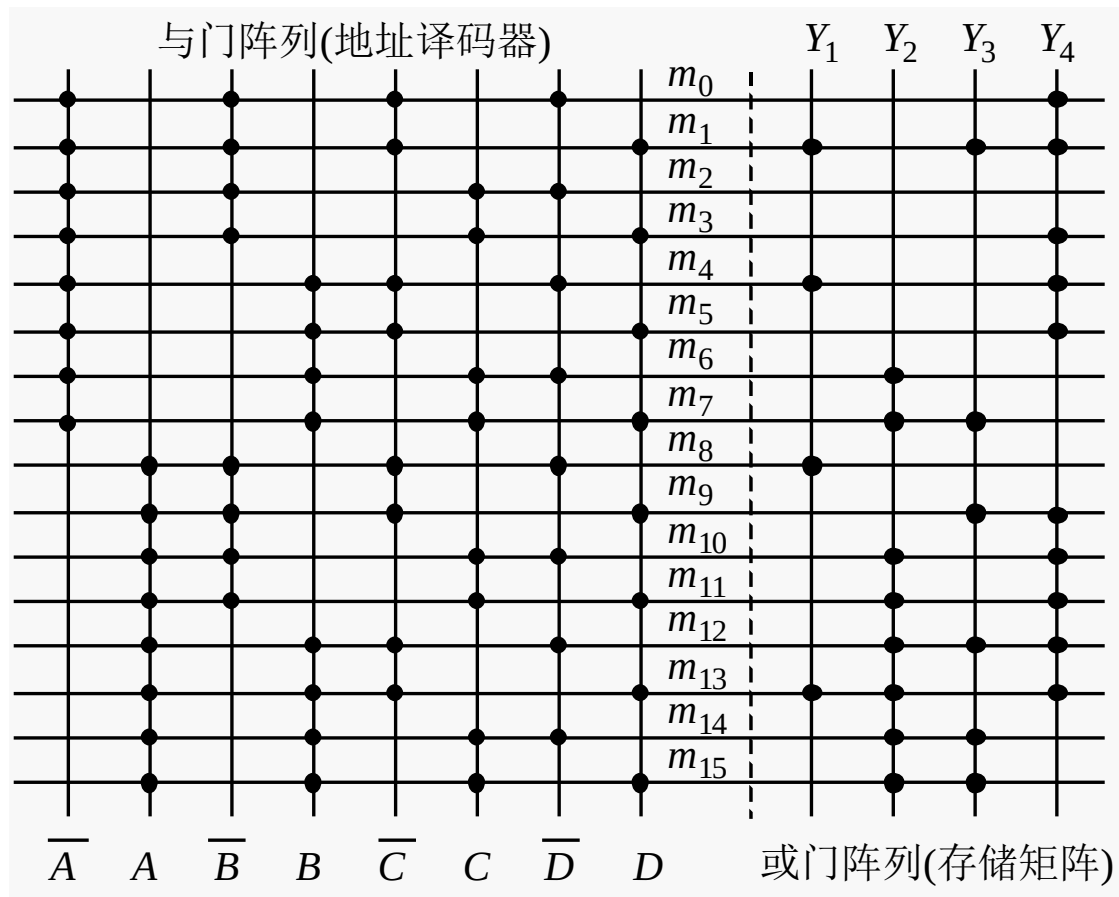
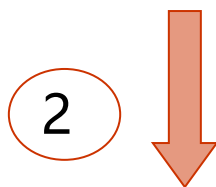


按A、B、C、D排列变量，并将 $Y_1 \sim Y_4$
扩展成为4变量的逻辑函数。

$$\begin{cases} Y_1 = \sum m(1,4,8,13) \\ Y_2 = \sum m(6,7,10,11,12,13,14,15) \\ Y_3 = \sum m(1,7,9,12,14,15) \\ Y_4 = \sum m(0,1,3,4,5,9,10,11,12,13) \end{cases}$$

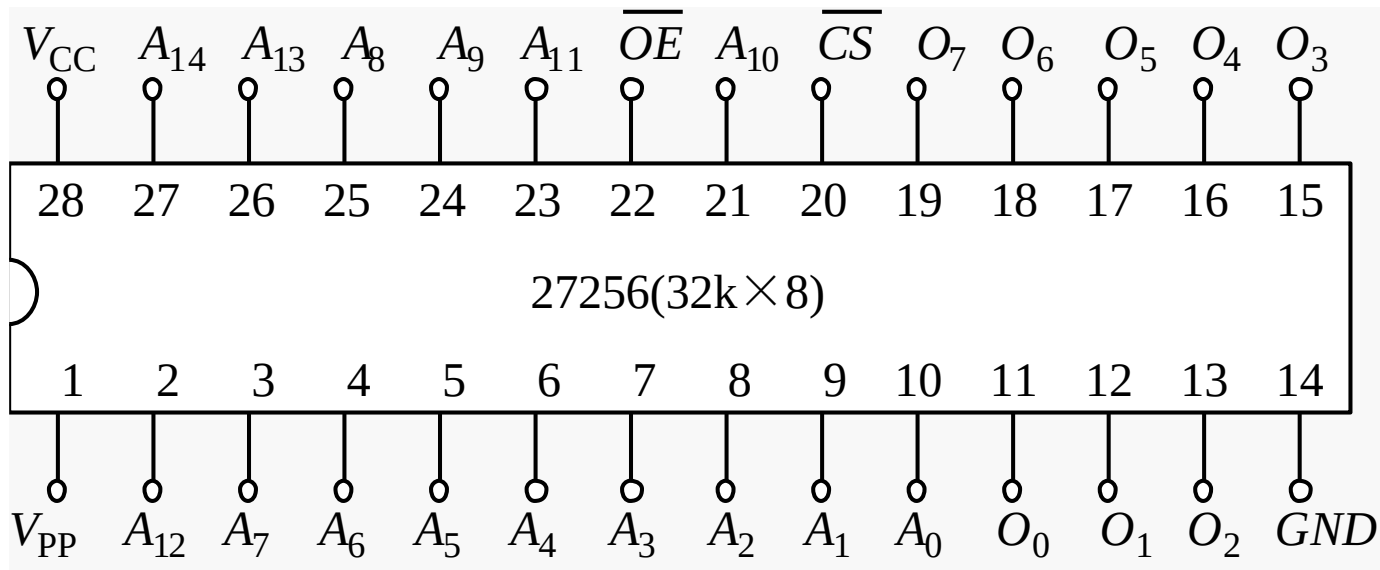


选择ROM,
画阵列图



ROM的容量扩展

EP
R
O
M
芯
片
2
7
2
5
6



正常使用时， $V_{CC}=5V$ ， $V_{PP}=5V$ 。编程时， $V_{PP}=25V$ 。

$\overline{OE}$ 为输出使能端， $\overline{OE}=0$ 时允许输出； $\overline{OE}=1$ 时，输出被禁止，ROM输出端为高阻态。

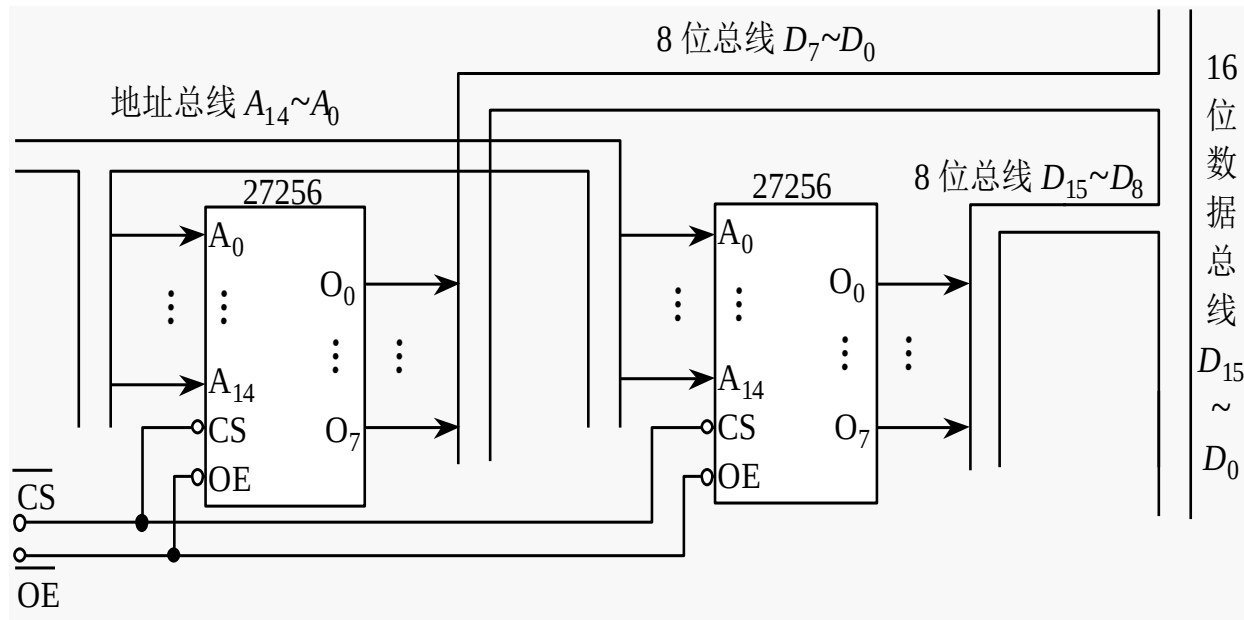
$\overline{CS}$ 为片选端， $\overline{CS}=0$ 时，ROM工作； $\overline{CS}=1$ 时，ROM停止工作，且输出为高阻态（不论 $\overline{OE}$ 为何值）。

ROM的容量扩展

- 位扩展 (字长的扩展)

用两片27256扩展成32k×16位EPROM

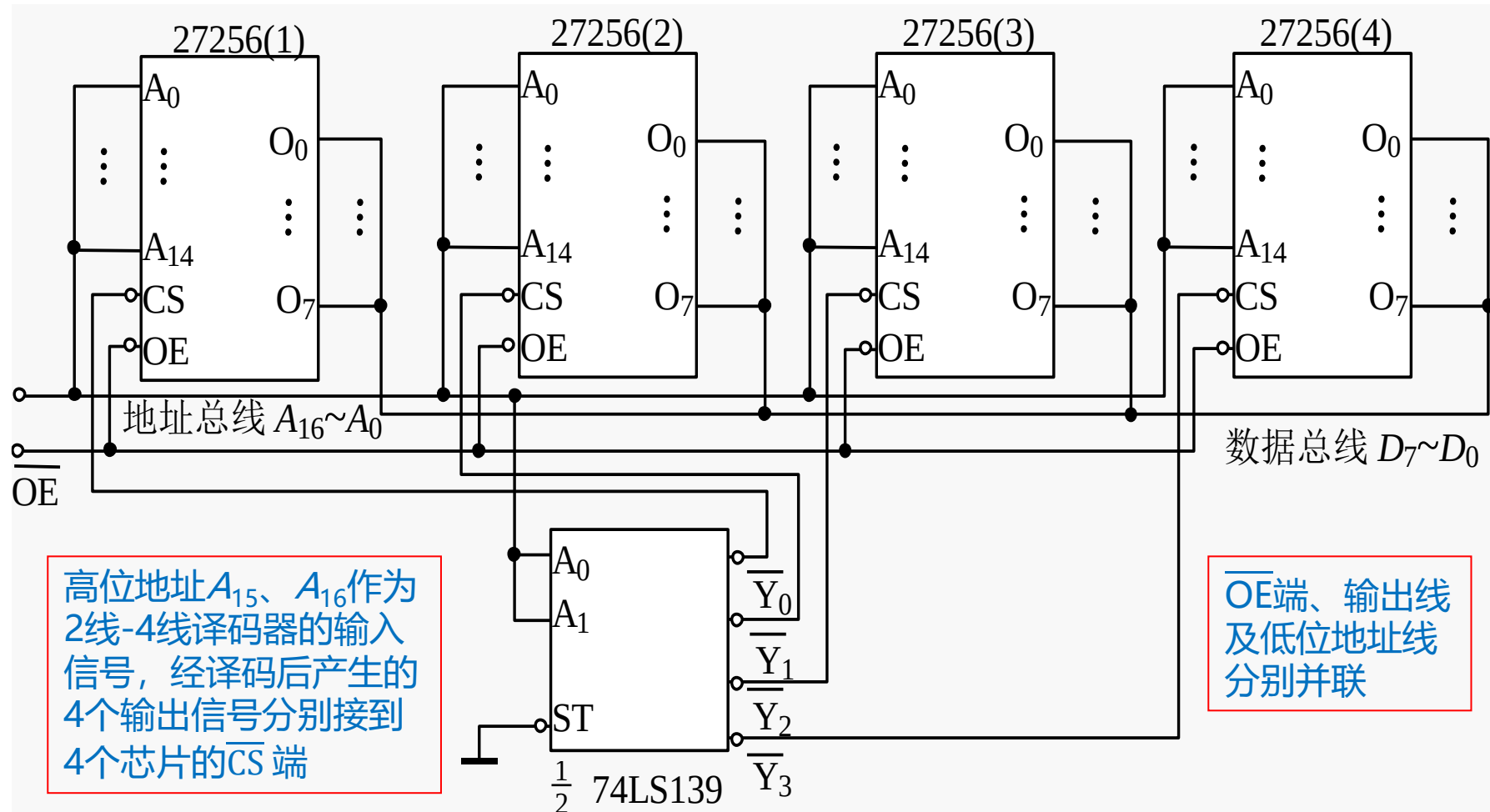
地址线
及控制
线分别
并联



输出一个作为
高8位,
另一个
作为低
8位

● 字扩展 (字数扩展, 地址码扩展)

用4片27256扩展成4×32k×8位EPROM



3.6 奇偶校验器

可靠性代码：代码传输时不易出错，或出现错误时容易被发现并进行校正，常见的有格雷码、奇偶校验码等。

1. 格雷码

格雷码：任何两个相邻代码之间只有一位码元不同，且0和最大数对应的格雷码之间也有一位不同，首尾相连。

$$G_i = B_i \oplus B_{i+1}$$

	典型4位格雷码				4位自然二进制码			
	G3	G2	G1	G0	B3	B2	B1	B0
0	0	0	0	0	0	0	0	0
1	0	0	0	1	0	0	0	1
2	0	0	1	1	0	0	1	0
3	0	0	1	0	0	0	1	1
4	0	1	1	0	0	1	0	0
5	0	1	1	1	0	1	0	1
6	0	1	0	1	0	1	1	0
7	0	1	0	0	0	1	1	1
8	1	1	0	0	1	0	0	0
9	1	1	0	1	1	0	0	1
10	1	1	1	1	1	0	1	0
11	1	1	1	0	1	0	1	1
12	1	0	1	0	1	1	0	0
13	1	0	1	1	1	1	0	1
14	1	0	0	1	1	1	1	0
15	1	0	0	0	1	1	1	1

2. 奇偶校验码

◆ **奇偶校验码(Parity Check)**: 一种校验代码传输正确性的方法, 由两部分组成:

- ① 需要传输的信息本身, n 位二进制代码。
- ② 1位奇偶校验位: 增加1位0或1使二进制代码中1的个数为奇数或偶性。
 - 奇校验: 1的个数为奇数
 - 偶校验: 1的个数为偶数

特点:

- ① 只能发现奇数个数传输错误, 没有错误定位和纠错能力, 发现错误后, 只能重传。
- ② 实现简单, 广泛应用。

十进制数	8421 奇校验码			8421 偶校验码		
	信息码	校验位	信息码	校验位	信息码	校验位
0	0 0 0 0	1	0 0 0 0	0	0 0 0 0	0
1	0 0 0 1	0	0 0 0 1	1	0 0 0 1	1
2	0 0 1 0	0	0 0 1 0	1	0 0 1 0	1
3	0 0 1 1	1	0 0 1 1	0	0 0 1 1	0
4	0 1 0 0	0	0 1 0 0	1	0 1 0 0	1
5	0 1 0 1	1	0 1 0 1	0	0 1 0 1	0
6	0 1 1 0	1	0 1 1 0	0	0 1 1 0	0
7	0 1 1 1	0	0 1 1 1	1	0 1 1 1	1
8	1 0 0 0	0	1 0 0 0	1	1 0 0 0	1
9	1 0 0 1	1	1 0 0 1	0	1 0 0 1	0

改进的奇偶校验码

双向偶校验：垂直水平两个维度校验

0110100	1
1011010	0
0010110	1
1110101	1
1001011	0
1000110	1

0110100	1
1011010	0
0010111	1
1110101	1
1001011	0
1000110	1

(可纠错)

可纠正1位错误

0110100	1
1011010	0
0110110	1
1110101	1
1001001	0
1000110	1

(可检错)

可检测出某行(列)上的奇数位

0110100	1
1011010	0
0110100	1
1110101	1
1001011	0
1000110	1

(可检错)

可检出一部分偶数位错误

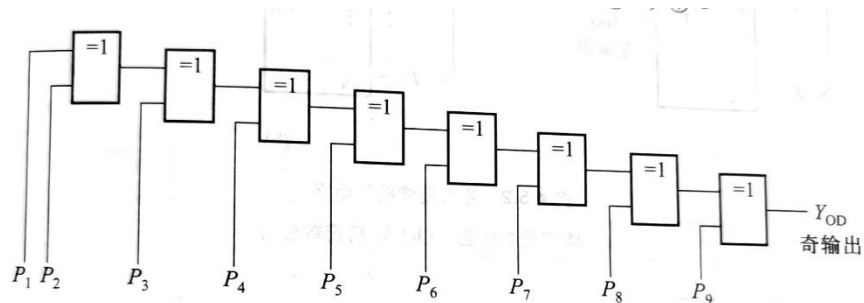
1110101	1
1011010	0
0010110	1
1110101	1
1001011	0
0000111	1

(不能检错)

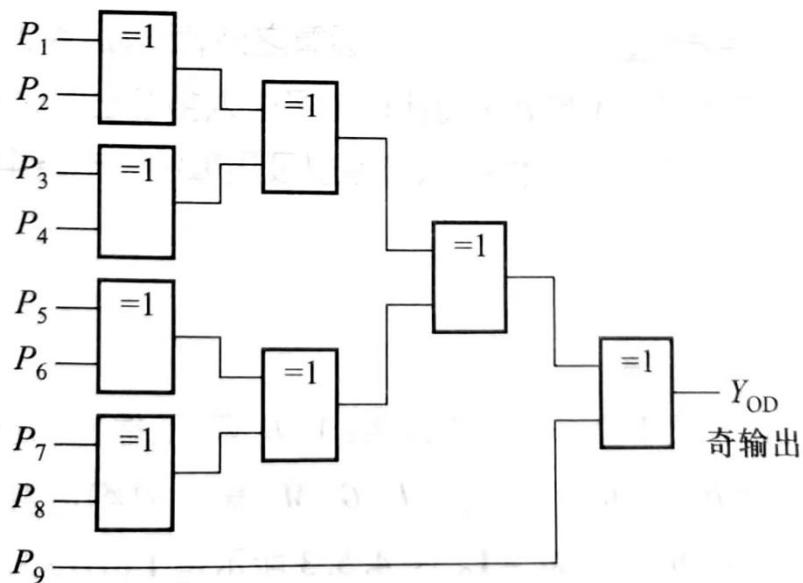
不能检测出错误码分布在矩形4个顶点上的错误

奇偶检验电路

$$Y_{OD} = P_1 \oplus P_2 \oplus P_3 \oplus P_4 \oplus P_5 \oplus P_6 \oplus P_7 \oplus P_8 \oplus P_9$$



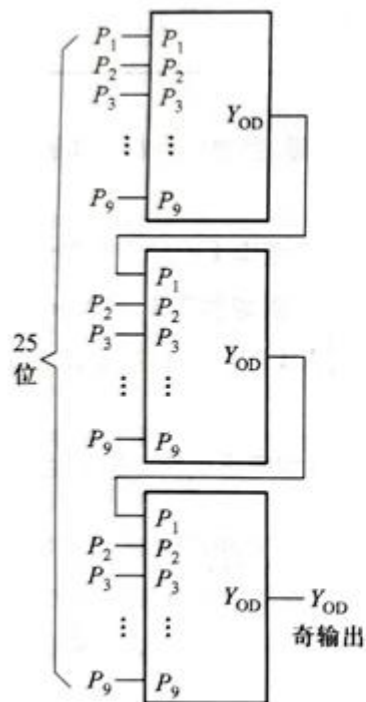
异或运算满足交换律和结合律，两种电路等价



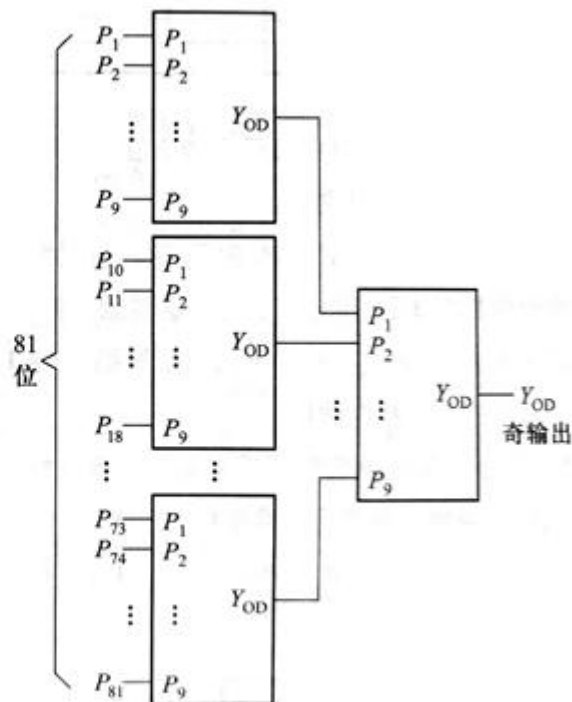
奇偶检验电路

集成电路74180和74280是常用的9位奇偶检验电路。

级联扩展有串联和并联两种方式，并联方式工作速度更快。



(a)



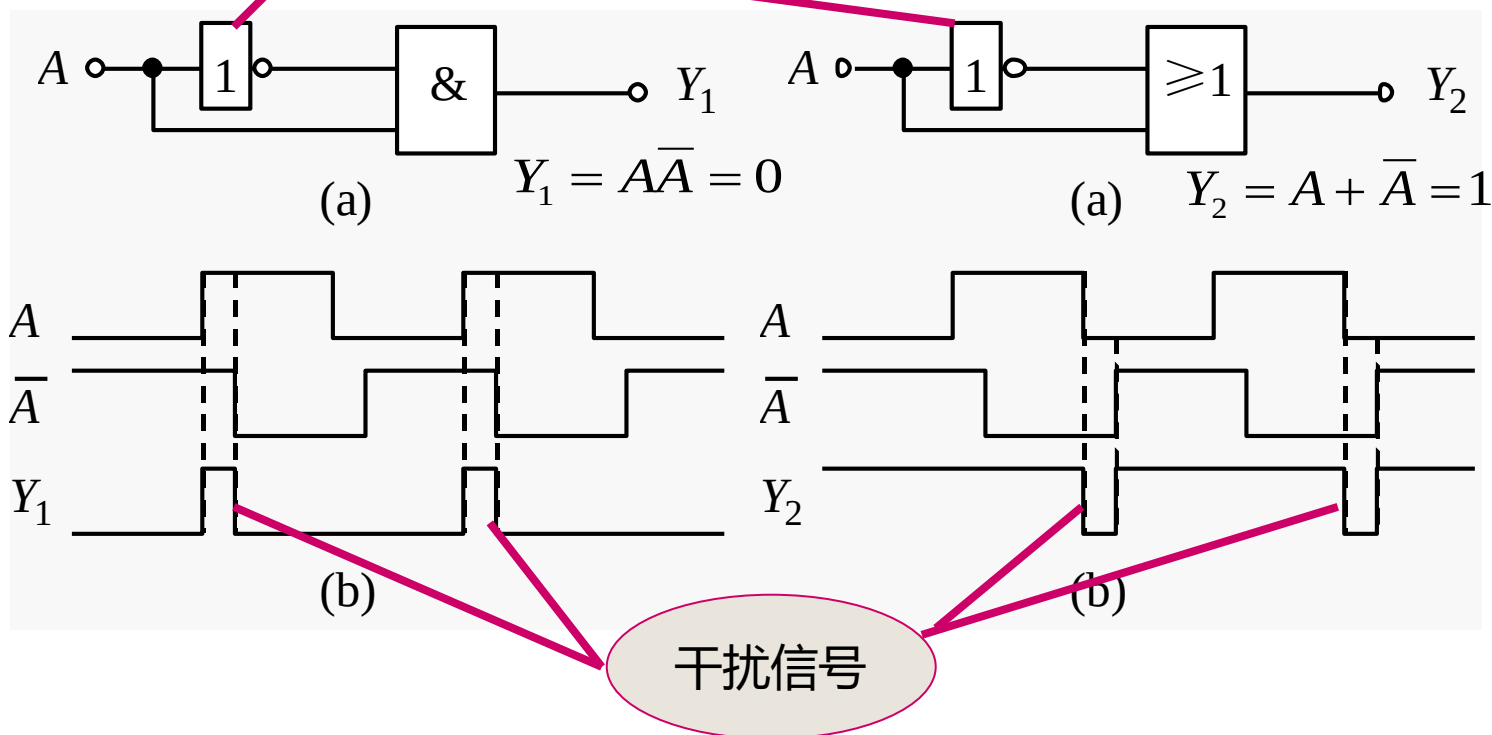
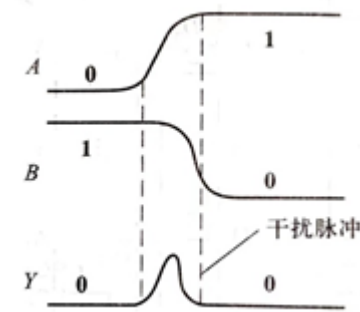
(b)

3.7 组合电路中的竞争冒险

产生竞争冒险的原因

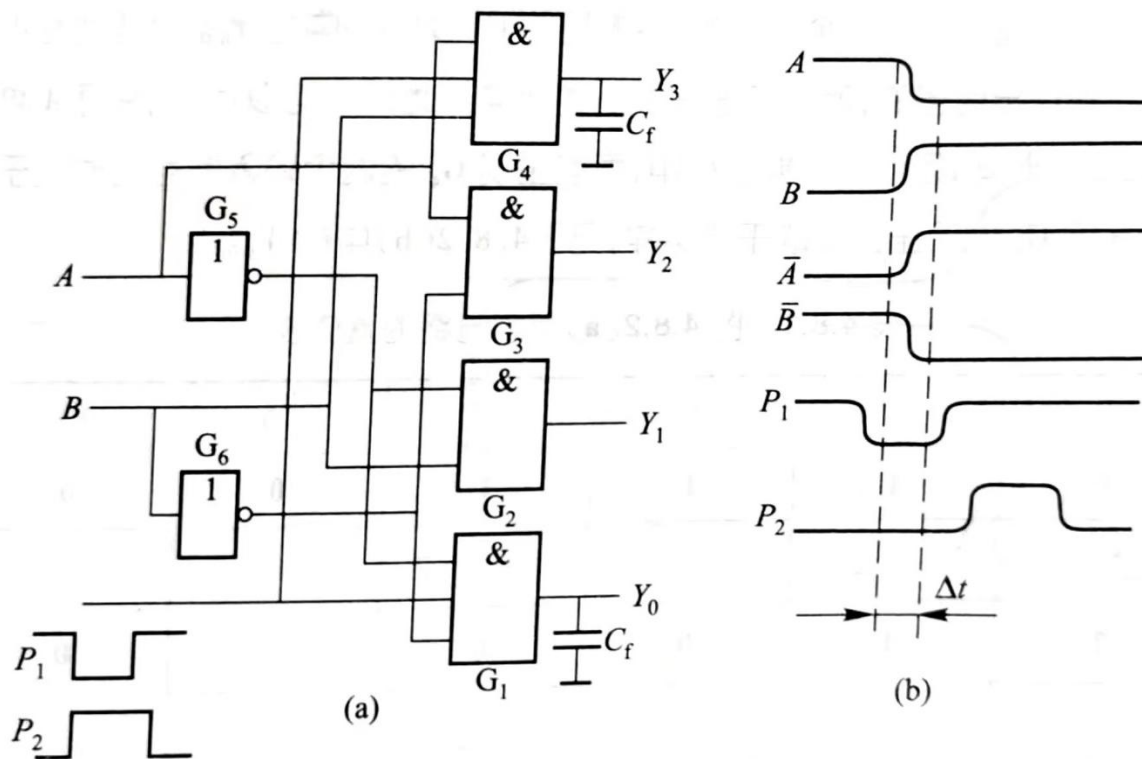
原因之一：在数字电路中，任何一个门电路只要有二个输入信号同时向相反方向变化，其输出端就可能产生干扰脉冲。

原因之二：由门电路的延迟时间产生的。

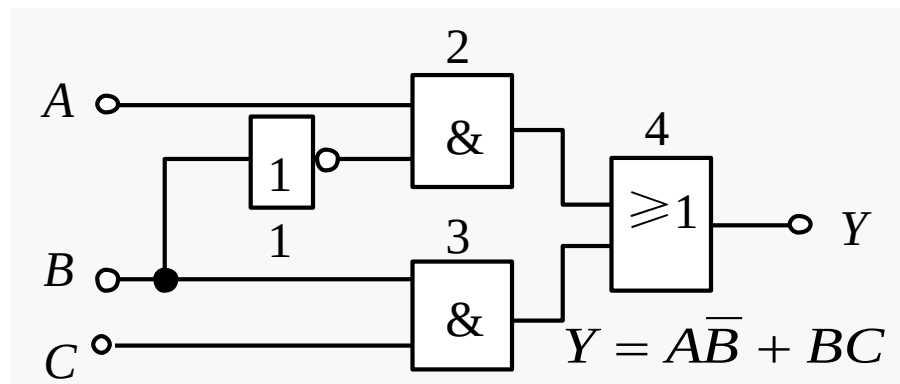


消除竞争冒险的方法:

- ① 引入封锁脉冲 (P_1) : 封锁脉冲的宽度大于过渡时间
- ② 引入选通脉冲 (P_2)
- ③ 接入滤波电容 (C_f)



④ 修改逻辑设计增加冗余项 冗余律: $A\bar{B} + BC + AC = A\bar{B} + BC$



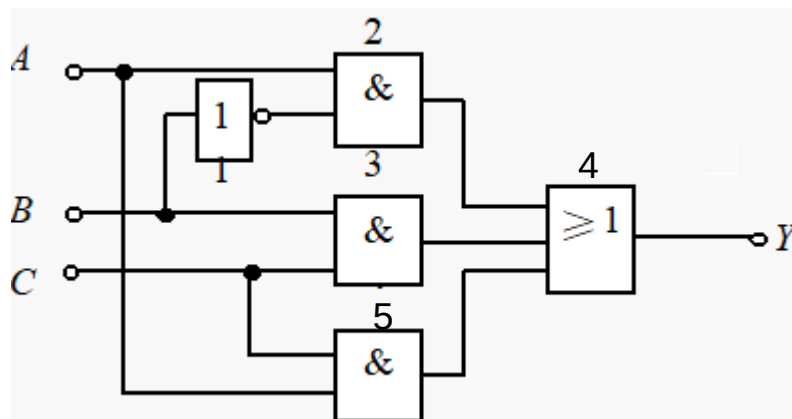
当 $A=C=1$, $Y = \bar{B} + B$, 门4有可能出现低电平的干扰脉冲。

AB					
C		00	01	11	10
	0	0	0	0	1
	1	0	1	1	1

有圈相切，则有竞争冒险

增加冗余项，消除竞争冒险

$$Y = A\bar{B} + BC + AC$$



门5输出的高电平封锁了门5，使竞争冒险不再发生。